
Státnice: rychlá příprava na trojku

Informatika - Umělá inteligence

zaměření **Strojové učení**

Lean studijní text optimalizovaný na **nejméně času k jistému průchodu** (známka 3, >98 %).
Strukturováno přesně podle oficiálních požadavků (požadavky/detailni-pozadavky.pdf).

Jak tento text funguje

- Každé téma má dvě části: šedý box **TÉMA** = **oficiální požadavek doslova** (co se může zkoušet), a pod ním *Učivo* = naše stručné, intuitivní vysvětlení.
- **Odpověď podle bodů** ukazuje, co stačí říct na 1 / 2 / 3 body. **Cíl pro průchod je zelená = 2 body.**
- **Vyzkoušej se** = otázky na aktivní vybavování; **Pozor** = častá past.
- Obrázky jsou tam, kde ušetří čas. Text je **přepsaný pro pochopení**, ne opsaný z poznámek.

Obsah

1	Co a jak hodně se učit (z rozboru 20 zkoušek 2019-2026)	1
2	Strategie: jak projít s minimem času	1
2.1	Rubrika zvládnutí (sebehodnocení)	2
3	Rozsah: co se učit a co ne	2
4	Část I: Společná matematika	3
4.1	Základy diferenciálního a integrálního počtu	3
4.2	Algebra a lineární algebra	4
4.3	Diskrétní matematika	6
4.4	Teorie grafů	8
4.5	Pravděpodobnost a statistika	10
4.6	Logika	11
5	Část II: Společná informatika	14
5.1	Automaty a jazyky	14
5.2	Algoritmy a datové struktury	17
5.3	Programovací jazyky	21
5.4	Architektura počítačů a operačních systémů	26
6	Část III: Specializace - Základy umělé inteligence	31
6.1	Řešení úloh prohledáváním	31
6.2	Splňování omezujících podmínek (CSP)	33
6.3	Logické uvažování	34
6.4	Pravděpodobnostní uvažování	35
6.5	Reprezentace znalostí	37
6.6	Automatické plánování	39
6.7	Markovské rozhodovací procesy (MDP)	41
6.8	Hry a teorie her	42
6.9	Strojové učení (přehledově)	44
7	Část IV: Specializace - Strojové učení	47
7.1	Učení s učitelem (základní pojmy, evaluace, regularizace)	47
7.2	Učení založené na příkladech - k nejbližších sousedů (kNN)	49
7.3	Lineární regrese	50
7.4	Logistická regrese	51

7.5	Rozhodovací stromy	52
7.6	Metoda podpůrných vektorů (SVM)	53
7.7	Kombinace více modelů (ensemble)	55
7.8	Statistické testy	56
7.9	Učení bez učitele (shlukování, PCA)	57
8	Vzorové výpočty a kódové vzory (drily na 2. a 3. bod)	59
8.1	Matematika: typové výpočty	59
8.2	Pravděpodobnost a statistika: typové výpočty	59
8.3	Strojové učení a UI: typové výpočty	59
8.4	Informatika: kódové vzory (C#)	60
9	Tahák: kostry odpovědí na 1 bod (sít proti nulám)	61

1 Co a jak hodně se učit (z rozboru 20 zkoušek 2019-2026)

Hlavní zjištění z reálných zadání

Tohle je výtah z rozboru 20 minulých zadání (zkousky/, audit v COMPLETENESS-EXAMS.md). Říká, kam dát čas, aby na jistou trojku stačilo co nejméně učení.

- **Anatomie otázky (podle vzorových řešení):** každá otázka má 3 podotázky a body se dávají po částech. (1) *přesná definice / znění věty / algoritmu*, (2) konstrukce, pseudokód nebo malý výpočet, (3) okrajový případ, složitost nebo aplikace. **Vždy si zajisti bod (1)**, je nejjistější a nejlevnější.
- **Definice se zkoušejí nejvíc** (slovo „Definujte“ je v zadáních 68×). Uměj *přesné* definice z paměti; to je největší výnos za nejmín času. Proto je v každém tématu **1** přímo přesná definice.
- **Specializace (UI-SU) je konceptuální.** Vzorová řešení říkají „popište“, „vysvětlete“, „*ne-musíte nic počítat*“, „zakreslete přibližně“. Uč se *principy* a uměj je vysvětlit a načrtnout; jen pár míst chce malý výpočet (jedna aktualizace EM, metriky z matice záměn). Sem dej **většinu času** (potřebuješ 7/12).
- **Společná matematika a informatika: definice + jeden výpočet.** Vzorová řešení mají i plné výpočty (integrál, diagonalizace, sestavení automatu, kód), ale stačí 5/12, takže jeď na šířku: definice u všech témat + natrénuj jeden typový výpočet u nejčastějších.
- **Kolik detailu:** úroveň „Nástinu řešení“ v zadáních = přesná definice + hlavní postup/algoritmus + jeden příklad. **Žádné dlouhé důkazy.**

Nejčastější (= nejvýnosnější) okruhy

- **Spol. informatika:** automaty (regulární, CFG/PDA, Chomského hierarchie), grafové algoritmy (BFS/DFS, topologické třídění, kostry, toky), haldy a vyhledávací stromy, třídění; OS (paměť, virtuální paměť, synchronizace, zařízení, reprezentace dat).
- **Spol. matematika:** lineární algebra, jednorozměrná analýza, základy grafů, pravděpodobnost a teorie informace, definice ve výrokové/predikátové logice.
- **Specializace UI-SU:** evaluační metriky, regrese, shlukování, rozhodovací stromy, ensemble, SVM/PCA, Bayesovské sítě/HMM/EM, CSP a prohledávání, hry/MDP.

Pozn. k formátu: zadání 2019-2022 jsou starší (specializace Strojové učení ještě neexistovala), proto z nich platí jen společné okruhy. Od 2023 je dělení čisté: 2 matematika + 2 informatika + 4 specializace (UI-SU).

2 Strategie: jak projít s minimem času

Bodový rozpočet (proč to stačí)

8 otázek po 0-3 bodech. **Průchod (známka 3)** = aspoň 12 bodů celkem a aspoň 5 ze společných okruhů (4 otázky) a aspoň 7 ze specializace (4 otázky).

- **Specializace je klíčová** (potřebuješ 7/12, tj. průměr 1,75 na otázku). Sem dej většinu času (ML + Základy UI $\approx 60\%$). Společné okruhy mají nízký floor 5/12 ($\approx 1,25$ na otázku) - stačí breadth, ne hloubka.
- **Hlavní riziko je nula z přidělené otázky**, ne chybějící důkaz. Proto měj u *každého* tématu aspoň „kostru na 1 bod“. Žádné nuly $\Rightarrow$ pravděpodobnost průchodu jde nad 98 %.
- **Trénuj nad hranici** (cíl $\sim 16/24$), protože $5+7=12$ nemá rezervu.
- **Vynech** (bezpečně): dlouhé důkazy, plná odvození, jazykové varianty mimo zvolený jazyk, vše mimo rozsah. Detaily viz sekce 3.

2.1 Rubrika zvládnutí (sebehodnocení)

stav	co umíš
● červená (0)	ani definici
● žlutá (1)	definice + pár pojmů
● zelená (2)	definice + algoritmus/věta + příklad + intuice (cíl)
● modrá (3)	zelená + přesné podmínky/detail (jen u nejdůležitějších)

Cíl: každé téma aspoň žlutě; specializace ~85 % zeleně; nejdůležitější modely modře. Podrobná strategie a rozpočet času: `STRATEGY.md` v repozitáři.

3 Rozsah: co se učit a co ne

Závazný zdroj: `pozadavky/detailni-pozadavky.pdf`. Pro zaměření **Strojové učení** se zkouší: společná matematika (6 témat), společná informatika (4 témata), a ze specializace *Základy UI* (téma 1) a *Strojové učení* (téma 3).

MEZERA V POZNÁMKÁCH - OVĚŘ / DOPLŇ

Záměrně NEzahrnuto (nezkouší se, neuč se): Robotika a NLP; neuronové sítě a hluboké učení (perceptron, MLP, CNN, word2vec patří k NLP); počítačové sítě, databáze a SQL, grafika, softwarové inženýrství, RSA/FFT a pokročilé algoritmy, výpočetní geometrie.

4 Část I: Společná matematika

Nižší priorita: ze společných okruhů (matematika + informatika) stačí 5/12. Cíl je **breadth, ne hloubka** - u každého tématu umět definice a hlavní pojmy (žlutá/zelená), aby z žádné přidělené otázky nebyla nula. Dlouhé důkazy nejsou potřeba.

4.1 Základy diferenciálního a integrálního počtu

priorita: střední cíl: žlutá-zelená (1-2 body)

Posloupnosti a jejich limity (aritmetika limit, věta o dvou polícajtech); řady (geometrická, harmonická); limita funkce a spojitost (nabývání mezihodnot a maxima); derivace (l'Hospital, průběh funkce: extrémy, monotonie, konvexita; Taylorův polynom); integrály (primitivní funkce, Riemannův/Newtonův integrál; aplikace: odhady součtů řad, obsahy, objemy a povrchy rotačních útvarů, délka křivky).

Učivo (jak na to):

Limita = kam se hodnoty blíží. *Derivace* = okamžitá rychlost změny (sklon tečny). *Integrál* = orientovaná plocha pod grafem. Derivace a integrál jsou navzájem inverzní (základní věta analýzy).

Odpověď podle bodů (cíle pro průchod: zelená = 2 body)

1 Definice: Posloupnost s hodnotami v M je zobrazení $\mathbb{N} \rightarrow M$. Pro reálnou posloupnost (a_n) a $L \in \mathbb{R}^*$ píšeme $a_n \rightarrow L$, právě když pro každé okolí cíle jsou všechny členy od jistého indexu uvnitř; pro vlastní $L \in \mathbb{R}$ tedy $\forall \varepsilon > 0 \exists n_0 \forall n \geq n_0 : |a_n - L| < \varepsilon$. Vlastní limita znamená konvergenci, jinak posloupnost diverguje; limita je nejvýše jedna. Řada $\sum_{n=1}^{\infty} a_n$ je dána částečnými součty $s_n = \sum_{k=1}^n a_k$; konverguje, pokud (s_n) má vlastní limitu, a tato limita je součet řady. Okolí $a \in \mathbb{R}$ je $U(a, \delta) = (a - \delta, a + \delta)$, prstencové okolí je bez bodu a ; pro $\pm\infty$ se používají intervaly typu $(1/\delta, +\infty)$. Limita funkce $\lim_{x \rightarrow a} f(x) = A$ znamená, že pro každé okolí A existuje prstencové okolí a , z něhož všechny hodnoty $f(x)$ leží v okolí A . Jednostranné limity se definují pravým nebo levým prstencovým okolím; oboustranná limita existuje právě tehdy, když existují a shodují se obě jednostranné limity. Spojitost v bodě je $\lim_{x \rightarrow a} f(x) = f(a)$; spojitost na intervalu znamená spojitost ve vnitřních bodech a příslušnou jednostrannou spojitost v krajních bodech. Derivace v bodě b je $f'(b) = \lim_{h \rightarrow 0} \frac{f(b+h)-f(b)}{h} = \lim_{x \rightarrow b} \frac{f(x)-f(b)}{x-b}$; vlastní derivace znamená diferencovatelnost. Primitivní funkce F k f na (a, b) splňuje $F' = f$ a je určena až na konstantu.

2 Věty, pravidla a algoritmy: Aritmetika limit posloupností i funkcí: z limit a, b plynou limity součtu, součinu a podílu, pokud je pravá strana definována a jmenovatel je od jistého místa nenulový. Neurčité výrazy jsou zejména součet nekonečen různých znamének, $0 \cdot \infty$, podíl nekonečen a podíl $0/0$. Je-li (a_n) omezená a $b_n \rightarrow 0$, pak $a_n b_n \rightarrow 0$ i bez existence $\lim a_n$. Uspořádání: z $\lim a_n = a < b = \lim b_n$ plyne od jistého indexu $a_n < b_n$; naopak $a_n \leq b_n$ od jistého indexu dává $\lim a_n \leq \lim b_n$. Stejně pro funkce v prstencovém okolí. Věta o dvou polícajtech: z $a_n \leq c_n \leq b_n$ a $a_n, b_n \rightarrow A \in \mathbb{R}$ plyne $c_n \rightarrow A$; pro nevlastní limity stačí jednostranné sevření. Pro funkce platí analogie $f \leq h \leq g$ a stejné limity f, g . Limita složené funkce: jestliže $g(x) \rightarrow B$ pro $x \rightarrow A$, $f(y) \rightarrow C$ pro $y \rightarrow B$ a buď je f spojitá v B , nebo g v prstencovém okolí A nenabývá hodnotu B , pak $f(g(x)) \rightarrow C$. Věta o mezihodnotách: spojitá $f : [a, b] \rightarrow \mathbb{R}$ nabývá každé hodnoty mezi $f(a)$ a $f(b)$. Weierstrassova věta: spojitá funkce na $[a, b]$ nabývá maxima i minima. l'Hospitalovo pravidlo: pro $a \in \mathbb{R}^*$, vlastní derivace f, g na $P(a, \delta)$ a $g' \neq 0$, pokud $f, g \rightarrow 0$ nebo $|g| \rightarrow +\infty$ a $f'/g' \rightarrow A \in \mathbb{R}^*$, pak $f/g \rightarrow A$.

Výšetření průběhu: lokální extrém ve vnitřním bodě může nastat jen při $f' = 0$ nebo neexistenci derivace; $f' > 0, \geq 0, < 0, \leq 0, = 0$ na vnitřku intervalu dává rostoucí, neklesající, klesající, nerostoucí, konstantní funkci. Podle druhé derivace je funkce ryze konvexní pro $f'' > 0$, konvexní pro $f'' \geq 0$, ryze konkávní pro $f'' < 0$ a konkávní pro $f'' \leq 0$. Newtonův integrál je $(N) \int_a^b f = [F]_a^b = F(b^-) - F(a^+)$. Per partes: $\int u'v = uv - \int uv'$. Riemannův integrál: pro dělení $D = (a_0, \dots, a_k)$, $I_i = [a_i, a_{i+1}]$, $s(f, D) = \sum |I_i| m_i$, $S(f, D) = \sum |I_i| M_i$; dolní integrál je supremum dolních sum, horní integrál infimum horních sum a integrál existuje, když se rovnají. Pokud existují Riemannův i Newtonův integrál, jsou si rovny. Integrovní kritérium: pro nezápornou nerostoucí f konverguje $\sum_{k=1}^{\infty} f(k)$ právě tehdy, když $\lim_{b \rightarrow +\infty} \int_1^b f < +\infty$.

3 Vzorce, případy a aplikace: Geometrická řada $\sum_{n=0}^{\infty} q^n$ má částečný součet $s_n = (q^{n+1} - 1)/(q - 1)$ pro $q \neq 1$ a součet $1/(1 - q)$ pro $-1 < q < 1$; pro $q \geq 1$ diverguje k $+\infty$ a pro $q \leq -1$ součet neexistuje. Harmonická řada $\sum 1/n$ diverguje, přestože $1/n \rightarrow 0$; obecně $\sum 1/n^s$ konverguje pro $s > 1$ a diverguje pro $s \leq 1$. I při ostrých nerovnostech členů mohou být limity stejné, například $1 - 1/n < 1$. Taylorův polynom řádu n : $T_n^{f,a}(x) = \sum_{i=0}^n \frac{f^{(i)}(a)}{i!} (x - a)^i$, kde nultý člen je $f(a)$. Taylorova řada je $\sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$ a tam, kde sečte na funkci, dává $f(x)$. Pro neklesající f na $[1, n]$ platí $\sum_{k=1}^{n-1} f(k) \leq \int_1^n f \leq \sum_{k=2}^n f(k)$, což se používá k odhadům konečných i nekonečných součtů. Obsah oblasti pod grafem nezáporné funkce je $\int_a^b f$. Rotací oblasti pod f kolem osy x vznikne těleso s objemem $\pi \int_a^b f(t)^2 dt$ a povrchem $2\pi \int_a^b f(t) \sqrt{1 + (f'(t))^2} dt$. Délka křivky grafu f od a do b je $\int_a^b \sqrt{1 + (f'(t))^2} dt$. Substituce se používá tak, že se změní proměnná i meze určitého integrálu; per partes převádí integrál součinu na jednodušší integrál.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Kdy konverguje geometrická řada a jaký má součet?
- Co poznám z f' a co z f'' ?
- Jak souvisí Riemannův integrál s primitivní funkcí?

Pozor (častá past): Harmonická řada diverguje, i když členy jdou k nule. l'Hospital jen na neurčité výrazy.

4.2 Algebra a lineární algebra

priorita: střední cíl: žlutá-zelená (1-2 body)

Algebraické struktury (grupy, permutace, tělesa včetně konečných); soustavy lineárních rovnic (Gaussova a Gauss-Jordanova eliminace); matice (hodnost, regulární a inverzní); vektorové prostory (lineární nezávislost, báze, dimenze, Steinitzova věta, podprostory); lineární zobrazení (obraz, jádro, isomorfismus); skalární součin (norma, Cauchy-Schwarz, ortonormální báze, Gram-Schmidt, ortogonální projekce a matice); determinanty (Laplaceův rozvoj, geometrie); vlastní čísla a vektory (charakteristický polynom, diagonalizace, spektrální rozklad, symetrické matice); pozitivně (semi)definitní matice (Choleského rozklad).

Učivo (jak na to):

Lineární algebra studuje *vektorové prostory* a *lineární zobrazení* (= násobení maticí). Hodně pojmů jsou jen různé pohledy na totéž: hodnost, dimenze obrazu, počet pivotů.

1 Definice: Binární operace na X je zobrazení $X \times X \rightarrow X$. Grupa $(G, \circ)$ má asociativní operaci, neutrální prvek e a ke každému a inverzní prvek b s $a \circ b = b \circ a = e$; je abelovská, pokud je operace komutativní. Podgrupa $(H, \bullet) \leq (G, \circ)$ splňuje $H \subseteq G$ a operace se shodují na H . Permutace na $[n]$ je bijekce $[n] \rightarrow [n]$; skládání je $(q \circ p)(i) = q(p(i))$, permutační matice má $(P)_{ij} = 1$ právě pro $p(i) = j$, cyklus $(1, 2, 3, 4)$ posílá 1 na 2, 2 na 3, 3 na 4, 4 na 1, transpozice je cyklus délky 2. Těleso $\mathbb{K}$ má sčítání a násobení tak, že $(\mathbb{K}, +)$ a $(\mathbb{K} \setminus \{0\}, \cdot)$ jsou abelovské grupy a platí distributivita. Charakteristika je nejmenší n s $1 + \dots + 1 = 0$, nebo 0, když takové n neexistuje. Soustava má tvar $Ax = b$, rozšířená matice je $(A|b)$, homogenní soustava je $Ax = 0$. Elementární řádkové úpravy jsou násobení řádku nenulovým skalárem a přičtení řádku k jinému; z nich plyne přičtení násobku řádku a záměna řádků. REF má nenulové řádky seřazené podle počátečních nul a nulové řádky dole; první nenulový prvek řádku je pivot. RREF má pivoty rovné 1 a nuly nad i pod pivoty. Hodnost je počet pivotů v libovolném REF. Inverzní matice A^{-1} splňuje $AA^{-1} = I_n$; regulární matice má inverzi, singulární ji nemá. Vektorový prostor V nad $\mathbb{K}$ je abelovská grupa $(V, +)$ s násobením skalárem splňujícím asociativitu skalárů, $1u = u$ a obě distributivity. Podprostor je neprázdná podmnožina uzavřená na součet a násobení skalárem. Lineární kombinace je $\sum \alpha_i v_i$, lineární nezávislost znamená $\sum \alpha_i v_i = 0 \Rightarrow \alpha_i = 0$. Lineární obal $\text{span}(X)$ je průnik všech podprostorů obsahujících X , ekvivalentně množina všech lineárních kombinací z X . Báze je lineárně nezávislá generující množina, dimenze je velikost libovolné báze a souřadnice v bázi $X = (v_1, \dots, v_n)$ jsou koeficienty v zápisu $u = \sum \alpha_i v_i$. Lineární zobrazení splňuje $f(u + v) = f(u) + f(v)$ a $f(\alpha u) = \alpha f(u)$; obraz a jádro jsou $f(U)$ a $\ker f = \{u : f(u) = 0\}$. Skalární součin nad $\mathbb{C}$ je pozitivně definitní, hermitovský symetrický a lineární v prvním argumentu; nad $\mathbb{R}$ je symetrický. Norma je $\|u\| = \sqrt{\langle u|u \rangle}$ a kolmost je $\langle u|v \rangle = 0$.

2 Věty, pravidla a algoritmy: Každou permutaci lze rozložit na transpozice; znaménko je $\text{sgn}(p) = (-1)^{\text{počet inverzí}}$, stejně $(-1)^{\text{počet transpozic}}$. $\mathbb{Z}_p$ je těleso právě pro prvočíslo p ; konečné těleso má řád p^k a konečná tělesa mohou mít i neprvočíselný řád, například $GF(4)$. Řádkové úpravy nemění množinu řešení soustavy. Gaussova eliminace převádí matici do REF: volí se pivot, vhodnými násobky se nulují prvky pod ním a pokračuje se ve zbytku matice. Gaussova-Jordanova eliminace pokračuje do RREF: pivotové řádky se vydělí pivotem a odspodu se nulují prvky nad pivoty. Je-li pivot v posledním sloupci rozšířené matice, soustava nemá řešení. Pokud x^0 řeší $Ax = b$, pak $\bar{x} \mapsto x^0 + \bar{x}$ je bijekce mezi řešeními $A\bar{x} = 0$ a $Ax = b$; obecné řešení je $y + \sum p_i \bar{x}_i$, kde p_i jsou volné parametry. Pro čtvercovou A jsou ekvivalentní: A je regulární, $\text{rank}(A) = n$, $A \sim I_n$, soustava $Ax = 0$ má jen nulové řešení. Inverzi počítáme $(A|I_n) \sim (I_n|A^{-1})$, selhání znamená singularitu. Lemma o výměně: je-li $y_1, \dots, y_n$ systém generátorů a $x = \sum \alpha_i y_i$ s $\alpha_k \neq 0$, lze y_k nahradit x a stále dostat systém generátorů. Steinitzova věta: je-li $x_1, \dots, x_m$ lineárně nezávislý systém a $y_1, \dots, y_n$ generuje V , pak $m \leq n$ a lze doplnit $x_1, \dots, x_m$ některými y_i na systém generátorů. Průnik podprostorů je podprostor. Pro $A \in \mathbb{K}^{m \times n}$ platí $\dim \ker A + \text{rank}(A) = n$ a dimenze řádkového i sloupcového prostoru je hodnost. Lineární zobrazení je určeno obrazy báze. Matice $[f]_{X,Y}$ má ve sloupcích souřadnice $[f(u_i)]_Y$ a $[f(w)]_Y = [f]_{X,Y}[w]_X$. Pro složení platí $[g \circ f]_{B,D} = [g]_{C,D}[f]_{B,C}$. Matice přechodu je $[id]_{X,Y}$, $[u]_Y = [id]_{X,Y}[u]_X$, je regulární, inverzní je přechod zpět a prakticky $[id]_{X,Y} = Y^{-1}X$. Isomorfismus je bijektivní lineární zobrazení; inverze je opět isomorfismus, isomorfní prostory mají stejnou dimenzi a f je isomorfismus právě tehdy, když jeho matice v bázích je regulární. Cauchyho-Schwarzova nerovnost: $|\langle u|v \rangle| \leq \|u\|\|v\|$; trojúhelníková nerovnost: $\|u + v\| \leq \|u\| + \|v\|$; pro kolmé vektory platí Pythagoras $\|u + v\|^2 = \|u\|^2 + \|v\|^2$. Gram-Schmidt: z báze u_i vytvoří ortonormální bázi v_i předpisem $w_i = u_i - \sum_{j < i} \langle u_i|v_j \rangle v_j$, $v_i = w_i/\|w_i\|$. Ortonormální báze dává Fourierův rozvoj $u = \sum_i \langle u|v_i \rangle v_i$. Ortogonální projekce na podprostor s ON bází v_i je $p(u) = \sum_i \langle u|v_i \rangle v_i$ a je lineární. Ortogonální matice Q splňuje $Q^T Q = I$, tedy $Q^{-1} = Q^T$; právě tehdy její sloupce tvoří ON bázi. Ortogonální matice i její transpozice a inverze zachovávají skalární součin, normy, úhly a délky, součin ortogonálních matic je ortogonální.

3 Vzorce, případy a aplikace: Sloupcový prostor $\mathcal{S}(A) = \{Ax : x \in \mathbb{K}^n\} \subseteq \mathbb{K}^m$, řádkový prostor $\mathcal{R}(A) = \{A^T y : y \in \mathbb{K}^m\} \subseteq \mathbb{K}^n$, jádro $\ker A = \{x : Ax = 0\}$. Jednotková matice má $(I_n)_{ij} = 1$ právě pro $i = j$, transpozice splňuje $(A^T)_{ij} = a_{ji}$, symetrická matice splňuje $A^T = A$ a součin matic je $(AB)_{ij} = \sum_{k=1}^n a_{ik}b_{kj}$. Determinant: $\det A = \sum_{p \in S_n} \operatorname{sgn}(p) \prod_{i=1}^n a_{i,p(i)}$, $\det(AB) = \det A \det B$, $\det A = \det A^T$, regularita je ekvivalentní $\det A \neq 0$, pro regulární A je $\det(A^{-1}) = (\det A)^{-1}$. Laplaceův rozvoj podle i -tého řádku: $\det A = \sum_{j=1}^n a_{ij}(-1)^{i+j} \det A^{ij}$, kde A^{ij} vznikne odstraněním řádku i a sloupce j . Geometricky $|\det A|$ měří objem rovnoběžnostěnu ze sloupců a faktor změny objemu lineárním zobrazem. Vlastní číslo $A \in \mathbb{C}^{n \times n}$ je λ , pro něž existuje $x \neq 0$ s $Ax = \lambda x$; vlastní vektor určuje invariantní směr a λ škálování v tomto směru. Charakteristický polynom je $p_A(t) = \det(A - tI_n)$, vlastní čísla jsou jeho kořeny včetně algebraických násobností, geometrická násobnost λ je $n - \operatorname{rank}(A - \lambda I_n)$ a je nejvýše algebraická. Determinant je součin vlastních čísel a stopa jejich součet; A je regulární právě tehdy, když 0 není vlastní číslo; A^T má stejná vlastní čísla, ale obecně jiné vlastní vektory. Pro regulární A má A^{-1} vlastní čísla λ_i^{-1} se stejnými vlastními vektory; A^2 , αA a $A + \alpha I$ mění vlastní čísla na λ_i^2 , $\alpha \lambda_i$ a $\lambda_i + \alpha$. Podobnost: $A = SBS^{-1}$ pro regulární S ; zachovává vlastní čísla a počty vlastních vektorů. A je diagonalizovatelná právě tehdy, když má n lineárně nezávislých vlastních vektorů; pak $A = S\Lambda S^{-1}$ a $A^2 = S\Lambda^2 S^{-1}$. Různá vlastní čísla zaručují diagonalizovatelnost. Reálná symetrická matice má reálná vlastní čísla a ortogonální spektrální rozklad $A = Q\Lambda Q^T$. Symetrická A je pozitivně semidefinitní, když $x^T Ax \geq 0$ pro všechna x , a pozitivně definitní, když $x^T Ax > 0$ pro $x \neq 0$. Ekvivalentně má nezáporná, respektive kladná vlastní čísla, a existuje U s $A = U^T U$; v definitním případě má U hodnotu n . Nesymetrickou matici lze pro kvadratickou formu symetrizovat na $(A + A^T)/2$. Součet pozitivně definitních matic je pozitivně definitní, kladný násobek také, pozitivně definitní matice je regulární a její inverze je pozitivně definitní. Skalární součin na $\mathbb{R}^n$ má tvar $\langle x|y \rangle = x^T Ay$ právě pro symetrickou pozitivně definitní A . Pozitivní definitnost lze testovat Gaussovou eliminací bez výměn řádků a se sledováním kladných pivotů. Choleského věta: pro každou pozitivně definitní $A \in \mathbb{R}^{n \times n}$ existuje jediná dolní trojúhelníková L s kladnou diagonálou tak, že $A = LL^T$; soustavu $Ax = b$ pak řešíme dvěma trojúhelníkovými soustavami $Ly = b$ a $L^T x = y$.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co znamená, že je matice regulární (vyjmenuj ekvivalentní podmínky)?
- Jak najdu vlastní čísla a co je vlastní vektor?
- Co je pozitivně definitní matice a jak souvisí s vlastními čísly?

Pozor (častá past): Vlastní čísla = kořeny char. polynomu, ne prvky na diagonále (pokud matice není trojúhelníková).

4.3 Diskrétní matematika

priorita: střední

cíl: žlutá-zelená (1-2 body)

Relace a jejich vlastnosti (reflexivita, symetrie, antisymetrie, tranzitivita); ekvivalence a rozkladové třídy; částečná uspořádání (řetězec, antiřetězec, výška a šířka a věta o jejich vztahu); funkce (prostá, na, bijekce; počty funkcí mezi konečnými množinami); permutace; kombinační čísla a binomická věta; princip inkluze a exkluze (důkaz a aplikace); Hallova věta o systému různých reprezentantů (vztah k párování v bipartitním grafu, polynomiální algoritmus).

Učivo (jak na to):

Hodně z toho je „počítání možností“ a struktura na konečných množinách. *Inkluze-exkluze* opravuje dvojí započítání; *Hallova věta* říká, kdy lze každého spárovat.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Binární relace mezi X, Y je podmnožina $X \times Y$; relace na X je podmnožina X^2 a píšeme $xRy \equiv (x, y) \in R$. Inverzní relace je $R^{-1} = \{(y, x) : (x, y) \in R\}$, složení $T = R \circ S$ splňuje $xTz \equiv \exists y : xRy \wedge ySz$ a diagonála je $\Delta_X = \{(x, x) : x \in X\}$. Reflexivita je $\Delta_X \subseteq R$, symetrie $R = R^{-1}$, antisymetrie $R \cap R^{-1} \subseteq \Delta_X$ a tranzitivita $R \circ R \subseteq R$. Ekvivalence je reflexivní, symetrická a tranzitivní; třída prvku je $R[x] = \{y : xRy\}$. Rozklad množiny X je systém neprázdných po dvou disjunktních podmnožin se sjednocením X . Částečné uspořádání je reflexivní, antisymetrická a tranzitivní relace; lineární uspořádání má každé dva prvky porovnatelné. Ostré uspořádání $a < b$ znamená $a \leq b$ a $a \neq b$, je ireflexivní, asymetrické a tranzitivní. Nejmenší prvek splňuje $x \leq y$ pro všechna y , minimální prvek nemá žádné $y < x$; duálně pro největší a maximální. Řetězec je podmnožina po dvou porovnatelných prvků, antiřetězec po dvou neporovnatelných. Funkce $X \rightarrow Y$ je relace s právě jedním obrazem pro každé x ; je prostá, když různé prvky mají různé obrazy, surjektivní na Y , když každý y má vzor, a bijektivní, když má každý y právě jeden vzor. Permutace je bijekce $[n] \rightarrow [n]$, pevný bod splňuje $\pi(i) = i$. Kombinační číslo je $\binom{n}{k} = \frac{n!}{k!(n-k)!}$, kde $m^n = m(m-1) \cdots (m-n+1)$. Párování v grafu je množina hran bez společných vrcholů; perfektní párování pokrývá všechny vrcholy. SRR v hypergrafu $H = (V, E)$ je prostá funkce $r : E \rightarrow V$ s $r(e) \in e$.

2 Věty, pravidla a algoritmy: Třídy ekvivalence jsou neprázdné, dvě třídy jsou stejné nebo disjunktní a množina tříd určuje ekvivalenci jednoznačně; ekvivalence a rozklady si tak odpovídají. Konečná neprázdna uspořádaná množina má minimální a maximální prvek. Výška $\omega(X, \leq)$ je délka nejdelšího řetězce, šířka $\alpha(X, \leq)$ délka nejdelšího antiřetězce a pro konečnou ČUM platí $\alpha\omega \geq |X|$, tedy $\max(\alpha, \omega) \geq \sqrt{|X|}$. Počet funkcí $N \rightarrow M$ pro $|N| = n$, $|M| = m$ je m^n , počet prostých funkcí je m^n , bijekcí na n prvcích je $n!$ a $|X^k| = |X|^k$. Počet surjekcí $X \rightarrow Y$ pro $n \geq m > 0$ je $\sum_{k=0}^m (-1)^k \binom{m}{k} (m-k)^n$. Permutací na $[n]$ je $n!$. Základní identity: $\binom{n}{k} = \binom{n}{n-k}$, $\binom{n-1}{k-1} + \binom{n-1}{k} = \binom{n}{k}$ a binomická věta $(x+y)^n = \sum_{i=0}^n \binom{n}{i} x^{n-i} y^i$; z ní plyne $\sum_{k=0}^n \binom{n}{k} = 2^n$, důkaz $(x^n)' = nx^{n-1}$ přes limitu difference a $e = \lim(1 + 1/n)^n$. Princip inkluze a exkluze: $|\bigcup_{i=1}^n A_i| = \sum_{k=1}^n (-1)^{k+1} \sum_{I \in \binom{[n]}{k}} |\bigcap_{i \in I} A_i|$. Důkaz počítá příspěvek jednoho prvku, který leží právě v t množinách: dostane $\sum_{k=1}^t (-1)^{k+1} \binom{t}{k} = 1$ z $(1-1)^t = 0$. Hallova věta pro bipartitní graf s partitami A, B : existuje párování velikosti $|A|$ právě tehdy, když pro každé $X \subseteq A$ platí $|N(X)| \geq |X|$. Nutnost je zřejmá ze spárovaných sousedů; postačitelnost lze dokázat přes Königovu-Egerváryho větu, že maximum párování se rovná minimu vrcholového pokrytí v bipartitním grafu. Hypergrafová Hallova věta: $H = (V, E)$ má SRR právě tehdy, když pro každé $F \subseteq E$ platí $|\bigcup_{e \in F} e| \geq |F|$; převod vede přes incidenční bipartitní graf. Maximální párování, a tedy i SRR, lze najít v polynomiálním čase.

3 Vzorce, případy a aplikace: Příklady relací jsou prázdná, univerzální a diagonální relace; příklady ekvivalencí jsou rovnost, shodnost modulo K a geometrická podobnost. Hasseův diagram zachycuje ČUM, větší prvky jsou výše; z nejmenšího prvku je cesta ke všem ostatním, minimální prvek nemá nic pod sebou. Pro problém šatnářky označme $A_i = \{\pi \in S_n : \pi(i) = i\}$; počet permutací bez pevného bodu je $D(n) = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$ a pravděpodobnost je $\sum_{k=0}^n \frac{(-1)^k}{k!} \approx e^{-1}$. Eulerova funkce $\varphi(n)$ počítá čísla z $[n]$ nesoudělná s n ; pro rozklad $n = p_1^{e_1} \cdots p_k^{e_k}$ platí $\varphi(n) = n \prod_{i=1}^k (1 - \frac{1}{p_i})$, což plyne z PIE na množiny čísel dělitelných p_i . Počet surjekcí se odvodí z počtu všech funkcí m^n odečtením funkcí, které vynechají aspoň jeden prvek cílové množiny. Pokud v hypergrafu existuje k hyperhran, jejichž sjednocení má méně než k vrcholů, SRR neexistuje; Hall říká, že tato překážka je jediná.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaké tři vlastnosti dělají z relace ekvivalenci?
- Jak zní Hallova podmínka?
- Kolik je funkcí (a kolik bijekcí) mezi konečnými množinami?

Pozor (častá past): Antisymetrie neznamena „není symetrická“. Bijekce mezi A, B existuje jen při $|A| = |B|$.

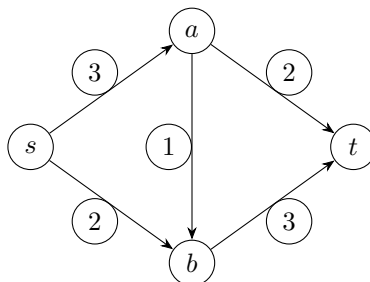
4.4 Teorie grafů

priorita: střední cíl: zelená (2 body), vděčné téma

Základní pojmy (izomorfismus, stupeň vrcholu, doplněk, bipartitní graf); příklady (úplný, cesty, kružnice); souvislost, komponenty, vzdálenost; stromy (charakteristiky); rovinné grafy (Eulerova formule a max. počet hran); barevnost (vztah barevnosti a klikovosti); hranová a vrcholová souvislost (Mengerova věta); orientované grafy (silná a slabá souvislost); toky v sítích (definice, existence max. toku, Ford-Fulkerson pro celočíselné kapacity).

Učivo (jak na to):

Grafy = vrcholy a hrany. Mnoho úloh = „dá se projít / spojit / obarvit / protlačit tok?“.



Sít s kapacitami. Max tok = min řez (zde 5).

Odpověď podle bodů (cíle pro průchod: zelená = 2 body)

1 Definice: Graf je $G = (V, E)$, kde V je konečná neprázdná množina vrcholů a $E \subseteq \binom{V}{2}$ množina hran. Izomorfismus grafů je bijekce vrcholů zachovávající sousednost; izomorfie je ekvivalence. Podgraf $G' = (V', E')$ splňuje $V' \subseteq V$, $E' \subseteq E$; indukovaný podgraf $G[A]$ má všechny hrany původního grafu uvnitř A . Okolí $N_G(v)$ je množina sousedů, stupeň je počet incidentních hran, k -regulární graf má všechny stupně k a skóre je posloupnost stupňů. Doplněk má stejné vrcholy a právě chybějící hrany. Bipartitní graf má rozklad $V = L \dot{\cup} P$ a všechny hrany vedou mezi partitami. Základní grafy: úplný K_n , úplný bipartitní $K_{m,n}$, prázdný E_n , cesta P_n s vrcholy $\{0, \dots, n\}$ a hranami $\{i, i+1\}$, kružnice C_n pro $n \geq 3$ s hranami $\{i, (i+1) \bmod n\}$. Souvislost znamená cestu mezi každými dvěma vrcholy; dosažitelnost je ekvivalence a její třídy indukují komponenty. Vzdálenost $d_G(u, v)$ je minimum délek cest v souvislém grafu. Sled může opakovat vrcholy i hrany, tah neopakuje hrany; eulerovský tah obsahuje všechny hrany a eulerovský graf má uzavřený eulerovský tah. Strom je souvislý acyklický graf, les je acyklický graf, list má stupeň 1. Rovinné nakreslení nemá křížení hran, stěny jsou oblasti roviny včetně vnější a graf je rovinný, pokud takové nakreslení má. Obarvení k barvami je $c: V \rightarrow [k]$ s různými barvami na koncích hran; barevnost $\chi(G)$ je minimální k . Klika je množina po dvou sousedních vrcholů, klikovost $\kappa(G)$ je největší k s podgrafem K_k , nezávislá množina nemá vnitřní

hranu. Hranový řez je $F \subseteq E$ s nesouvislým $G - F$; vrcholový řez je $A \subseteq V$ s nesouvislým $G - A$. Orientovaný graf má $E \subseteq V^2 \setminus \Delta_V$; podkladový graf zapomene orientace, vstupní a výstupní stupeň počítají hrany do a z vrcholu, vyvážený vrchol má oba stupně stejné. Slabá souvislost je souvislost podkladového grafu, silná souvislost je orientovaná cesta pro každou uspořádanou dvojici. Toková síť je (V, E, z, s, c) se zdrojem z , stokem s a kapacitami $c : E \rightarrow [0, +\infty)$; tok $f : E \rightarrow [0, +\infty)$ splňuje $0 \leq f(e) \leq c(e)$ a Kirchhoffův zákon ve vrcholech mimo z, s .

2 Věty, pravidla a algoritmy: Dosažitelnost je ekvivalence a graf je souvislý právě tehdy, když má jednu komponentu. Grafová vzdálenost je metrika: je nezáporná, nulová právě na diagonále, symetrická a splňuje trojúhelníkovou nerovnost. Eulerovský neorientovaný graf je právě souvislý graf se všemi stupni sudými. Každý strom s aspoň dvěma vrcholy má aspoň dva listy; je-li v list, pak G je strom právě tehdy, když $G - v$ je strom. Strom na v vrcholech má $v - 1$ hran. Ekvivalentní charakteristiky stromů: souvislý a acyklický; mezi každými dvěma vrcholy je právě jedna cesta; je souvislý a smazání libovolné hrany ho rozpojí; je acyklický a přidání libovolné hrany vytvoří cyklus; je souvislý a má $|E| = |V| - 1$. Eulerova formule pro souvislý rovinný graf: $v + f = e + 2$. Důkaz jde indukcí podle hran: strom má jednu stěnu, a při odstranění hrany ležící na cyklu se sníží e i f o 1. Maximální rovinný graf je triangulace, proto pro jednoduchý rovinný graf s $v \geq 3$ platí $e \leq 3v - 6$; průměrný stupeň je menší než 6. Bez trojúhelníků platí $e \leq 2v - 4$, průměrný stupeň je menší než 4 a existuje vrchol stupně nejvýše 3. Kuratowského věta: graf je rovinný právě tehdy, když neobsahuje podgraf izomorfní dělení K_5 nebo $K_{3,3}$. Pro barevnost platí $H \subseteq G \Rightarrow \chi(H) \leq \chi(G)$, $\chi(K_n) = n$, cesty a sudé kružnice mají barevnost 2, liché kružnice 3, stromy jsou 2-obarvitelné a rovinné grafy jsou 4-obarvitelné. Dále $\chi(G) \leq 2$ právě tehdy, když G je bipartitní, právě tehdy, když neobsahuje lichou kružnici, a vždy $\chi(G) \geq \kappa(G)$. Mengerova hranová xy -verze: mezi různými x, y existuje k hranově disjunktních cest právě tehdy, když neexistuje hranový xy -řez velikosti menší než k ; globálně je graf hranově k -souvislý právě tehdy, když mezi každými dvěma vrcholy existuje k hranově disjunktních cest. Vrcholová verze: pro různé nesousední x, y existuje k navzájem vnitřně vrcholově disjunktních cest právě tehdy, když neexistuje vrcholový xy -řez velikosti menší než k ; globálně obdobně pro vrcholovou k -souvislost. Pro orientované grafy platí: vyvážený a slabě souvislý právě tehdy, když je eulerovský, právě tehdy, když je vyvážený a silně souvislý. Maximální tok existuje v každé konečné síti: množina toků je kompaktní podmnožina $\mathbb{R}^m$ a velikost toku je spojitá. Minimaxová věta o toku a řezu: hodnota maximálního toku se rovná kapacitě minimálního řezu.

3 Vzorce, případy a aplikace: Operace s grafy zahrnují přidání a odebrání vrcholu nebo hrany, dělení hrany a kontrakci hrany. Topologický graf je graf spolu s konkrétním nakreslením; hranice stěny je uzavřený sled. Grafy K_5 a $K_{3,3}$ nejsou rovinné. Vrcholová souvislost $k_v(G)$ je největší k , pro který je G vrcholově k -souvislý; pro K_n je $k_v(K_n) = n - 1$ a pro neúplný graf je to velikost nejmenšího vrcholového řezu. Hranová souvislost $k_e(G)$ je velikost nejmenšího hranového řezu. V orientovaném grafu je součet vstupních stupňů roven součtu výstupních stupňů a počtu hran; silná souvislost implikuje slabou. Velikost toku je $w(f) = f[\text{Out}(z)] - f[\text{In}(z)]$, kde $f[A] = \sum_{e \in A} f(e)$. Řez v síti je množina hran, kterou protne každá orientovaná cesta ze zdroje do stoku. Rezerva hrany uv ve Ford-Fulkersonově metodě je $r(uv) = c(uv) - f(uv) + f(vu)$; nasycená hrana má nulovou rezervu. Algoritmus opakuje: najdi nenasyčenou cestu ze zdroje do stoku, vezmi minimum rezerv na cestě, v protisměru nejdříve odečti existující tok a zbytek přičti po směru. Pro celočíselné kapacity vrátí celočíselný maximální tok, racionální kapacity lze převést na celočíselné a pro iracionální kapacity se běh může pokazit.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Kolik hran má strom a jak poznám bipartitní graf?
- Co říká Eulerova formule a kolik nejvíc hran má rovinný graf?
- Jak funguje Ford-Fulkerson a co je max-flow = min-cut?

Pozor (častá past): Eulerova formule platí pro souvislý rovinný graf; odhad $3v - 6$ navíc

předpokládá jednoduchost a $v \geq 3$. Bipartitní $\iff$ bez liché kružnice.

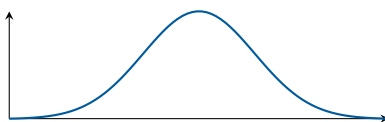
4.5 Pravděpodobnost a statistika

priorita: střední cíl: žlutá-zelená (1-2 body)

Pravděpodobnostní prostor, jevy, nezávislost, podmíněná pravděpodobnost, Bayesův vzorec; náhodné veličiny (diskrétní i spojité), distribuční funkce, hustota/pravděpodobnostní funkce; střední hodnota (linearita, Markovova nerovnost); rozptyl (vzorec pro součet); konkrétní rozdělení (geometrické, binomické, Poissonovo, normální, exponenciální); limitní věty (zákon velkých čísel, centrální limitní věta); bodové a intervalové odhady; testování hypotéz (chyby 1. a 2. druhu, hladina významnosti).

Učivo (jak na to):

Pravděpodobnost popisuje nejistotu. *Střední hodnota* = dlouhodobý průměr, *rozptyl* = míra rozkolísanosti. CLV říká, proč se všude objevuje normální rozdělení.



Normální rozdělení (Gaussova křivka): limita součtu mnoha nezávislých vlivů (CLV).

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Pravděpodobnostní prostor je $(\Omega, \mathcal{F}, P)$, kde Ω je množina elementárních jevů, $\mathcal{F}$ je prostor jevů obsahující $\emptyset, \Omega$, uzavřený na doplněk a spočetná sjednocení, a $P: \mathcal{F} \rightarrow [0, 1]$ splňuje $P(\Omega) = 1$ a spočetnou aditivitu na po dvou disjunktních jevech. Klasický prostor je konečný s uniformní pravděpodobností, diskrétní připouští neuniformní pravděpodobnosti, geometrický má pravděpodobnost úměrnou míře oblasti a spojitý popisuje neuniformní hustoty. Jevy A, B jsou nezávislé, když $P(A \cap B) = P(A)P(B)$; rodina je nezávislá, když totéž platí pro každou konečnou podrodinu, a po dvou nezávislá, když to platí jen pro dvojice. Podmíněná pravděpodobnost pro $P(B) > 0$ je $P(A|B) = P(A \cap B)/P(B)$. Náhodná veličina je funkce na Ω ; diskrétní veličina má pravděpodobnostní funkci $p_X(x) = P(X = x)$, distribuční funkce je $F_X(x) = P(X \leq x)$ a spojitá veličina má hustotu $f_X \geq 0$ s $F_X(x) = \int_{-\infty}^x f_X(t) dt$ a $\int_{-\infty}^{\infty} f_X = 1$. Střední hodnota je $\mathbb{E}X = \sum_x xP(X = x)$ nebo $\int x f_X(x) dx$; rozptyl je $\text{var}(X) = \mathbb{E}((X - \mathbb{E}X)^2)$ a směrodatná odchylka $\sigma_X = \sqrt{\text{var}(X)}$. Kovariance je $\text{cov}(X, Y) = \mathbb{E}((X - \mathbb{E}X)(Y - \mathbb{E}Y))$ a korelace $\rho(X, Y) = \text{cov}(X, Y)/(\sigma_X \sigma_Y)$.

2 Věty, pravidla a algoritmy: Základní pravidla: $P(A) + P(A^c) = 1$, $A \subseteq B \Rightarrow P(B \setminus A) = P(B) - P(A)$ a $P(A) \leq P(B)$, $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ a Booleova nerovnost $P(\bigcup_i A_i) \leq \sum_i P(A_i)$. Řetězové pravidlo: $P(A_1 \cap \dots \cap A_n) = P(A_1)P(A_2|A_1) \dots P(A_n|A_1 \cap \dots \cap A_{n-1})$. Věta o úplné pravděpodobnosti pro rozklad B_i : $P(A) = \sum_i P(B_i)P(A|B_i)$. Bayesův vzorec: $P(B_j|A) = \frac{P(B_j)P(A|B_j)}{\sum_i P(B_i)P(A|B_i)}$. Distribuční funkce je neklesající, zprava spojitá, má limity 0 v $-\infty$ a 1 v $+\infty$ a $P(a < X \leq b) = F_X(b) - F_X(a)$. Pravidlo naivního statistika: $\mathbb{E}(g(X)) = \sum_x g(x)P(X = x)$ nebo $\int g(x)f_X(x) dx$; pro dvě veličiny $\mathbb{E}(g(X, Y)) = \sum_x \sum_y g(x, y)P(X = x, Y = y)$. Linearita: $\mathbb{E}(aX + bY) = a\mathbb{E}X + b\mathbb{E}Y$ vždy; pro nezávislé X, Y platí $\mathbb{E}(XY) = \mathbb{E}X \mathbb{E}Y$. Podmíněná střední hodnota je $\mathbb{E}(X|B) = \sum_x xP(X = x|B)$ a celková střední hodnota $\mathbb{E}X = \sum_i P(B_i)\mathbb{E}(X|B_i)$. Markovova nerovnost: pro $X \geq 0$ a $a > 0$ je $P(X \geq a) \leq \mathbb{E}X/a$. Pro rozptyl platí $\text{var}(X) = \mathbb{E}(X^2) - (\mathbb{E}X)^2$ a $\text{var}(aX + b) = a^2\text{var}(X)$. Dále $\text{cov}(X, Y) = \mathbb{E}(XY) - \mathbb{E}X \mathbb{E}Y$, nezávislost implikuje nulovou kovarianci, $\text{cov}(X, X) = \text{var}(X)$ a kovariance je

lineární v argumentu. Pro $X = \sum_i X_i$ platí $\text{var}(X) = \sum_i \sum_j \text{cov}(X_i, X_j)$; pro nezávislé veličiny je rozptyl součtu součtem rozptylů. Silný zákon velkých čísel: pro nezávislé stejně rozdělené X_i s průměrem μ platí $\bar{X}_n \rightarrow \mu$ skoro jistě. Slabý zákon: $P(|\bar{X}_n - \mu| > \varepsilon) \rightarrow 0$, tedy $\bar{X}_n \xrightarrow{P} \mu$. Pro diskrétní veličinu je entropie

$$H(X) = - \sum_x P(X = x) \log P(X = x),$$

kde členy s nulovou pravděpodobností vynecháme. Podmíněná entropie je $H(X|Y) = \sum_y P(Y = y) H(X|Y = y)$ a vzájemná informace je

$$I(X; Y) = H(X) - H(X|Y) = H(X) + H(Y) - H(X, Y).$$

Platí $I(X; Y) \geq 0$, tedy $H(X) \geq H(X|Y)$: znalost Y nemůže v průměru zvýšit nejistotu o X . Zároveň $H(X, Y) = H(X) + H(Y) - I(X; Y) \leq H(X) + H(Y)$, s rovností právě při nezávislosti. Centrální limitní věta: pro stejně rozdělené nezávislé veličiny se střední hodnotou μ a rozptylem $\sigma^2 > 0$ má $Y_n = \frac{(X_1 + \dots + X_n) - n\mu}{\sigma\sqrt{n}}$ limitní rozdělení $N(0, 1)$, tj. $F_{Y_n}(x) \rightarrow \Phi(x)$.

3 Vzorce, případy a aplikace: Bernoulliho rozdělení $\text{Ber}(p)$: $P(X = 1) = p$, $P(X = 0) = 1 - p$, $\mathbb{E}X = p$, $\text{var}(X) = p(1 - p)$. Geometrické $\text{Geom}(p)$ pro první úspěch: $P(X = k) = (1 - p)^{k-1}p$, $\mathbb{E}X = 1/p$, $\text{var}(X) = (1 - p)/p^2$. Binomické $\text{Bin}(n, p)$: $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$, $\mathbb{E}X = np$, $\text{var}(X) = np(1 - p)$. Poissonovo $\text{Pois}(\lambda)$: $P(X = k) = \lambda^k e^{-\lambda}/k!$, je limitou $\text{Bin}(n, \lambda/n)$, $\mathbb{E}X = \text{var}(X) = \lambda$. Uniformní $U(a, b)$: $f(x) = 1/(b - a)$, $F(x) = (x - a)/(b - a)$ uvnitř intervalu, $\mathbb{E}X = (a + b)/2$, $\text{var}(X) = (b - a)^2/12$. Exponenciální $\text{Exp}(\lambda)$: $f(x) = \lambda e^{-\lambda x}$, $F(x) = 1 - e^{-\lambda x}$ pro $x \geq 0$, $\mathbb{E}X = 1/\lambda$, $\text{var}(X) = 1/\lambda^2$. Normální $N(0, 1)$ má $\varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$; obecné $N(\mu, \sigma^2)$ má hustotu $\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}((x-\mu)/\sigma)^2}$ a standardizace $(X - \mu)/\sigma \sim N(0, 1)$. Součet konečně mnoha nezávislých normálních veličin je normální se součtem středních hodnot a rozptylů. Pravidlo 3σ dává přibližně 0.68, 0.95, 0.997 pro intervaly $\mu \pm \sigma$, $\mu \pm 2\sigma$, $\mu \pm 3\sigma$. Bodový odhad $\hat{\theta}_n$ pro $g(\theta)$ je nestranný, když $\mathbb{E}\hat{\theta}_n = g(\theta)$, asymptoticky nestranný, když se k tomu limita blíží, a konzistentní, když $\hat{\theta}_n \xrightarrow{P} g(\theta)$. Bias je $\mathbb{E}\hat{\theta}_n - \theta$, MSE je $\mathbb{E}((\hat{\theta}_n - \theta)^2)$ a $\text{MSE} = \text{bias}^2 + \text{var}(\hat{\theta}_n)$. Metoda momentů klade teoretické momenty $m_r(\theta) = \mathbb{E}(X^r)$ rovny výběrovým momentům $n^{-1} \sum_i X_i^r$; $m_1 = \mu$, $m_2 = \sigma^2 + \mu^2$. MLE volí $\hat{\theta}_{ML} = \arg \max_{\theta} L(x; \theta)$, často přes logaritmus ℓ ; například pro k leváků z n je $L = \theta^k (1 - \theta)^{n-k}$ a derivujeme $k \log \theta + (n - k) \log(1 - \theta)$. Konfidenční interval $[D, H]$ má spolehlivost $1 - \alpha$, když $P(D \leq \theta \leq H) = 1 - \alpha$; pro normální aproximaci $\hat{\theta} \pm z_{\alpha/2} \text{se}$, kde $z_{\alpha/2} = \Phi^{-1}(1 - \alpha/2)$. Testování: zvolí se testová statistika T , nulová hypotéza H_0 , alternativa a kritický obor W ; chybu 1. druhu tvoří zamítnutí platné H_0 s pravděpodobností α , chybu 2. druhu nezamítnutí neplatné H_0 s pravděpodobností β , síla je $1 - \beta$. p -hodnota je nejmenší hladina, na níž by se H_0 zamítla, tedy pravděpodobnost při H_0 vidět naměřenou nebo extrémnější hodnotu statistiky.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak zní Bayesův vzorec?
- Kdy platí $\text{var}(X + Y) = \text{var}X + \text{var}Y$?
- Co říká centrální limitní věta?

Pozor (častá past): Linearita střední hodnoty platí vždy, ale součet rozptylů jen pro nezávislé veličiny.

4.6 Logika

priorita: střední cíl: žlutá-zelená (1-2 body)

Syntaxe (výroková a predikátová logika, normální tvary CNF/DNF/PNF, převody, použití pro SAT a rezoluci); sémantika (model teorie, splnitelnost, tautologie, důsledek); extenze teorií (konzervativnost, skolemizace); dokazatelnost (formální důkaz, dokazovací systémy: tablo, rezoluce, příp. Hilbertovský kalkul); věty o kompaktnosti a úplnosti; rozhodnutelnost.

Učivo (jak na to):

Syntaxe = forma formulí, *sémantika* = jejich pravdivost v modelech. *Dokazatelnost* (syntaktická) a *platnost* (sémantická) spojuje věta o úplnosti.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Výrokový jazyk je určen neprázdnou množinou výrokových proměnných $\mathbb{P}$. Predikátový jazyk je dán signaturou $\langle \mathcal{R}, \mathcal{F} \rangle$ s relačními a funkčními symboly včetně konstant, aritami a případně rovností; obsahuje proměnné, logické spojky, kvantifikátory a závorky. Struktura je $\mathcal{A} = \langle A, \mathcal{R}^{\mathcal{A}}, \mathcal{F}^{\mathcal{A}} \rangle$ s neprázdnou doménou A , interpretacemi relačních symbolů jako relací na A a funkčních symbolů jako funkcí na A . Term je proměnná, konstanta nebo $f(t_1, \dots, t_n)$. Atomická formule je $R(t_1, \dots, t_n)$ nebo rovnost termů; formule vznikají z atomů negací, binárními spojkami a kvantifikátory. Výskyt proměnné je vázaný kvantifikátorem, jinak volný; otevřená formule nemá kvantifikátory, sentence nemá volné proměnné. Term t je substituovatelný za x ve φ , když nahrazením volných výskytů x nevznikne zachycení proměnné z t ; instance se značí $\varphi(x/t)$. Literál je atom nebo jeho negace, opačný literál je $\bar{\ell}$, klauzule je disjunkce literálů, prázdná klauzule $\square$ není splněna, CNF je konjunkce klauzulí a prázdná CNF je $\top$. DNF je disjunkce elementárních konjunkcí literálů a prázdná DNF je $\perp$. Hornova formule je konjunkce klauzulí s nejvýše jedním pozitivním literálem. PNF má všechny kvantifikátory v prefixu a otevřené jádro. Model výrokové logiky je ohodnocení proměnných; model predikátové teorie je struktura, ve které platí všechny axiomy. Tautologie je formule pravdivá ve všech modelech, splnitelná v aspoň jednom, lživá ve všech modelech a nezávislá vzhledem k teorii, když v některém modelu teorie platí a v jiném ne. Důsledek $T \models \varphi$ znamená, že φ platí v každém modelu T .

2 Věty, pravidla a algoritmy: PNF se získá vytahováním kvantifikátorů před formuli, s přejmenováním proměnných podle potřeby; při průchodu negací se $\forall$ a $\exists$ prohodí a u implikace se levá strana chová jako negovaná. Sémantický převod na DNF vezme modely a pro každý napíše elementární konjunkci; převod na CNF vezme nemodely a každou klauzuli jeden zakáže. Syntaktický převod odstraní implikace a ekvivalence, přesune negace k literálům a použije distributivitu. Horn-SAT: pokud jsou opačné jednotkové klauzule, formule není splnitelná; pokud není jednotková klauzule, je splnitelná nastavením zbytku na 0; jinak nastav jednotkový literál ℓ na 1, odstraň klauzule obsahující ℓ a odstraň $\bar{\ell}$ z ostatních klauzulí. Horn-SAT je lineární. DPLL opakuje jednotkovou propagaci a čisté literály, potom skončí na prázdné formuli jako splnitelné nebo prázdné klauzuli jako nesplnitelné, jinak větví podle proměnné p na $\varphi \wedge p$ a $\varphi \wedge \neg p$; obecně je exponenciální. Rezoluce z $\varphi \vee p$ a $\psi \vee \neg p$ odvodí $\varphi \vee \psi$; prázdná klauzule znamená spor. Pro důkaz φ z T hledáme rezoluční zamítnutí $T \cup \{\neg\varphi\}$. Formální důkaz je konečný syntaktický objekt z axiomů a pravidel; korektnost říká $T \vdash \varphi \Rightarrow T \models \varphi$, úplnost opačný směr. Tablo je strom s položkami $T\varphi$ a $F\varphi$; důkaz φ z T je sporné tablo z T s kořenem $F\varphi$, zamítnutí má kořen $T\varphi$. Větev je sporná, když obsahuje $T\psi$ i $F\psi$; tablo je sporné, když jsou sporné všechny větve. Věta o kompaktnosti: teorie má model právě tehdy, když každá její konečná část má model. Věta o úplnosti: sémantický důsledek a dokazatelnost se shodují, tedy $T \models \varphi$ právě tehdy, když $T \vdash \varphi$. Extenze T' teorie T v větším jazyce splňuje zahrnutí důsledků staré teorie; jednoduchá extenze nemění jazyk. Konzervativní extenze nepřidá nové důsledky ve starém jazyce; modelově konzervativní extenze navíc dovolí každý model T expandovat na model T' . Extenze o definice je konzervativní, každý model staré teorie má jednoznačnou

expansi a každá nová formule je v T' ekvivalentní starojazykové. Skolemizace PNF sentence s různými vázanými proměnnými odstraňuje existenční kvantifikátor $\exists y_i$ a nahrazuje y_i termem $f_i(x_1, \dots, x_{n_i})$ podle předchozích univerzálních proměnných.

3 Vzorce, případy a aplikace: Pro bezespornou výrokovou teorii T nad n proměnnými a s m modely platí: neekvivalentních výroků nad jazykem je 2^{2^n} ; pravdivých v T i lživých v T je vždy 2^{2^n-m} ; nezávislých je $2^{2^n} - 2 \cdot 2^{2^n-m}$. Jednoduchých extenzí až na ekvivalenci je 2^m , jedna je sporná, kompletních jednoduchých extenzí je m , T -neekvivalentních výroků je 2^m , pravdivá a lživá třída jsou po jedné a nezávislých tříd je $2^m - 2$. Aplikace kompaktnosti: spočetně nekonečný graf je bipartitní právě tehdy, když každý konečný podgraf je bipartitní; nestandardní model přirozených čísel vznikne přidáním konstanty větší než každý standardní numerál. První Gödelova věta: každá bezesporná rekurzivně axiomatizovaná extenze Robinsonovy aritmetiky má sentenci pravdivou ve standardních $\mathbb{N}$, ale nedokazatelnou v teorii; je-li standardní model modelem T , teorie není kompletní. $\text{Th}(\mathbb{N})$ není rekurzivně axiomatizovatelná. Rosserův trik dává v každé bezesporné rekurzivně axiomatizované extenzi Robinsonovy aritmetiky nezávislou sentenci. Druhá Gödelova věta: každá bezesporná rekurzivně axiomatizovaná extenze Peanovy aritmetiky nedokáže vlastní Con_T ; pokud je ZFC bezesporná, ZFC nedokáže Con_{ZFC} . Teorie je sporná právě tehdy, když nemá model, platí v ní spor nebo v ní platí všechny formule. Kompletní teorie je bezesporná a každou sentenci rozhoduje; ekvivalentně má právě jeden model až na elementární ekvivalenci. Struktury jsou elementárně ekvivalentní, když mají stejnou teorii. Teorie je rozhodnutelná, pokud algoritmus vždy rozhodne $T \models \varphi$, a částečně rozhodnutelná, pokud pro platné důsledky doběhne s odpovědí ano. Rekurzivně axiomatizovaná teorie má algoritmus pro členství mezi axiomy; taková teorie je částečně rozhodnutelná a je-li kompletní, je rozhodnutelná. Konečná struktura v konečném jazyce s rovností má rozhodnutelnou, tedy i rekurzivně axiomatizovatelnou teorii. Nerozhodnutelnost: neexistuje algoritmus pro rozhodnutí, zda diofantická rovnice $p(\bar{x}) = q(\bar{x})$ s přirozenými koeficienty má přirozené řešení, a neexistuje algoritmus rozhodující logickou platnost libovolné predikátové formule. Rozhodnutelné jsou například teorie čisté rovnosti, unárního predikátu, hustých lineárních uspořádání, komutativních grup, Booleových algeber, následníka s nulou a Presburgerova aritmetika. Nerozhodnutelné jsou Robinsonova a Peanova aritmetika, ZFC a teorie obecných grup.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Rozdíl mezi tautologií, splnitelností a důsledkem?
- Jak funguje rezoluční pravidlo a co dokazuje prázdná klauzule?
- Co říká věta o úplnosti?

Pozor (častá past): Nezaměňuj „splnitelná“ (aspoň jeden model) a „tautologie“ (všechny modely). Rezoluce dokazuje nesplnitelnost (hledá spor).

5 Část II: Společná informatika

Společná informatika je floor pro společné okruhy: cíl je umět každé téma bez nuly a rychle poskládat zelenou odpověď. Tady stačí kostra, definice, jeden příklad a hlavní pasti.

5.1 Automaty a jazyky

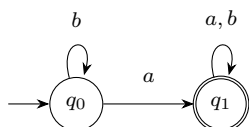
priorita: střední cíl: zelená (2 body)

Automaty a jazyky

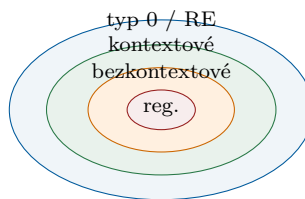
- Regulární jazyky
- regulární gramatiky
- deterministický a nedeterministický konečný automat
- regulární výrazy
- Bezkontextové jazyky
- bezkontextové gramatiky, jazyk generovaný gramatikou
- zásobníkový automat, třída jazyků přijímaných zásobníkovými automaty
- Rekurzivně spočetné jazyky
- gramatika typu 0
- Turingův stroj
- algoritmicky nerozhodnutelné problémy
- Chomského hierarchie
- schopnost zařazení konkrétního jazyka do Chomského hierarchie (zpravidla sestavení odpovídajícího automatu či gramatiky)

Učivo (jak na to):

Jazyk je množina slov. Čím silnější model výpočtu dovolím, tím složitější jazyky umím popsat: regulární přes konečnou paměť, bezkontextové přes zásobník, rekurzivně spočetné přes Turingův stroj.



DFA pro slova obsahující aspoň jedno a .



Chomského hierarchie jako vnořené třídy.

Odpověď podle bodů (cíle průchod: zelená = 2 body)

1 Definice. Jazyk nad abecedou Σ je podmnožina Σ^* . DFA je pětice $A = (Q, \Sigma, \delta, q_0, F)$, kde Q je konečná množina stavů, Σ konečná neprázdná abeceda, $\delta : Q \times \Sigma \rightarrow Q$ totální přechodová funkce, $q_0 \in Q$ počáteční stav a $F \subseteq Q$ přijímající stavy. Rozšířená funkce δ^* čte celé slovo a jazyk DFA je

$$L(A) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}.$$

Slovo w je přijato právě tehdy, když $w \in L(A)$. Jazyk je rozpoznatelný konečným automatem, pokud existuje konečný automat A s $L = L(A)$; třída těchto jazyků se značí $\mathcal{F}$ a nazývá se regulární jazyky.

Nedeterministický konečný automat s ϵ -přechody je pětice $N = (Q, \Sigma, \delta, q_0, F)$, kde $\delta : Q \times$

$(\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$ vrací množinu možných stavů. Místo jednoho počátečního stavu se někdy používá množina počátečních stavů $S_0 \subseteq Q$. ϵ -uzávěr množiny stavů je množina všech stavů dosažitelných po libovolně mnoha ϵ -krocích. NFA přijímá, když existuje aspoň jeden přijímající výpočet.

Regulární výraz nad Σ se definuje induktivně. Základní výrazy jsou ϵ s hodnotou $\{\epsilon\}$, $\emptyset$ s hodnotou $\emptyset$ a symbol $a \in \Sigma$ s hodnotou $\{a\}$. Operace jsou sjednocení $L(\alpha + \beta) = L(\alpha) \cup L(\beta)$, zřetězení $L(\alpha\beta) = L(\alpha)L(\beta)$, iterace $L(\alpha^*) = L(\alpha)^*$ a závorky, které hodnotu nemění. Priorita je $*$, potom zřetězení, potom $+$.

Formální gramatika je čtveřice $G = (V, T, P, S)$, kde V je konečná množina neterminálů, T neprázdná konečná množina terminálů, $S \in V$ počáteční symbol a P konečná množina pravidel. V obecné gramatice má pravidlo tvar $\beta A \gamma \rightarrow \omega$, kde $A \in V$ a $\beta, \gamma, \omega \in (V \cup T)^*$. Jazyk generovaný gramatikou je

$$L(G) = \{w \in T^* \mid S \Rightarrow_G^* w\}.$$

Jazyk neterminálu A je $L(A) = \{w \in T^* \mid A \Rightarrow_G^* w\}$. Bezkontextová gramatika má pravidla pouze tvaru $A \rightarrow \omega$.

Zásobníkový automat je sedmice $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, kde Γ je konečná zásobníková abeceda, $Z_0 \in \Gamma$ počáteční symbol zásobníku a

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \mathcal{P}_{\text{FIN}}(Q \times \Gamma^*).$$

Přechod $(q, \gamma) \in \delta(p, a, X)$ znamená: v jednom kroku přečti a nebo nic, přejdi do q a nahraď vrchol zásobníku X řetězcem γ ; nejlevější znak γ bude na vrcholu. Hrany v diagramu se značí “vstup, vrchol zásobníku $\rightarrow$ push řetězec”. Přijímání koncovým stavem:

$$L(P) = \{w \in \Sigma^* \mid (\exists q \in F)(\exists \alpha \in \Gamma^*)(q_0, w, Z_0) \vdash_P^* (q, \epsilon, \alpha)\}.$$

Přijímání prázdným zásobníkem:

$$N(P) = \{w \in \Sigma^* \mid (\exists q \in Q)(q_0, w, Z_0) \vdash_P^* (q, \epsilon, \epsilon)\}.$$

Turingův stroj je sedmice $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, kde $\Gamma \supseteq \Sigma$ je pásková abeceda, $Q \cap \Gamma = \emptyset$, $B \in \Gamma - \Sigma$ je blank a

$$\delta : (Q - F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

je částečná přechodová funkce. Pokud $\delta(q, X) = (p, Y, D)$, stroj v aktuální buňce přepíše X na Y , přejde do p a pohne hlavou doleva nebo doprava. Přijímaný jazyk je

$$L(M) = \{w \in \Sigma^* \mid (\exists p \in F)(\exists \alpha, \beta \in \Gamma^*) q_0 w \vdash_M^* \alpha p \beta\}.$$

Páska po přijetí nemusí být uklizená. Jazyk je rekurzivně spočetný, pokud ho přijímá nějaký TM. TM rozhoduje jazyk L , pokud $L = L(M)$ a pro každé $w \in \Sigma^*$ zastaví; takové jazyky jsou rekurzivní a platí rekurzivní $\subseteq$ rekurzivně spočetné.

2 Věty, konstrukce a hlavní algoritmy. Iterační lemma pro regulární jazyky: je-li L regulární, pak existuje $n \in \mathbb{N}$ takové, že každé $w \in L$ s $|w| \geq n$ lze rozložit $w = xyz$, kde $y \neq \epsilon$, $|xy| \leq n$ a pro všechna $k \geq 0$ platí $xy^kz \in L$. Důkaz neregularity $L_{01} = \{0^i 1^i \mid i \geq 0\}$: pro $w = 0^n 1^n$ leží xy celé v nulách, y je neprázdné, takže xz má méně nul než jedniček a neleží v L_{01} .

Pravá kongruence na Σ^* je ekvivalence $\sim$ splňující $u \sim v \Rightarrow uw \sim vw$ pro všechna $u, v, w \in \Sigma^*$. Má konečný index, pokud rozklad $\Sigma^* / \sim$ má konečně mnoho tříd; třída slova u se značí $[u]_\sim$. Příklad pravé kongruence: “končí stejným písmenem”. Relace “mají stejný počet znaků” nemá konečný index. Myhillova-Nerodova věta: pro jazyk L nad konečnou abecedou jsou ekvivalentní tvrzení, že L je rozpoznatelný konečným automatem, a že existuje pravá kongruence konečného indexu, jejíž některé třídy mají sjednocení přesně L . K důkazu neregularity se ukáže nekonečné

mnoho rozlišitelných prefixů. Typicky pro jazyk obsahující slova $ab^i c^i$: pokud má kongruence index m , množina $\{ab, abb, \dots, ab^{m+1}\}$ má dvě slova $ab^i \sim ab^j$, $i \neq j$; po připojení c^i jedno slovo v jazyce je a druhé není, spor.

Podmnožinová konstrukce převádí ϵ NFA N na ekvivalentní DFA D . Stav D jsou ϵ -uzavřené podmnožiny Q_N včetně případně $\emptyset$, počáteční stav je ϵ -closure(q_{0N}), přijímající stavy jsou

$$F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\},$$

a pro $S \subseteq Q_N$, $x \in \Sigma$ je

$$\delta_D(S, x) = \epsilon\text{-closure} \left(\bigcup_{p \in S} \delta_N(p, x) \right).$$

Tedy jazyk je rozpoznatelný ϵ NFA právě tehdy, když je regulární; NFA a DFA mají stejnou vyjadřovací sílu. Kleeneho věta: každý jazyk rozpoznatelný konečným automatem lze popsat regulárním výrazem a každý regulární výraz lze převést na ϵ NFA.

PDA a CFG popisují stejnou třídu. Věta: pro jazyk L jsou ekvivalentní tvrzení, že L je bezkontextový, že je přijímán nějakým PDA koncovým stavem, a že je přijímán nějakým PDA prázdným zásobníkem. Převod z koncového stavu na prázdný zásobník: na dno přidáme špunt, z přijímajících stavů vedeme ϵ -přechod do vypouštěcího stavu a tam zásobník vyprázdníme. Převod z prázdného zásobníku na koncový stav: na dno přidáme špunt a z každého stavu vedeme přechod do nového přijímajícího stavu, jakmile je špunt vidět. Konstrukce PDA z CFG $G = (V, T, P, S)$: vezmeme jeden stav q , vstupní abecedu T , zásobníkovou abecedu $V \cup T$, počáteční zásobníkový symbol S a přijímání prázdným zásobníkem. Pro každé pravidlo $A \rightarrow \beta$ dáme $\delta(q, \epsilon, A) \ni (q, \beta)$ a pro každý terminál a dáme $\delta(q, a, a) = \{(q, \epsilon)\}$. Na zásobníku se nedeterministicky generuje derivace a terminály se průběžně párují se vstupem.

Lemma o vkládání pro bezkontextové jazyky: je-li L bezkontextový, pak existuje n takové, že každé $w \in L$ s $|w| \geq n$ lze rozložit $w = u_1 u_2 u_3 u_4 u_5$, kde $u_2 u_4 \neq \epsilon$, $|u_2 u_3 u_4| \leq n$ a pro všechna $k \geq 0$ platí $u_1 u_2^k u_3 u_4^k u_5 \in L$. Idea: na nejdelší cestě derivačního stromu se opakuje neterminál; dva odpovídající podstromy lze opakovat nebo vynechat. Příklad: $L_{012} = \{0^i 1^i 2^i \mid i \geq 1\}$ není bezkontextový, protože pumpovaná část délky nejvýše n zasáhne nejvýše dva druhy symbolů a pumpování poruší rovnost tří počtů.

Chomského normální tvar pro CFG bez zbytečných symbolů má všechna pravidla tvaru $A \rightarrow BC$ nebo $A \rightarrow a$, kde $A, B, C \in V$ a $a \in T$; případně dovolí $S \rightarrow \epsilon$, pokud $\epsilon \in L(G)$ a S není na pravé straně pravidla. Převod: odstranit ϵ -pravidla pomocí nulovatelných symbolů a doplněných variant pravidel, odstranit jednotková pravidla přes jednotkové páry, odstranit zbytečné symboly nejprve negenerující a potom nedosažitelné, nahradit terminály v dlouhých pravých stranách novými neterminály a pravé strany délky aspoň tři rozdělit binárními pravidly.

Rozhodovací problém je množina kódovaných instancí s odpovědí ano/ne. Je rozhodnutelný, pokud existuje TM, který na každém vstupu zastaví a přijme právě instance s odpovědí ano. Redukce problému P_1 na P_2 je algoritmus R , který pro každou instanci zastaví a zachová odpověď: $P_1(w) = \text{ano} \Leftrightarrow P_2(R(w)) = \text{ano}$. Pokud se P_1 redukuje na P_2 a P_1 je nerozhodnutelný, pak je nerozhodnutelný i P_2 ; stejně, pokud P_1 není rekurzivně spočetný, není rekurzivně spočetný ani P_2 .

Diagonální jazyk L_d je množina binárních slov w , která kódují TM nepřijímající vlastní kód w . Věta: L_d není rekurzivně spočetný; jinak by spuštění přijímače L_d na vlastním kódu dalo paradox. Problém zastavení má vstup (M, w) a ptá se, zda stroj M na vstupu w zastaví. Věta: problém zastavení není rozhodnutelný, například redukcí z diagonálního jazyka. Univerzální jazyk $L_u = \{\langle M, w \rangle \mid w \in L(M)\}$ je rekurzivně spočetný, protože existuje univerzální Turingův stroj simulující M na w , ale není rekurzivní. Také problém prázdnosti jazyka daného TM není rozhodnutelný.

Chomského hierarchie: typ 0 má obecná pravidla $\alpha \rightarrow \omega$, kde α obsahuje neterminál, a odpovídá rekurzivně spočetným jazykům a TM. Typ 1 jsou kontextové gramatiky s pravidly $\gamma A \beta \rightarrow \gamma \omega \beta$,

kde $\omega \in (V \cup T)^+$; výjimka je $S \rightarrow \epsilon$, pokud se S nevyskytuje na pravé straně. Kontextové jazyky rozpoznává lineárně omezený automat. Typ 2 jsou bezkontextové gramatiky $A \rightarrow \omega$ a odpovídají nedeterministickým PDA. Typ 3 jsou regulární, pravé lineární gramatiky $A \rightarrow \omega B$ nebo $A \rightarrow \omega$, kde $\omega \in T^*$; neterminály odpovídají stavům konečného automatu a pravidla přechodům. Platí vnoření $\mathcal{L}_3 \subseteq \mathcal{L}_2 \subseteq \mathcal{L}_1 \subseteq \mathcal{L}_0$.

3 Detaily, uzávěry a zařazování jazyků. Regulární jazyky jsou uzavřené na sjednocení, průnik, rozdíl, doplněk, zřetězení, iteraci L^* , pozitivní iteraci L^+ , reverzi L^R , levý i pravý kvocient, homomorfismus a inverzní homomorfismus. Doplněk DFA vznikne prohozením přijímajících a nepřijímajících stavů. Pro průnik, sjednocení a rozdíl se použije součinný automat $(Q_1 \times Q_2, \Sigma, \delta, (q_{01}, q_{02}), F)$ s $\delta((p, q), x) = (\delta_1(p, x), \delta_2(q, x))$. Pro průnik je $F = F_1 \times F_2$, pro sjednocení $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$ a pro rozdíl $F = F_1 \times (Q_2 - F_2)$. Bezkontextové jazyky jsou uzavřené na sjednocení, zřetězení, iteraci a reverzi, nejsou obecně uzavřené na průnik, ale jsou uzavřené na průnik s regulárním jazykem. Uzávěry lze použít i negativně: kdyby z jazyka L regulárními operacemi vyšel známý neregulární jazyk jako $\{0^i 1^i\}$, pak L nemohl být regulární. Při zařazování konkrétního jazyka se hledá co nejpřesnější třída. Regularitu ukáže konečný automat nebo regulární gramatika; neregularitu pumping lemma nebo Myhillova-Nerodova věta. Bezkontextovost ukáže CFG nebo PDA; nebezkontextovost lemma o vkládání. Rekurzivní nebo rekurzivně spočetný jazyk se ukáže algoritmem nebo TM; negativní tvrzení se obvykle dokazují redukcí z L_d , L_u nebo problému zastavení. Příklad TM pro $\{0^n 1^n\}$: opakovaně jezdí po pásce, přepisuje jednu dosud neoznačenou nulu na X a jednu odpovídající jedničku na Y ; přijme, když zbyly jen X a Y a nuly a jedničky došly současně.

Kontextový příklad z poznámek: $L = \{a^i b^j c^k \mid 1 \leq i \leq j \leq k\}$ je kontextový, ale není bezkontextový. Generování lze popsat pravidly $S \rightarrow aSBC \mid aBC$, $B \rightarrow BBC$, $C \rightarrow CC$, $CB \rightarrow BC$, $aB \rightarrow ab$, $bB \rightarrow bb$, $bC \rightarrow bc$, $cC \rightarrow cc$. Pravidlo $CB \rightarrow BC$ není přímo kontextové, opraví se pomocnými označenými symboly, které mění písmena po jednom, například $CB \rightarrow \bar{C}\bar{B} \rightarrow \bar{C}B \rightarrow B\bar{B} \rightarrow BC$. Důležité je, že chybí pravidlo $cB \rightarrow cb$, takže po zahájení přepisu C už nelze přepisovat další B .

Vyzkoušej se (zakryj text a odpověz nahlas)

- Čím se liší DFA a NFA a proč mají stejnou vyjadřovací sílu?
- Jaký automat přijímá bezkontextové jazyky a k čemu je zásobník?
- Kam v hierarchii patří regulární, bezkontextové a rekurzivně spočetné jazyky?
- Uveď příklad nerozhodnutelného problému.

Pozor (častá past): Neříkej, že NFA je silnější než DFA. Je pohodlnější, ale pro konečné automaty popisuje stejnou třídu jazyků.

5.2 Algoritmy a datové struktury

priorita: střední cíl: zelená (2 body)

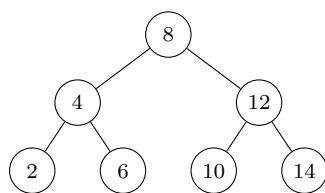
Algoritmy a datové struktury

- Časová složitost algoritmů
- časová a prostorová složitost algoritmu
- měření velikosti dat
- složitost v nejlepším, nejhorším a průměrném případě
- asymptotická notace
- Třídy složitosti
- třídy P a NP

- převoditelnost problémů, NP-těžkost a NP-úplnost
- příklady NP-úplných problémů a převodů mezi nimi
- Metoda rozděl a panuj
- princip rekurzivního dělení problému na podproblémy
- výpočet složitosti pomocí rekurentních rovnic
- Master theorem (kuchařková věta) (bez důkazu)
- aplikace
- Mergesort
- násobení dlouhých čísel
- Binární vyhledávací stromy
- definice vyhledávacího stromu
- operace s nevyvažovanými stromy
- AVL stromy (definice)
- Třídění
- primitivní třídící algoritmy (Bubblesort, Insertsort)
- Quicksort
- dolní odhad složitosti porovnávacích třídících algoritmů
- Grafové algoritmy
- prohledávání do šířky a do hloubky
- topologické třídění orientovaných grafů
- nejkratší cesty v ohodnocených grafech (Dijkstrův a Bellmanův-Fordův algoritmus)
- minimální kostra grafu (Jarníkův a Borůvkův algoritmus)
- toky v sítích (Ford-Fulkerson algoritmus)

Učivo (jak na to):

U algoritmů se zkouší hlavně rozpoznat správnou techniku, říct invariant nebo datovou strukturu a umět odhadnout složitost. U společné části stačí mít mapu pojmů a typické časy v ruce.



BST: vlevo menší, vpravo větší
 AVL: v každém uzlu rozdíl výšek nejvýše 1

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice. Velikost vstupu závisí na úloze: někdy je to počet prvků, jindy počet bitů čísel nebo jiný přirozený parametr. Pro vstup x je doba běhu $t(x)$ součet cen provedených instrukcí, případně ∞ . Časová složitost v nejhorším případě je

$$T(n) = \max\{t(x) \mid x \text{ je vstup velikosti } n\}.$$

Prostor běhu $s(x)$ je nejvyšší navštívená adresa minus nejnižší navštívená adresa plus jedna; prostorová složitost je

$$S(n) = \max\{s(x) \mid x \text{ je vstup velikosti } n\}.$$

Do času se obvykle nepočítá samotné načtení vstupu; do pracovního prostoru se nepočítá nezapisovaný vstup ani výstup, pokud se jen zapisuje a dál nečte. Pro cenu instrukcí je nutné omezit model paměťové buňky: jednotková cena omezuje šířku slova na W bitů, logaritmická cena počítá bity operandů včetně adres a relativní logaritmická cena se chová jednotkově pro polynomiálně velká čísla, ale zdrazuje extrémně velká čísla.

Nejhorsí případ bere maximum přes vstupy dané velikosti. Nejlepší případ bere minimum. Průměrný případ průměruje přes vstupy podle zvoleného rozdělení; bývá výstižný, ale těžší na odvození. Asymptoticky pro $f, g : \mathbb{N} \rightarrow \mathbb{R}$: $f \in O(g)$ právě tehdy, když existuje $c > 0$ takové, že pro všechna až na konečně mnoho n platí $0 \leq f(n) \leq cg(n)$; $f \in \Omega(g)$ analogicky znamená $f(n) \geq cg(n)$ pro nějaké $c > 0$ a od jistého n ; $\Theta(g) = O(g) \cap \Omega(g)$.

Třída P obsahuje rozhodovací problémy L , pro něž existuje algoritmus A a polynom p tak, že pro každý vstup x doběhne $A(x)$ do $p(|x|)$ kroků a odpoví $L(x)$. Třída NP obsahuje problémy s polynomiálně ověřitelným krátkým certifikátem: existuje verifikátor $V \in P$ a polynom g tak, že

$$L(x) = 1 \Leftrightarrow (\exists y, |y| \leq g(|x|)) V(x, y) = 1.$$

Platí $P \subseteq NP$, rovnost se neví. Polynomiální převod $A \leq_P B$ je funkce $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ spočitatelná v polynomiálním čase taková, že $A(\alpha) = B(f(\alpha))$ pro všechna α . Problém L je NP -těžký, pokud $K \leq_P L$ pro každý $K \in NP$, a NP -úplný, pokud je NP -těžký a zároveň $L \in NP$.

BVS je binární strom s unikátními klíči $k(v)$, kde pro každý vrchol v platí: každý klíč v levém podstromu $L(v)$ je menší než $k(v)$ a každý klíč v pravém podstromu $R(v)$ je větší než $k(v)$. AVL strom je hloubkově vyvážený BVS s podmínkou $|h(L(v)) - h(R(v))| \leq 1$ v každém vrcholu. Topologické uspořádání orientovaného grafu $G = (V, E)$ je lineární uspořádání vrcholů takové, že pro každou hranu $uv \in E$ je $u \leq v$.

2 Věty, algoritmy a vlastnosti. Převoditelnost je reflexivní, tranzitivní, není antisymetrická a existují i nepřevoditelné dvojice problémů; jde o částečné kvaziuspořádání problémů. Pokud $A \leq_P B$ a $B \in P$, pak $A \in P$. Pokud $A \leq_P B$, $B \in NP$ a A je NP -úplný, pak je B NP -úplný. Typické NP -úplné problémy: SAT, CNF-SAT, 3-SAT, 3,3-SAT, obvodový SAT, nezávislá množina, klika, k -obarvitelnost pro $k \geq 3$, Hamiltonovská cesta a kružnice, 3D-párování, součet podmnožiny, batoh, 2 loupežníci a nulajedničkové řešení soustavy lineárních rovnic v $\mathbb{Z}$. Klika se ptá na úplný podgraf na alespoň k vrcholech. Nezávislá množina se ptá na množinu aspoň k vrcholů, mezi nimiž nevede žádná hrana. SAT se ptá na splnitelnost výrokové formule, CNF-SAT na splnitelnost CNF formule, 3-SAT na formule s nejvýše třemi literály v klauzuli, 3,3-SAT navíc s nejvýše třemi výskyty každé proměnné. 3D-párování má množiny A, B, C a kompatibilní trojice $T \subseteq A \times B \times C$ a ptá se na perfektní podmnožinu trojic, v níž se každý prvek účastní právě jedné trojice.

Základní redukce: klika a nezávislá množina se převádějí doplňkem grafu, tedy prohozením hran a nehran. Při převodech SAT vyrábíme ekvivalentní formuli. Převod CNF-SAT na 3-SAT štěpí dlouhou klauzuli pomocí nových proměnných, například $(\ell_1 \vee \dots \vee \ell_r)$ pro $r > 3$ nahradí $(\ell_1 \vee \ell_2 \vee z_1) \wedge (\neg z_1 \vee \ell_3 \vee z_2) \wedge \dots \wedge (\neg z_{r-3} \vee \ell_{r-1} \vee \ell_r)$; počet nových proměnných a klauzulí je polynomiální v délce formule. Převod 3-SAT na 3,3-SAT nahradí proměnnou s $k > 3$ výskyty novými proměnnými $x_1, \dots, x_k$ a vynutí jejich ekvivalenci cyklem implikací $x_1 \Rightarrow x_2, \dots, x_k \Rightarrow x_1$, takže každá nová proměnná má omezený počet výskytů. Varianta 3,3-SAT* navíc omezuje výskyty každého literálu na nejvýše dva.

Převod 3-SAT na nezávislou množinu: pro každou z k klauzulí vytvoříme trojúhelník vrcholů označených literály klauzule, u kratších klauzulí přebytečné vrcholy smažeme, a přidáme hrany mezi konfliktními literály. Formule je splnitelná právě tehdy, když má graf nezávislou množinu velikosti aspoň k : ze splňujícího ohodnocení vybereme v každé klauzuli pravdivý literál; naopak nezávislá množina velikosti k vybírá právě jeden nekonfliktní literál z každé klauzule. Převod nezávislé množiny na 3-SAT: proměnná x_j říká, zda je vrchol j vybrán, každá hrana dává klauzuli $(\neg x_i \vee \neg x_j)$ a velikost k se vynutí pomocnou tabulkou proměnných y_{ij} , v níž je v

každém řádku právě jedna jednička, v každém sloupci nejvýše jedna jednička a implikace $y_{ij} \rightarrow x_j$ propojí tabulku s výběrem vrcholů.

Rozděl a panuj: problém se rekurzivně dělí na nezávislé podproblémy, ty se řeší až do triviální konstantní velikosti a výsledky se skládají. Rekurenci lze řešit dosazováním, součtem nákladů po úrovních nebo Master theorem. Master theorem pro

$$T(n) = aT(n/b) + \Theta(n^c), \quad T(1) = 1,$$

kde $a \in \{1, 2, \dots\}$, $b > 1$, $c \geq 0$, říká: pokud $a/b^c = 1$, pak $T(n) = \Theta(n^c \log n)$; pokud $a/b^c < 1$, pak $T(n) = \Theta(n^c)$; pokud $a/b^c > 1$, pak $T(n) = \Theta(n^{\log_b a})$.

Mergesort: pro posloupnost délky nejvýše jedna vrátí vstup, jinak setřídí levou a pravou polovinu a lineárně je slije. Rekurence $T(n) = 2T(n/2) + \Theta(n)$ dává $\Theta(n \log n)$, protože v každém z $\log n$ pater se slévá celkem n prvků. Karacubovo násobení: pro n -ciferná čísla v základu z napiš $x = az^{n/2} + b$, $y = cz^{n/2} + d$. Pak

$$xy = acz^n + (ad + bc)z^{n/2} + bd, \quad ad + bc = (a + b)(c + d) - ac - bd.$$

Stačí tři rekurzivní násobení, rekurence $T(n) = 3T(n/2) + cn$ má řešení $\Theta(n^{\log_2 3})$; součtem pater $\sum_{i=0}^{\log_2 n} n(3/2)^i \in \Theta(n(3/2)^{\log_2 n})$.

Operace v nevyvažovaném BVS: Find porovnává hledaný klíč s aktuálním vrcholem a jde vlevo nebo vpravo; Insert vloží uzel tam, kde by se klíč našel, pokud by ve stromu byl; Delete smaže nalezený vrchol, u jednoho syna připojí syna pod otce a u dvou synů vrchol prohodí s nejlevějším vrcholem v pravém podstromu, případně symetricky. Čas je úměrný výšce stromu, bez vyvažování může být lineární. Věta: AVL strom na n vrcholech má hloubku $\Theta(\log n)$, proto operace po vyvážení rotacemi běží logaritmičtě.

BubbleSort opakovaně prochází pole zleva doprava, porovnává sousední prvky a prohodí je, jsou-li ve špatném pořadí; skončí po průchodu bez prohození. InsertSort udržuje levou část pole setříděnou, bere další prvek z neseříděné části, posune větší prvky vpravo a vloží ho na správné místo. Quicksort: pro $n \leq 1$ vrátí vstup, jinak zvolí pivota p , rozdělí prvky na menší než p , rovné p a větší než p , rekurzivně setřídí krajní části a spojí výsledek. Náhodný pivot dává střední čas $\Theta(n \log n)$; nejhorší čas je $\Theta(n^2)$. Zdůvodnění průměru: s pravděpodobností $1/2$ je pivot v prostředních dvou čtvrtinách pořadí, tedy skoromedián, a očekávaný počet pokusů na skoromedián je konstantní; po takovém kroku se největší podproblém zmenší na nejvýše $3/4$.

Dolní odhad pro porovnávací třídění: rozhodovací strom algoritmu musí mít list pro každou z $n!$ permutací. Hloubka binárního stromu s $n!$ listy je aspoň $\log_2(n!) \geq \log_2(n^{n/2}) = (n/2) \log n$, proto každý porovnávací třídící algoritmus potřebuje $\Omega(n \log n)$ porovnání v nejhorším případě.

BFS používá frontu. Začne otevřením počátečního vrcholu, opakovaně odebere vrchol z fronty, prohlédne sousedy, dosud nenavštívené označí jako otevřené a vloží do fronty, a zpracovaný vrchol zavře. Končí s prázdnou frontou, běží v $\Theta(n + m)$ a najde z počátečního vrcholu cesty s nejmenším počtem hran, tedy nejkratší cesty v grafech s jednotkovými délkami. DFS lze implementovat rekurzí nebo zásobníkem: otevři v , pro každou hranu vw spusť DFS na neviděném w , nakonec v zavři. Jedno DFS navštíví dosažitelné vrcholy, opakované DFS přes všechny neviděné vrcholy projde všechny komponenty; ekvivalentně lze přidat superzdroj. Složitost je $\Theta(n + m)$.

Topologické třídění existuje právě pro DAG a může být nejednoznačné. Lze ho konstruovat postupným odtrháváním zdrojů: najít zdroj, dát ho do pořadí a odstranit. Efektivně se použije DFS: opakované DFS zavírá vrcholy v pořadí opačném topologickému uspořádání. V DAGu leží vrchol x na nějaké orientované cestě z u do v právě tehdy, když je x dosažitelný z u a v je dosažitelný z x . Proto stačí jedno DFS/BFS z u a jedno DFS/BFS z v v obráceném grafu a vzít průnik označených vrcholů; čas je $\Theta(n + m)$.

Dijkstra hledá nejkratší cesty z počátečního vrcholu v grafu s kladnými, respektive nezápornými délkami hran. Inicializuje $h(v_0) = 0$, ostatní $h(v) = +\infty$, otevře v_0 , a dokud existují otevřené

vrcholy, vybere otevřený vrchol s nejmenším h , uzavře ho a relaxuje následníky: pokud $h(w) > h(v) + l(v, w)$, sníží $h(w)$ a uloží předchůdce. Každý vrchol se zavře jednou. Bez haldy je hledání minima $\Theta(n^2)$. S abstraktní prioritní frontou je složitost $O(nT_I + mT_D + nT_X)$ pro Insert, Decrease a ExtractMin; s binární haldou $O((n+m) \log n)$, lepší husté varianty používají d -regulární nebo Fibonacciho haldy.

Bellmanův-Fordův algoritmus je relaxační algoritmus pro grafy bez záporných cyklů. Obecný relaxační algoritmus inicializuje všechny vrcholy jako nenalezené s $h = +\infty$, otevře v_0 s $h(v_0) = 0$, a dokud existují otevřené vrcholy, vezme jeden otevřený vrchol, relaxuje jeho následníky a vrchol uzavře. Bellman-Ford bere nejstarší otevřený vrchol z fronty. Fáze F_0 otevře v_0 , fáze F_i zavírá vrcholy otevřené ve F_{i-1} a otevírá jejich následníky. Invariant: na konci F_i je $h(v)$ délka nejkratšího v_0v -sledu s nejvýše i hranami. Složitost je $\Theta(nm)$.

Pro minimální kostru platí řezové lemma: je-li G graf s unikátními vahami, R elementární řez a e nejlehčí hrana řezu, pak e leží v každé minimální kostře. Jarníkův algoritmus začne stromem T s libovolným vrcholem v_0 a bez hran, opakovaně přidá nejlehčí hranu mezi T a zbytkem grafu, dokud lze. Klasická implementace má $O(nm)$. Varianta podle Dijkstry udržuje haldy aktivních hran aktuálního řezu, přidává minimum a pro jeden vnější vrchol nechává jen nejlehčí aktivní hranu; s haldou běží $O(m \log n)$. Borůvkův algoritmus začne izolovanými vrcholy a v každé iteraci pro každou komponentu přidá nejlehčí hranu do zbytku grafu. Komponenty a hrany se zpracují v $O(m)$ na iteraci a po nejvýše $\lceil \log_2 n \rceil$ iteracích skončí, tedy $O(m \log n)$.

Fordův-Fulkersonův algoritmus pro toky: cesta je nenasycená, pokud všechny její hrany mají kladnou rezervu. Dokud existuje nenasycená cesta ze zdroje do stoku, spočti rezervu cesty jako minimum rezerv hran a uprav tok po cestě: v protisměru odečti, co lze, a zbytek přičti po směru. Pokud se algoritmus zastaví, vydá maximální tok. Pro racionální kapacity se zastaví a vydá maximální tok i minimální řez; racionální kapacity lze převést na celočíselné a operace z celých kapacit nevytvoří necelé toky. Pro iracionální kapacity může volba cest selhat. Důsledek: velikost maximálního toku je rovna kapacitě minimálního řezu. Sít s celočíselnými kapacitami má celočíselný maximální tok a Ford-Fulkerson ho najde.

3 Další detaily a exam-ready poznámky. Při měření algoritmů je nutné říct, co je n , zda jde o nejhorší, nejlepší nebo průměrný případ, a jaký model ceny instrukcí používáme. U číselných úloh není totéž počet čísel a počet bitů vstupu. U prostorové složitosti se obvykle měří pracovní prostor, nikoli neměnný vstup a jednorázově zapisovaný výstup.

Převody zachovávají ano i ne instance. Při důkazu NP-úplnosti nového problému B se standardně ukáže $B \in NP$ a vezme se známý NP-úplný problém A , pro který se sestojí $A \leq_P B$. NP-těžký problém nemusí ležet v NP, například může být optimalizační nebo nerozhodnutelný.

U grafových algoritmů značme $n = |V|$, $m = |E|$. BFS a DFS pracují v lineárním čase vzhledem k velikosti grafu. Dijkstra nesmí být použit na záporné hrany. Bellman-Ford zvládá záporné hrany, ale ne záporný cyklus dosažitelný ze zdroje, protože nejkratší cesta pak není dobře definovaná. U topologického třídění cyklus znemožňuje lineární uspořádání respektující orientované hrany.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co musí platit, aby byl problém NP-úplný?
- Jak spočítáš mergesort přes rekurentní rovnici?
- Jaký je rozdíl mezi BST a AVL stromem?
- Kdy použiješ Dijkstru a kdy Bellman-Forda?

Pozor (častá past): NP-těžký problém nemusí být v NP. NP-úplnost vyžaduje obojí: patří do NP a každý problém z NP se na něj polynomiálně převede.

5.3 Programovací jazyky

priorita: střední cíl: zelená (2 body)

Programovací jazyky

Některé následující body definují varianty požadavků pro různé individuální volby povinně volitelných předmětů. Vyžaduje se zvládnutí všech bodů bez označení ∇ a zvládnutí všech bodů s označením ∇ pro jeden z jazyků C#, C++ nebo Java.

- Koncepty pro abstrakci, zapouzdření a polymorfismus.
- související konstrukty programovacích jazyků
- třídy, rozhraní, metody, datové položky, dědičnost, viditelnost
- (dynamický) polymorfismus, statické a dynamické typování
- jednoduchá dědičnost
- ∇ virtuální a nevirtuální metody v C++ a C#
- vícenásobná dědičnost a její problémy
- ∇ interfaces v C#
- implementace rozhraní (interface)
- vhodné použití uvedených konceptů
- Primitivní a objektové typy a jejich reprezentace.
- číselné a výčtové typy
- ∇ hodnotové a referenční typy v C#
- ∇ reference, imutabilní typy a boxing v C# a Javě
- Generické typy a funkcionální prvky (procedurálních programovacích jazyků).
- ∇ generické typy v Javě a C# (bez omezení typových parametrů)
- ∇ typy reprezentující funkce v C++, C#, nebo Javě
- lambda funkce a funkcionální rozhraní
- Manipulace se zdroji a mechanismy pro ošetření chyb.
- správa životního cyklu zdrojů v případě výskytu chyb
- ∇ using v C#
- konstrukce pro obsluhu a propagaci výjimek
- Životní cyklus objektů a správa paměti.
- alokace (alokace statická, na zásobníku, na haldě)
- inicializace (konstruktory, volání zděděných konstruktorů)
- destrukce (destruktory, finalizátory)
- explicitní uvolňování objektů, reference counting, garbage collector
- Vlákna a podpora synchronizace.
- reprezentace vláken v programovacích jazycích
- specifikace funkce vykonávané vláknem a základní operace na vlákny
- časově závislé chyby a mechanismy pro synchronizaci vláken
- Implementace základních prvků objektových jazyků.
- základní objektové koncepty v konkrétním jazyce
- implementace a interní reprezentace primitivních typů
- implementace a interní reprezentace složených typů a objektů
- implementace dynamického polymorfismu (tabulka virtuálních metod)
- Nativní a interpretovaný běh, řízení překladu a sestavení programu.
- reprezentace programu, bytecode, interpret jazyka
- just-in-time (JIT) a ahead-of-time (AOT) překlad
- proces sestavení programu, oddělený překlad, linkování
- staticky a dynamicky linkované knihovny
- běhové prostředí procesu a vazba na operační systém

Učivo (jak na to):

Drž se C#. Odpověď stav na třech vrstvách: jazykové konstrukty, běhová reprezentace a bezpečné zacházení se zdroji a chybami.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice. Abstrakce odděluje rozhraní od implementačních detailů, zapouzdření chrání stav objektu pomocí viditelnosti a invariantů, polymorfismus dovoluje pracovat se stejným rozhraním nad různými konkrétními typy. V C# i C++ existují třídy a struktury; v C++ musí deklarace končit středníkem a rozdíl `class/struct` neurčuje uložení, zatímco v C# jsou `struct` hodnotové typy. C# podporuje jednoduchou dědičnost tříd a vícenásobnou implementaci rozhraní. Rozhraní v C# se konvenčně jmenuje s počátečním I, obsahuje metody a podobné členy včetně properties, ale ne fields. Instance samotného rozhraní neexistuje, existuje proměnná typu rozhraní, například `IPostovniZvire zvire = new Kachna();`. Třída může implementovat více rozhraní a rozhraní může dědit od jiného rozhraní. V C++ se rozhraní typicky modeluje dědičností s virtuálními metodami a při dědění lze určit viditelnost.

Viditelnost v C#: `public` znamená neomezený přístup, `private` přístup jen v daném typu včetně jeho vnořených typů, `protected` přístup v daném typu a potomcích. Výchozí viditelnost je `private` ve třídě a `public` v rozhraní. Statické typování zná typy proměnných při překladu a dovoluje statickou kontrolu metod. Dynamické typování určuje typ za běhu; chybné volání selže až za běhu, v C# lze použít `dynamic`. Použití `object` nebo obecného rozhraní je slabší forma práce přes obecnější typ. Statický polymorfismus je přetížení metody nebo operátoru, případně zakrytí rodičovské metody. Dynamický polymorfismus používá virtuální metody a rozhoduje podle skutečné třídy instance; požadavek rozhraní může být uspokojen i virtuální metodou a metoda rozhraní může být implementována explicitně.

Vícenásobná dědičnost přináší diamantový problém: pokud mají rodiče společného předka a oba implementují jeho metodu, není zřejmé, která implementace se má zavolat. C# proto nepodporuje vícenásobnou dědičnost tříd, ale podporuje vícenásobnou implementaci rozhraní. U návrhu v C# platí rozdíl mezi abstraktní metodou a rozhraním: rozhraní vynucuje veřejný kontrakt hned, abstraktní metoda může být i `protected`; virtuální metoda slibuje rozšiřitelnost opt-out pomocí `sealed`, rozhraní opt-in; rozhraní nemá fields kvůli vícenásobné dědičnosti a abstraktní třída neřeší vícenásobné kontrakty stejně volně.

Číselné typy C# zahrnují `byte`, `sbyte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `nint`, `nuint`, `float`, `double` a `decimal`; `nint/nuint` mají nativní velikost platformy. Výčtový typ se definuje například `enum Season { Spring, Summer, Autumn, Winter }`, může mít zadaný podkladový integrální typ, například : `ushort`, a hodnoty položek lze explicitně určit. V C++ existují `raw pointers`, reference včetně `const` a `rvalue` referencí a `smart pointers`.

C# typy se základně dělí na `pointery`, hodnotové typy a referenční typy. Hodnotové typy zahrnují `enumy` a struktury včetně jednoduchých typů `Int32`, `Int64`, `Double`, `Boolean`, `Char`, `nullable` typů a uživatelských `struct`. Referenční typy zahrnují třídy, například `string`, rozhraní, pole a delegáty. Proměnná hodnotového typu obsahuje hodnotu dané velikosti; u struktury může `new` jen zavolat konstruktor bez haldové alokace. Proměnná referenčního typu obsahuje adresu objektu na managed haldě, alokuje se typicky pomocí `new`; skutečnou adresu nelze spolehlivě získat, protože GC může objekty přesouvat.

2 Hlavní mechanismy a vlastnosti. Managed halda C# je spravovaná garbage collectorem, který nepoužívá prosté počítání referencí jako hlavní princip, ale graf dosažitelnosti. Objekt na haldě má typicky overhead kolem 16 B: `syncblock` a ukazatel na typ. `Syncblock` obsahuje informace pro zamykání, například vlastníci vlákna, počítadlo rekurzivního zamčení a seznam čekajících vláken, a také část pro `condition variable` se seznamem uspaných vláken. Ukazatel na typ vede zjednodušeně na `metadata` nebo tabulku metod typu; `GetType()` z nich vrací instanci `System.Type`.

Boxing nastane, když se hodnotový typ přiřadí do proměnné typu `object` nebo jiného refe-

renčního obalu. Alokuje se instance hodnotového typu na GC haldě včetně overheadu. V C# je například `int` zároveň `System.Int32`; v Javě jsou `int` a `Integer` odlišné typy. Unboxing z referenčního obalu získá zpět hodnotu. `object` není vhodný pro slabé typování; pokud chceme hodnotový typ sdílet referencí, bývá čistší jednodušší třída. Imutabilita se podporuje `readonly` fieldy, do nichž se zapisuje jen v konstruktoru nebo inicializaci, skrytým setterem property a `readonly struct`.

Tracking reference v C# se používají u parametrů `ref`, `out` a `in`. `ref` se píše v hlavičce i při volání a vyžaduje inicializovanou proměnnou. `out` je na úrovni CIL podobné, ale volající proměnná nemusí být inicializovaná; ve funkci se s ní zachází jako s neinicializovanou a před normálním návratem musí být zapsána. Při výjimce zápis povinný není. `if (int.TryParse(s, out int x))` deklaruje `x` před `if`. `in` předává `readonly` referenci a lze ho volat i bez klíčového slova; hodí se hlavně pro výkonnostní optimalizace velkých hodnotových typů. Podobná varianta je `ref readonly`. U referenčních typů obvykle tracking reference nedává smysl; výjimkou je například zvětšení pole, kdy chceme přiřadit novou referenci zpět volajícímu.

Generický typ v C# parametrizuje typ, například `class GenericList<T> {}`; při vytvoření instance se typový parametr uvádí, u generických metod ho často odvodí překladač. V C++ se statický polymorfismus často dělá šablonami, například `template<typename T> const T& max(const T& a, const T& b)`; v C# odpovídá generická metoda s případným omezením, například na `Comparable`. Funkce jako hodnoty se v C# reprezentují delegáty. Delegátní typ lze definovat například `public delegate void Callback(string message)`; přiřadit metodu kompatibilní signatury a volat jej jako funkci. Delegát na instanční metodu obsahuje ukazatel na funkci a na `this`; u statické metody je `this` `null`. Delegáti jsou `immutable`, mohou být generičtí a existují předdefinované typy `Action<...>`, `Func<..., TResult>` a `Predicate<T>`. V C++ lze použít šablony, function pointer nebo `std::function`. Lambda v C# má tvar `(parameters) => expression` nebo blok s příkazy; může být `static`, jinak může zachytávat proměnné z okolí podobně jako `capture by reference`, a je přiřaditelná do delegáta. V Javě se používají funkcionální rozhraní.

Vícerozměrná pole v C#: zubaté pole je pole polí, například `int[][] a`, každá řádka je samostatné pole a má vlastní overhead; obdélníkové pole se píše `int[,] a = new int[2, 3]` a indexuje `a[0, 1]`. Zubatá pole bývají rychlejší, obdélníková mohou být paměťově úspornější a lépe se zapisují. Indexer je speciální property, například `public T this[int i] { get ... set ... }`. Nullable hodnotové typy mají `HasValue` a `Value`. C# dovoluje přetížít operátory a definovat implicitní i explicitní konverze.

Správa zdrojů: RAII v C++ znamená, že zdroj se získá v konstruktoru a uvolní v destrukturu, takže je uvolnění vázané na život objektu; typické příklady jsou `lock_guard` pro zámek a smart pointer pro `new/delete`. V C# slouží `using (var f = new FileStream(...)) {...}` jako syntaktická zkratka pro `try/finally`, ve `finally` se volá `Dispose`. Výjimky se vyhazují `throw new Ex(...)` a zachycují bloky `try/catch/finally`; více `catch` bloků rozlišuje typy výjimek a stejnou výjimku lze znovu vyhodit pomocí `throw`.

Alokace může být statická, na zásobníku nebo na haldě. Statická alokace vzniká během překladu pro data dostupná po celý běh programu. Zásobníková alokace vzniká při volání funkcí pro lokální proměnné, po návratu se uvolní, je rychlá, ale zásobník má omezenou velikost; v poznámkách je řečeno, že funguje zásobníkově, tedy LIFO. Halda se používá pro sdílené objekty, polymorfismus, velká data nebo situace, kdy velikost a životnost nevyhovují zásobníku. V C++ se haldová paměť získaná `new` musí právě jednou uvolnit `delete`, nebo se použijí smart pointer: `unique_ptr` uvolní při opuštění scope, `shared_ptr` počítá reference a uvolní při nule, cykly řeší `weak_ptr`. V C existuje `malloc/free`. V C# jsou instance referenčních typů a jejich fieldy na GC haldě.

Konstruktor v C# má viditelnost a název třídy a volá se s `new`. Konstruktor předka se volá přes `: base(args)` a volá se vždy, implicitně bez parametrů; konstruktor předka proběhne před konstruktorem potomka. V C++ je syntaxe podobná, jen se místo `base` píše název

rodičovské třídy. Finalizátory v C# jsou pro specifické uvolnění unmanaged zdrojů a nejsou běžnou náhradou `Dispose`. V C++ jsou destruktory zásadní pro vlastnictví zdrojů a kopírování objektů, viz rule of five; базовé třídy určené k polymorfnímu mazání mají mít virtuální destruktory.

Vlákna v C# reprezentuje `System.Threading.Thread`, ale vytváření vláken je drahé, proto se často používá `ThreadPool`. `Thread` dostane delegáta bez parametru nebo s jedním parametrem typu `object`, spouští se `Start()` a čeká se `Join()`. Race condition nastává, když výsledek závisí na pořadí událostí, které nemáme pod kontrolou. Základní řešení je mutual exclusion zámkem. V C# lze použít `Monitor.Enter(obj)` a `Monitor.Exit(obj)` nebo zkratku `lock(obj)` {...}. Zámek je reentrantní pro stejné vlákno, takže se musí odemknout stejný počet krát. `Synchblock` existuje u referenčních objektů; jako zámek je vhodné používat soukromý field typu `object`, ne `lock(this)`. Condition variable v `synchblocku` umožňuje `Monitor.Wait(obj)`, `Pulse(obj)` a `PulseAll(obj)`; tyto operace musí být uvnitř `lock` bloku na stejném objektu. Semafor omezuje počet vláken, která mohou současně pracovat se zdrojem.

Dynamický polymorfismus se implementuje tabulkou virtuálních metod. Každý typ s virtuálními metodami má VMT, při dědění se tabulka kopíruje a prodlužuje, u `override` se přepíše odkaz na implementaci. Volaná implementace tak závisí na skutečné třídě vytvořené instance, ne jen na statickém typu proměnné.

Nativní běh znamená, že se zdrojový kód přeloží na strojový kód přímo spustitelný na dané platformě. Interpretovaný běh zpracovává kód interpretem. C++ překládá soubory `.cpp` do objektových souborů `.o/.obj` se strojovým kódem a linker je spojí do spustitelného souboru. Strojový kód cílí konkrétní platformu; pro jinou platformu je obvykle nutný nový překlad, i když existují techniky jako Apple fat binary, které nesou více verzí najednou a řeší hlavně zpětnou kompatibilitu. Překladač používá hlavičkové soubory knihoven.

C# používá soubory `.cs` a projekt `.csproj`; build systém spustí kompilátor `csc`, výstupem je assembly s CIL kódem. Ke spuštění je potřeba CLR, tedy virtual machine a runtime pro managed kód. CLR typicky používá JIT, který CIL překládá za běhu po metodách na strojový kód dané platformy; interpretace CIL by byla pomalá. Optimalizace dělá JIT, ale má omezený čas, aby start programu nebyl pomalý. Alternativou je AOT překlad, kdy je v executable kromě assembly i strojový kód pro podporovanou platformu, ale některé vlastnosti pak nemusí fungovat. (Jen pro srovnání, neučit se zvláště: Java funguje obdobně - bytecode nad JVM.) Všechny jazyky .NET se překládají do CIL. Proces sestavení nativního programu zahrnuje preprocesor, kompilátor, assembler a linker. Knihovna je statická nebo dynamická sada binárek; statickou knihovnu použije linker, dynamickou až loader. Loader načte program do paměti. V C# velkou část řeší CLR a používají se DLL.

3 Doplnky, příklady a návrhové vzory. Běhové prostředí procesu je vazba programu na operační systém. Program je soubor, proces je spuštěná instance spravovaná OS. OS poskytuje virtuální paměť, soubory, periferie a další systémové služby; v C# je bezprostředním běhovým prostředím CLR nad OS.

U rozhraní v C# je vhodné psát `public` explicitně, i když členy rozhraní jsou defaultně veřejné. Názvy parametrů v metodách rozhraní dokumentují kontrakt. Veřejná metoda může vrátit delegáta ukazujícího na privátní statickou metodu, protože viditelnost se řeší při vytvoření delegáta. U delegáta na instanční metodu struktury se `this` zkopíruje, tedy prakticky zaboxuje. Návrhové vzory z poznámek: Decorator obalí objekt, přijme původní objekt v konstruktoru a má stejné rozhraní, ale přidává chování. Adapter převádí rozhraní jedné třídy na rozhraní jiné, aby se objekt dal použít na místě očekávaného typu. Factory vyrábí instance, často podle parametrů; abstract factory používá virtuální vyráběcí metody a různé továrny vyrábějí související rodiny objektů. Visitor řeší speciální operace nad potomky společné báze: rozhraní visitora má metody pro jednotlivé typy, báze má `Accept(Visitor v)` a potomek `A` volá `v.VisitA(this)`. Místo `switch` podle typu tak dispatch zůstává objektový.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaký je rozdíl mezi hodnotovým a referenčním typem v C#?
- Co v C# znamená **virtual**, **override** a **interface**?
- Kdy použiješ **using** a proč nestačí garbage collector?
- Jak spolu souvisí IL, CLR a JIT?

Pozor (častá past): V C# není vícenásobná dědičnost tříd. Vícenásobně lze implementovat rozhraní.

5.4 Architektura počítačů a operačních systémů

priorita: střední cíl: zelená (2 body)

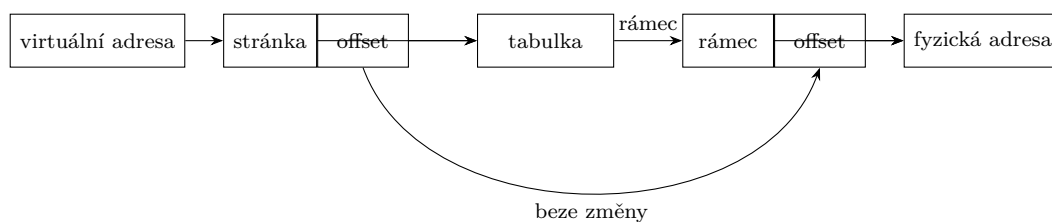
Architektura počítačů a operačních systémů

- Základní architektura počítače, reprezentace čísel, dat a programů.
- reprezentace a přístup k datům v paměti, adresa, adresový prostor
- ukládání jednoduchých a složených datových typů
- základní aritmetické a logické operace
- Instrukční sada, vazba na prvky vyšších programovacích jazyků.
- Implementovat běžné programové konstrukce vyšších jazyků (přiřazení, podmínka, cyklus, volání funkce) pomocí instrukcí procesoru
- Zapsat běžnou konstrukci vyššího jazyka (přiřazení, podmínka, cyklus, volání funkce), která odpovídá zadané sekvenci (vysvětlených) instrukcí procesoru
- Podpora pro běh operačního systému.
- privilegovaný a neprivilegovaný režim procesoru
- jádro operačního systému
- Rozhraní periferních zařízení a jejich obsluha.
- Popsat roli řadiče zařízení při programem řízené obsluze zařízení (PIO), pro zadané adresy a funkce vstupních a výstupních portů implementovat programem řízenou obsluhu zadaného zařízení (myš, disk)
- Popsat roli přerušování při programem řízené obsluze zařízení (PIO), na úrovni vykonávání instrukcí popsat reakci procesoru (hardware) a operačního systému (software) na žádost o přerušování
- Základní abstrakce, rozhraní a mechanismy OS pro běh programů, sdílení prostředků a vstup/výstup.
- neprivilegované (uživatelské) procesy
- sdílení procesoru
- procesy, vlákna, kontext procesu a vlákna
- přepínání kontextu, kooperativní a preemptivní multitasking
- plánování běhu procesů a vláken, stavy vlákna
- sdílení paměti
- Vysvětlit rozdíl mezi virtuální a fyzickou adresou a identifikovat, zda se v zadaném kontextu či fragmentu kódu používá virtuální nebo fyzická adresa
- Na zadaném příkladu identifikovat a vysvětlit význam komponent virtuální a fyzické adresy (číslo stránky, číslo rámce, offset)
- Pro konkrétní adresy a obsah jednoúrovňové stránkovací tabulky řešit úlohy překladu adres
- Vysvětlit roli virtuálních adresových prostorů v ochraně paměti procesů a vláken
- sdílení úložného prostoru
- soubory, analogie s adresovým prostorem
- abstrakce a rozhraní pro práci se soubory

- Paralelismus, vlákna a rozhraní pro jejich správu, synchronizace vláken.
- časově závislé chyby (race conditions)
- kritická sekce, vzájemné vyloučení
- základní synchronizační primitiva, jejich rozhraní a použití
- zámky
- aktivní a pasivní čekání

Učivo (jak na to):

Procesor vykonává instrukce nad registry a pamětí, OS odděluje uživatelský kód od jádra a dává procesům iluze: vlastní CPU, vlastní adresový prostor a soubory jako pojmenované úložiště.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice. Harvardská architektura odděluje procesor, kódovou paměť s instrukcemi, datovou paměť s proměnnými a vstupně-výstupní zařízení. CPU vykonává příkazy, z datové paměti čte a do ní zapisuje, zatímco kódová paměť je v základním modelu zapisována zvenku. Von Neumannova architektura má jednu operační paměť pro kód i data; program counter, také instruction pointer, ukazuje na instrukce a adresy proměnných leží v jiných oblastech. Výhoda je flexibilita a možnost zavést do RAM počáteční program z nevolatilní paměti, nevýhodou je i bezpečnostní riziko, pokud útočník uloží spustitelné byty do dat a zařídí jejich spuštění. Operační paměť RAM je volatilní.

Data i programy se reprezentují binárně. Každému bytu paměti se přiřazuje adresa, typicky n -bitové unsigned číslo, a tím vzniká adresový prostor. Paměť 256 B má 8bitový adresový prostor $0, \dots, 2^8 - 1$, 32bitový adresový prostor přímo pokryje 4 GB adres a pro větší přímo adresovatelný prostor je potřeba více bitů adresy. Jednoduché číselné typy se ukládají ve dvojkové soustavě. Znaménková celá čísla se běžně ukládají ve dvojkovém doplňku: kladná jako unsigned, záporná jako negace absolutní hodnoty plus jedna, nejvyšší bit určuje znaménko a procesor potřebuje znaménkové porovnání. Floating point má tvar

$$\text{value} = (-1)^{\text{sign}} \cdot \text{significand} \cdot 2^{\text{exponent} - \text{bias}}$$

Endianita určuje pořadí bytů čísla v paměti. Text se ukládá pomocí kódování, základ je ASCII a běžně Unicode.

Složené typy musí respektovat zarovnání. Moderní procesory často vyžadují, aby například 4bajtový `int` ležel na adrese dělitelné čtyřmi. Celá struktura se zarovná podle největšího relevantního typu nebo požadavku CPU, v mezerách vzniká padding a `sizeof` vrací velikost včetně mezer. U tříd a runtime objektů mohou některé jazyky nebo běhová prostředí přehazovat položky, aby mezery omezily; u C/C++ struktur se pořadí položek zachovává. Běžné konstrukce vyšších jazyků se na úrovni instrukcí skládají z čtení a zápisu paměti nebo registrů, aritmetických a logických operací, porovnání, podmíněných a nepodmíněných skoků, práce se zásobníkem a volání a návratu z funkce.

Procesor má uživatelský neprivilegovaný režim pro aplikace a kernel mode, privilegovaný režim pro OS nebo jeho části. Uživatelský režim má omezený přístup ke zdrojům, systémovým registrům a instrukcím, kernel má plný přístup. Přejít do kernelu je možný jen jasně definovanými mechanismy, například systémovým voláním. Jádro OS inicializuje hardware, spravuje

prostředky a spouští a obsluhuje programy. Architektury jádra: monolitické jádro má služby v kernelu a interní volání, typicky Linux; vrstvené jádro má vrstvy s rozhraními a každá používá jen vrstvu pod sebou; mikrokernél přesouvá služby do user space modulů komunikujících s jádrem, což zvyšuje rozšiřitelnost, spolehlivost a bezpečnost, ale může být pomalejší. Poznámky uvádějí Windows jako mikrokernélový směr s hardwarovou abstrakční vrstvou a službami běžícími i v kernel režimu.

Program je pasivní soubor na disku. Loader ho načte do paměti a použije informaci v hlavičce, kde začíná vykonávání. Proces je instance programu vytvořená OS a datová struktura se zdroji, kódem, pamětí a dalšími prostředky. Vlákno je místo uvnitř procesu, kde se vykonávají instrukce; má vlastní program counter, zásobník a kontext procesoru, tedy uložené registry, když neběží. Fiber je lehčí jednotka bez celého kontextu a kooperativním plánováním. Neprivilegované procesy přistupují k prostředkům přes OS a mají virtuální paměť.

Virtuální adresa je adresa, se kterou pracují instrukce procesu. Fyzická adresa je adresa v operační paměti. Překlad dělá MMU transparentně a velmi rychle. Dnešní hlavní mechanismus je stránkování, segmentace se už běžně nepoužívá. Virtuální adresový prostor je rozdělen na stránky, fyzický adresový prostor na stejně velké rámce. Virtuální adresa se dělí na číslo stránky a offset; tabulka stránek mapuje číslo stránky na číslo rámce a offset se přeneseme beze změny. Záznam tabulky obsahuje číslo rámce, atributy a příznak, zda stránka fyzicky existuje; při chybě vzniká page fault. Výhody virtuální paměti jsou větší adresový prostor a hlavně ochrana procesů, protože každý proces má vlastní mapování. Vlákna jednoho procesu adresový prostor sdílejí. Sdílená paměť mezi procesy je možná mapováním různých virtuálních adres na stejný fyzický rámec.

2 Mechanismy, algoritmy a postupy. Řadič zařízení je hardwarová součástka komunikující se zařízením konkrétním protokolem. Ovladač zařízení je softwarová komponenta OS, která komunikuje s řadičem a poskytuje jednotné rozhraní OS a procesům. PIO, programem řízená obsluha, znamená, že CPU přes porty nebo registry řadiče čte stav a zapisuje příkazy a data; pro zadané porty se program skládá z testování stavového registru, čtení nebo zápisu datového registru a opakování podle protokolu zařízení.

Zařízení lze obsluhovat pollingem, přerušením nebo DMA. Polling znamená, že CPU aktivně opakovaně čte stav zařízení, což je neefektivní. Přerušování znamená, že řadič vyšle signál, až je operace hotová; data však může stále kopírovat CPU. DMA přenáší data mezi zařízením a pamětí hardwarově bez průběžné účasti CPU a po dokončení vyvolá přerušování; scatter/gather dovoluje použít nesouvislé části paměti. Typy přerušování: externí hardwarové přes IRQ, které lze maskovat; hardwarová výjimka při problému s instrukcí; softwarové přerušování speciální instrukcí. Trap se hlásí po dokončení instrukce, zatímco fault, například stránkovací chyba nebo dělení nulou, vrací procesor do stavu před instrukcí, OS může problém opravit a instrukce se pustí znovu.

Průběh přerušování: CPU zjistí zdroj přerušování a adresu handleru buď pevně, nebo přes tabulku, přeruší proud instrukcí, obvykle se přepne do privilegovaného režimu a vykonává handler v kernelu. Handler uloží stav CPU, provede obsluhu, obnoví stav a CPU pokračuje v původním proudu instrukcí. Systémové volání je řízený přechod z uživatelského kódu do jádra přes tenkou vrstvu nad OS; například výpis do konzole vyžaduje přepnutí do kernelu a zpět, proto se používá buffering.

Překlad jednoúrovňovou stránkovací tabulkou: u 32bitové virtuální adresy s 12bitovým offsetem tvoří horních 20 bitů číslo stránky. Tím se indexuje stránkovací tabulka, záznam dá číslo rámce a atributy, a fyzická adresa vznikne připojením stejného offsetu k číslu rámce. Pokud záznam není platný, nastane page fault. Když dojde fyzická paměť, některá stránka se může odložit na disk; stránky mají stejnou velikost, což je praktičtější než staré segmenty různých velikostí.

Sdílení procesoru řídí scheduler. Při přerušování běhu vlákna se provede context switch, uloží se registry včetně PC a obnoví se kontext jiného vlákna. Kooperativní multitasking vyžaduje

dobrovolné předání řízení, často `yield`. Preemptivní multitasking dá vláknům `time slice` a po vypršení ho plánovač přeruší. Stavby vláken: `created`, `ready`, `running`, `blocked`, `terminated`, také `zombie`. Real-time scheduling pracuje s časem spuštění a `deadline`; `hard deadline` znamená, že po termínu nemá smysl pokračovat, `soft deadline` že pokračování ještě může mít hodnotu. Priorita může mít statickou složku při spuštění a dynamickou složku, která se `ready` vláknům časem zvyšuje a po spuštění se sníží na nulu.

Plánovací algoritmy: FCFS je kooperativní FIFO fronta v pořadí příchodu. Shortest job first vyžaduje známý očekávaný čas běhu, existuje i longest job first. Round robin je preemptivní varianta podobná FCFS, ale po vyčerpání `time slice` se vlákno zařadí na konec fronty. Multilevel feedback queue má několik front s různou délkou `time slice`; horní fronty mají kratší kvantum, dlouho běžící vlákno klesá níže a vlákno, které se brzy zablokuje, může stoupat výše. Completely fair scheduler používá červeno-černý strom.

Soubor je jednotka organizace dat a kolekce souvisejících informací na perzistentním úložišti; jádro OS nerozumí obecným formátům souborů. Interně je soubor unikátní číslo, pro lidi má jméno a cestu. Cesta je analogická virtuální adrese: musí se přeložit na konkrétní objekt souborového systému. Přípona nebo počáteční tečka mohou mít konvenční význam, ale nemusí odpovídat reálnému formátu. File handle je číslo specifické pro proces odkazující na soubor; v POSIXu mají `stdin`, `stdout`, `stderr` tradičně deskriptory 0, 1, 2. Operace zahrnují vytvoření, smazání, změnu atributů, vyčištění cache, čtení a zápis.

Buffering cachuje sektory disku a existuje ve více úrovních, například systémové a runtime cache. Sekvenční přístup se liší od náhodného. Alternativy jsou memory mapping a asynchronní přístup k souborům. Adresář je kolekce souborů, obvykle reprezentovaná také jako soubor; zrychluje hledání, navigaci a logické seskupení, může držet atributy souborů a má operace vytvořit, smazat, přejmenovat, najít název a vypsát obsah. Obvykle existuje kořenový adresář. Úložiště může být tradiční sekundární nebo externí, file system v RAM pro dočasné soubory, síťové úložiště nebo virtuální soubory jako `/dev/null`. Hard link znamená více adresářových záznamů na stejný fyzický soubor a OS musí počítat odkazy, aby smazání jednoho linku nesmazalo data předčasně. Symlink je speciální soubor obsahující cestu k jinému souboru. File system řeší překlad jmen, správu datových bloků, metadata souborů a adresáře; blok je několik sektorů. Může být lokální, například FAT nebo NTFS, nebo síťový, například NFS nebo CIFS/SMB.

Race condition vzniká, když více vláken přistupuje ke sdílenému prostředku a výsledek závisí na plánování OS nebo procesoru. Řešení je atomizovat read-modify-write operaci nebo vymezit kritickou sekci a použít synchronizační primitivum. Kritická sekce je nejmenší kus kódu přistupující ke sdílenému prostředku, který nesmí používat více procesů nebo vláken současně. Vzájemné vyloučení, mutex, zajišťuje, že do kritické sekce nevstoupí více jednotek najednou; zámek se obvykle chápe pro vlákna, mutex pro procesy. Některé mutexy dovolují rekurzivní zamčení týmž procesem a vyžadují stejný počet odemčení.

Synchronizační primitiva mohou čekat aktivně nebo pasivně. Aktivní čekání stále vykonává instrukce a testuje, zda lze vstoupit, pasivní čekání vlákno zablokuje. Hardwarová podpora zahrnuje atomické instrukce `test-and-set` a `compare-and-swap`. Spin-lock je aktivní čekání pomocí TSL nebo CAS a hodí se pro velmi krátké čekání. Semafor má čítač a frontu čekajících vláken a atomické operace UP a DOWN. Mutex je semafor s počítadlem jedna nebo příbuzné primitivum pro jediného držitele s operacemi LOCK a UNLOCK. Barrier zastaví vlákna, dokud se jich u stejné bariéry nesejde požadovaný počet.

3 Detaily a zkouškové příklady. Při převodu instrukcí na konstrukty vyššího jazyka hleděj vzory: přiřazení je výpočet hodnoty a uložení do adresy, podmínka je porovnání a podmíněný skok přes větev, cyklus je test se skokem zpět, volání funkce uloží návratovou adresu a argumenty podle konvence a návrat obnoví řízení. Opačný směr je stejný: z konstrukce vyššího jazyka popiš registry, adresy, skoky a zásobník.

Virtuální adresové prostory chrání procesy navzájem, ale nevlákna jednoho procesu mezi sebou,

protože vlákna sdílejí stejný adresový prostor. To je důvod, proč sdílená data mezi vlákny vyžadují synchronizaci. Sdílení mezi procesy je záměrná výjimka nastavená OS mapováním stránek.

U PIO a přerušení je rozdíl v tom, kdo čeká. Při čistém pollingu čeká CPU opakovaným testem portu. Při přerušení může CPU dělat jinou práci a zařízení si vyžádá obsluhu. Při DMA CPU nastaví přenos, data přesouvá řadič a CPU řeší až dokončení nebo chybu.

U souborů se nesmí směřovat jméno a identita. Jméno je cesta v adresářích, handle je procesový odkaz na otevřený soubor a hard link znamená, že více jmen může ukazovat na stejná data. Souborový systém poskytuje abstrakci bloků, jmen, adresářů, metadat a rozhraní pro aplikace.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak instrukcemi vyjádříš podmínku, cyklus a volání funkce?
- Co se stane při přerušení z pohledu procesoru a OS?
- Jak přeložíš virtuální adresu přes jednoúrovňovou stránkovací tabulku?
- Jaký je rozdíl mezi procesem, vláknem a jejich kontextem?
- Čím se liší aktivní a pasivní čekání?

Pozor (častá past): Virtuální adresa není fyzické místo v RAM. Je to adresa v prostoru procesu, kterou MMU a tabulka stránek překládají na fyzický rámec.

6 Část III: Specializace - Základy umělé inteligence

6.1 Řešení úloh prohledáváním

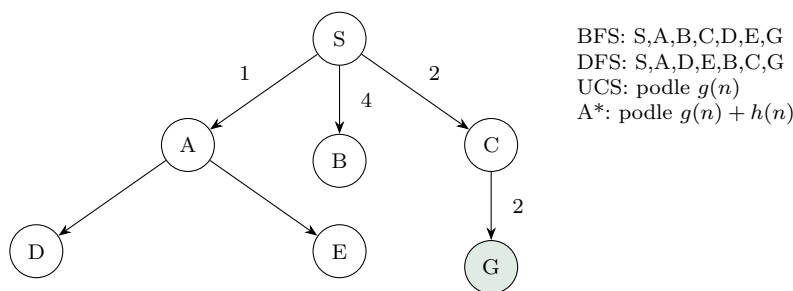
priorita: vysoká cíl: zelená (2 body)

Řešení úloh prohledáváním

- formulace úlohy
- stromové vs. grafové prohledávání
- neinformované prohledávání (DFS, BFS, uniform-cost search)
- informované prohledávání (algoritmus A*, přípustné a konzistentní heuristiky)

Učivo (jak na to):

Prohledávání je vhodné, když stav umíme brát jako uzel grafu a akci jako hranu. Odpověď u zkoušky má vždy říct: jak problém formuluji, jak držím frontier, podle čeho vybírám další uzel a kdy je řešení optimální.



Úloha je dána počátečním stavem, množinou akcí nebo přechodovým modelem, testem cíle a cenou cesty. Stromové prohledávání si pamatuje cesty ve stromu možností, grafové prohledávání navíc uzavírá již navštívené stavy, aby se netočilo v cyklech.

- **BFS:** fronta, bere nejmenší uzel. Je úplné pro konečné větvení a optimální při stejných cenách kroků.
- **DFS:** zásobník nebo rekurze, jde do hloubky. Má malou paměť, ale není obecně optimální a ve stromové verzi se může zacyklit.
- **Uniform-cost search:** prioritní fronta podle dosavadní ceny $g(n)$. Je Dijkstra nad stavovým prostorem, optimální pro nezáporné ceny (v nekonečném prostoru typicky pro kladnou dolní mez ceny kroku).
- **A*:** vybírá minimum $f(n) = g(n) + h(n)$. Přípustná heuristika nikdy nepřeceňuje cenu do cíle. Konzistentní heuristika splňuje $h(n) \leq c(n, a, n') + h(n')$ a dává optimalitu i v grafovém prohledávání bez znovuotevírání uzlů.

► **Klíč:** BFS = hloubka, DFS = paměť, UCS = skutečná cena, A* = skutečná cena plus odhad.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Agent vnímá prostředí senzory a jedná aktuátory. Racionální agent volí akci maximalizující výkonovou míru. Simple reflex agent mapuje pozorování přímo na akci; model-

based reflex agent z minulého stavu, minulé akce a nového pozorování udržuje stav a z něj volí akci; goal-based agent vybírá akci podle stavu a cíle. Reprezentace stavu může být atomická, kde je stav černá skříňka a hodí se pro prohledávání, faktorizovaná, kde je stav vektor hodnot a hodí se pro CSP, výrokovou logiku a plánování, nebo strukturovaná, kde svět tvoří objekty a vztahy a hodí se pro logiku prvního řádu. Problem-solving agent je goal-based agent s atomickými stavy: cíl je množina cílových stavů, akce jsou přechody a řešením je posloupnost akcí z počátečního stavu do cíle. Předpokládá se plně pozorovatelné, diskrétní, statické a deterministické prostředí. Dobře definovaný problém obsahuje počáteční stav, přechodový model $(state, action) \mapsto state$, test cíle a cenu cesty nebo kroků. Abstrakce prostředí je validní, pokud abstraktní řešení lze expandovat na reálné řešení, a užitečná, pokud je vykonání abstraktních akcí jednodušší než řešení původního problému.

2 Algoritmy a tvrzení: Tree search drží frontier cest ve stromu možností. Graph search navíc ukládá uzavřené nebo už viděné stavy, obvykle v hash tabulce, protože stavový prostor může být nekonečný; tím předchází cyklům. Tree search lze spustit i na grafu s cykly, ale může se chovat patologicky; i na DAGu může opakovat stav, pokud do něj vedou dvě různé orientované cesty. BFS používá frontu a expanduje nejmenší neexpandovaný uzel; je úplné pro konečně větvičí grafy a optimální, pokud cena cesty je neklesající funkcí hloubky, typicky pro jednotkové ceny. DFS používá zásobník nebo rekurzi a expanduje nejhlubší neexpandovaný uzel; graph-search verze je úplná na konečném grafu, tree-search verze na grafu s cykly úplná není, obecně není optimální. Uniform-cost search je Dijkstrův algoritmus nad stavovým prostorem: používá prioritní frontu podle $g(n)$, ceny nejlevnější známé cesty ze startu do n , a je optimální pro nezáporné ceny s vhodnou dolní mezí kladného kroku. Best-first search vybírá uzel podle ohodnocení $f(n)$. Greedy best-first má $f(n) = h(n)$ a není obecně optimální ani úplný. A^* má $f(n) = g(n) + h(n)$; při přípustné heuristice je optimální ve stromovém prohledávání, při konzistentní heuristice je optimální v grafovém prohledávání. Přípustnost znamená $0 \leq h(n) \leq h^*(n)$, tedy heuristika nepřeceňuje optimální cenu do cíle. Konzistence znamená $h(n) \leq c(n, a, n') + h(n')$ pro každý přechod a typicky $h(G) = 0$ pro cíl. Konzistence implikuje přípustnost, opačně to neplatí. Pro konzistentní heuristiku hodnoty $f(n)$ neklesají podél libovolné cesty.

3 Detaily a příklady: U Lloydovy patnáctky nejsou všechny permutace dosažitelné, protože se zachovává invariant složený z parity permutace kamenů a polohy prázdného pole. U BFS je časová i prostorová složitost $O(b^{d+1})$, kde b je branching factor a d hloubka nejbližšího cíle; hlavní potíž je paměť. U DFS je čas $O(b^m)$ a paměť $O(bm)$, kde m je maximální hloubka; při backtrackingu, kdy se generuje vždy jen jeden následník místo všech najednou, stačí $O(m)$ paměti, ale ne každý stav umí generovat následníky po jednom. A^* obvykle narazí na paměť dříve než na čas. Pokud heuristika h_2 dává pro všechny uzly aspoň tak velké hodnoty jako h_1 a je přípustná, říkáme, že h_2 dominuje h_1 ; pokud není výrazně dražší na výpočet, je obvykle lepší, protože vede k menšímu počtu expanzí. Důkaz optimality A^* ve stromu s přípustnou heuristikou stojí na tom, že cílový uzel s neoptimální cestou nemůže být vybrán dříve než nějaký uzel na optimální cestě. V grafu je potřeba konzistence, jinak by se mohl uzavřít stav s později zlepšitelnou cenou.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jakých pět částí má formulace problému prohledáváním?
- Kdy je BFS optimální a kdy nestačí?
- Jaký je rozdíl mezi přípustnou a konzistentní heuristikou?
- Co se stane, když DFS pustíš jako tree search na graf s cyklem?

Pozor (častá past): Neříkej, že A^* je vždy optimální. Potřebuje správnou heuristiku a u grafového prohledávání se typicky požaduje konzistence.

6.2 Splňování omezujících podmínek (CSP)

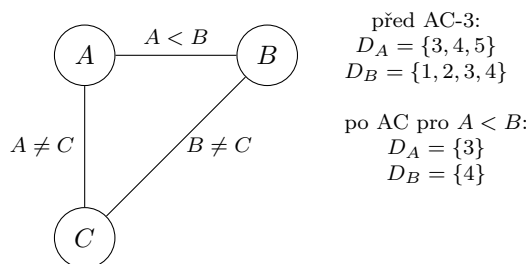
priorita: vysoká cíl: zelená (2 body)

Splňování omezujících podmínek

- problém splňování podmínek
- hranová konzistence (algoritmus AC-3)
- algoritmus hledání řešení (MAC)

Učivo (jak na to):

CSP neřeší cestu akcí, ale tabulku hodnot: každé proměnné chceme dát hodnotu tak, aby všechny podmínky seděly. Velký zisk je škrtnat nemožné hodnoty dřív, než rozvětvíme backtracking.



CSP má proměnné X_i , domény D_i a podmínky jako relace na proměnných. Řešení je úplné přiřazení hodnot, které splní všechny podmínky. Hrana (X_i, X_j) je hranově konzistentní, když pro každou hodnotu $x \in D_i$ existuje hodnota $y \in D_j$, která splní binární podmínku mezi X_i a X_j .

AC-3: vlož všechny orientované hrany do fronty. Vyber (X_i, X_j) a smaž z D_i hodnoty bez podpory v D_j . Pokud se D_i změnila, vrať do fronty všechny hrany (X_k, X_i) kromě právě opačné. Prázdná doména znamená neúspěch. Jinak dostaneš lokálně vyčištěný CSP, ne nutně hotové řešení.

MAC: backtracking nad proměnnými, po každém přiřazení se znovu udržuje hranová konzistence. Tím se často odhalí spor dřív než hluboko ve stromu.

Odpověď podle bodů (cíľ pro průchod: zelená = 2 body)

1 Definice: Problém splňování omezujících podmínek, CSP, tvoří konečná množina proměnných, pro každou proměnnou konečná doména možných hodnot a konečná množina podmínek. Podmínka je relace na podmnožině proměnných, tedy podmnožina kartézského součinu jejich domén. Aritá podmínky je počet proměnných, které omezuje. Přípustné řešení je úplné konzistentní přiřazení hodnot proměnným; úplné znamená, že každá proměnná má hodnotu, konzistentní znamená, že všechny podmínky jsou splněny. CSP lze řešit stromovým backtrackingem, kde se proměnné přiřazují v určitém pořadí; efektivitu zvyšuje vhodná volba pořadí a předběžné čištění hodnot, které už nemohou být součástí žádného řešení.

2 AC-3 a MAC: Hranová konzistence se definuje pro binární podmínky; obecnou n -ární podmínku lze převést na soubor binárních podmínek, případně s pomocnými proměnnými. V síti podmínek jsou vrcholy proměnné a binární podmínka mezi V_i, V_j odpovídá dvěma orientovaným hranám; více podmínek mezi stejnou dvojicí lze sloučit do jedné. Orientovaná hrana (V_i, V_j) je hranově konzistentní právě tehdy, když pro každé $x \in D_i$ existuje $y \in D_j$ takové, že přiřazení $V_i = x, V_j = y$ splní všechny binární podmínky mezi V_i, V_j . Může platit konzistence (V_i, V_j) , ale ne (V_j, V_i) . CSP je hranově konzistentní, když je hranově konzistentní každá orientovaná hrana.

AC-3 začne frontou všech orientovaných hran, opakovaně vyjme hranu (V_i, V_j) a odstraní z D_i hodnoty bez podpory v D_j . Pokud se D_i změnila, vrátí do fronty všechny hrany směřující do V_i kromě opačné hrany k právě zpracované. Prázdná doména znamená spor. MAC, maintaining arc consistency, kombinuje backtracking s AC: nejprve problém zkonzistentní, po každém přiřazení obnoví hranovou konzistenci. Této technice se říká také look ahead nebo constraint propagation.

3 Detaily a příklady: Příklad čištění: při $a \in \{3, 4, 5, 6, 7\}$, $b \in \{1, 2, 3, 4, 5\}$ a podmínce $a < b$ nemají hodnoty $a = 5, 6, 7$ podporu a hodnoty $b = 1, 2, 3$ také ne, takže zůstanou $a \in \{3, 4\}$ a $b \in \{4, 5\}$. AC-3 má složitost $O(cd^3)$, kde c je počet podmínek a d velikost domény: kontrola jedné hrany stojí $O(d^2)$ a každá podmínka se může ve frontě objevit nejvýše d -krát, protože z domény lze odebrat nejvýše d hodnot. Optimální algoritmy dosahují $O(cd^2)$. Hranová konzistence je lokální konzistence a negarantuje globální konzistenci ani existenci řešení. Forward checking je jednodušší a rychlejší než AC: po aktuálním přiřazení čistí domény dosud nepřirazených proměnných. AC navíc udržuje vzájemnou kompatibilitu domén nepřirazených proměnných. Kontroly FC a AC jsou polynomiální, zatímco backtrackingový strom se větví exponenciálně.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co přesně znamená, že hodnota má podporu v sousední doméně?
- Proč je hrana v CSP orientovaná, i když podmínka mezi proměnnými je binární?
- Jak se liší forward checking, AC-3 a MAC?
- Proč může být CSP hranově konzistentní a přesto nemít řešení?

Pozor (častá past): Nezaměň hranovou konzistenci s globální konzistencí. AC-3 škrte zjevně nemožné hodnoty, ale nemusí najít celé přiřazení.

6.3 Logické uvažování

priorita: vysoká cíl: zelená (2 body)

Logické uvažování

- základy výrokové logiky (konjunktivní a disjunktivní normální forma)
- algoritmus DPLL
- dopředné a zpětné řetězení (Hornovské klauzule)
- rezoluce

Učivo (jak na to):

Logické uvažování převádí znalosti na formule a otázku na test důsledku. Prakticky chceš umět dvě cesty: SAT přes DPLL a odvozování přes Hornovy klauzule nebo rezoluci.

CNF je konjunkce klauzulí, kde klauzule je disjunkce literálů. DNF je disjunkce konjunkcí literálů. Hornova klauzule má nejvýše jeden pozitivní literál, například $\neg p \vee \neg q \vee r$, což odpovídá pravidlu $p \wedge q \Rightarrow r$.

DPLL rozhoduje splnitelnost CNF. Opakuj jednotkovou propagaci, nastav čisté literály tak, aby splnily své klauzule, zkontroluj konec: žádné klauzule znamenají SAT, prázdná klauzule znamená UNSAT. Když není rozhodnuto, vyber proměnnou a větví na pravda/nepravda.

Řetězení: dopředné řetězení jde od faktů k novým faktům, je data-driven. Zpětné řetězení jde od cíle k podcílům, je goal-driven. Obojí je levné a přirozené pro Hornovy klauzule.

Rezoluce: pro dotaz α testuj nesplnitelnost $KB \wedge \neg\alpha$ v CNF. Z klauzulí $(A \vee p)$ a $(B \vee \neg p)$ odvod $(A \vee B)$. Když vznikne prázdná klauzule, $KB \models \alpha$.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Výroková logika se používá k odvozování znalostí ze znalostní báze KB . Literál je proměnná nebo její negace. Klauzule je disjunkce literálů. CNF je konjunkce klauzulí, DNF je disjunkce konjunkcí literálů. Dotaz α se testuje jako sémantický důsledek $KB \models \alpha$; ekvivalentně je $KB \wedge \neg\alpha$ nesplnitelná. Hornova klauzule je disjunkce literálů, z nichž nejvýše jeden je pozitivní. Klauzule $\neg p \vee \neg q \vee \neg r \vee s$ odpovídá implikaci $p \wedge q \wedge r \Rightarrow s$. Hornovy báze umožňují efektivní dopředné a zpětné řetězení.

2 Algoritmy: Literál ℓ má čistý výskyt ve formuli φ , pokud se ℓ ve φ vyskytuje a opačný literál $\bar{\ell}$ se nevyskytuje. V DPLL se pro CNF opakuje jednotková propagace: pokud existuje jednotková klauzule ℓ , nastaví se $\ell = 1$, odstraní se všechny klauzule obsahující ℓ a z ostatních klauzulí se odstraní $\bar{\ell}$. Potom se čisté literály nastaví na 1 a odstraní se klauzule, které obsahují daný čistý literál. Pokud nezůstane žádná klauzule, formule je splnitelná; pokud vznikne prázdná klauzule, formule je nesplnitelná. Jinak se vybere dosud neohodnocená proměnná p a rekurzivně se zkouší větve $\varphi \wedge p$ a $\varphi \wedge \neg p$. Dopředné řetězení je data-driven: z faktů postupně spouští pravidla, například z p zjednoduší $p \wedge q \wedge r \Rightarrow s$ na $q \wedge r \Rightarrow s$, a jakmile pravidlu nezbývá žádný předpoklad, přidá závěr s . Lze ho implementovat počítadly nesplněných předpokladů. Zpětné řetězení je goal-driven: pro cíl hledá pravidla, kde je cíl na pravé straně, a jejich předpoklady se stanou podcíli; končí na známých faktech. Používá se v Prologu.

3 Detaily a rezoluce: Čistý literál neovlivní splnitelnost, ale může zmenšit množinu modelů, které algoritmus najde; v praxi se pravidlo čistých literálů nepoužívá tak často jako jednotková propagace. Jednotková propagace je hledání klauzulí o jedné proměnné a je analogická hranové konzistenci; větvení hodnot proměnných je tree search. DPLL má v nejhorším exponenciální čas. Forward chaining je speciální použití rezoluce, kdy rezolvujeme známé fakty s libovolnou klauzulí; může procházet mnoho klauzulí nesouvisejících s dotazem. Backward chaining rezolvuje to, co chceme zjistit, s klauzulemi a může být výrazně levnější, pokud dotaz zasahuje malou část báze. Obecná rezoluce pro důkaz dotazu převede $KB \wedge \neg\alpha$ do CNF na množinu klauzulí S . Z klauzulí $\varphi \vee p$ a $\psi \vee \neg p$ odvodí rezolventu $\varphi \vee \psi$. Zkouší se rezolvovat dvojice klauzulí; pokud vznikne prázdná klauzule, existuje rezoluční zamítnutí a platí $KB \models \alpha$. Pokud už nelze přidat žádnou novou rezolventu a prázdná klauzule nevznikla, $KB \models \alpha$ neplatí.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co udělá DPLL s jednotkovou klauzulí?
- Jak přepíšeš $\neg p \vee \neg q \vee r$ jako pravidlo?
- Proč se při důkazu dotazu přidává $\neg\alpha$?
- Kdy je výhodnější zpětné řetězení než dopředné?

Pozor (častá past): Nepleť DPLL a rezoluci: DPLL hledá ohodnocení CNF, rezoluce odvozuje nové klauzule a hledá prázdnou klauzuli.

6.4 Pravděpodobnostní uvažování

priorita: vysoká cíl: zelená (2 body)

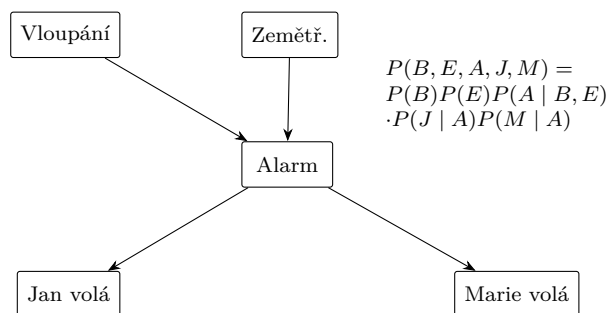
Pravděpodobnostní uvažování

- základy teorie pravděpodobnosti (úplná sdružená distribuce, nezávislost, Bayesovo pravidlo)
- pravděpodobnostní uvažování (vysčítání, normalizace)
- Bayesovské sítě

- konstrukce a vztah k úplné sdružené distribuci
- exaktní odvozování (enumerace, eliminace proměnných)
- aproximační odvozování (Monte Carlo, zamítání, vážení věrohodností)

Učivo (jak na to):

Pravděpodobnost je způsob, jak uvažovat při nejistotě. Úplná sdružená distribuce umí odpovědět na všechno, ale je obrovská. Bayesovská síť ji komprimuje pomocí podmíněných nezávislostí.



Bayesovská síť je orientovaný acyklický graf náhodných proměnných a tabulky $P(X_i | Parents(X_i))$. Společná distribuce se faktorizuje jako

$$P(x_1, \dots, x_n) = \prod_i P(x_i | pa_i),$$

kde pa_i jsou hodnoty rodičů proměnné X_i .

Základní počty: z úplné sdružené distribuce získáš marginál vysčítáním skrytých proměnných, například $P(X) = \sum_y P(X, y)$. Podmíněnou pravděpodobnost získáš normalizací: $P(X | e) = \alpha P(X, e)$. Bayesovo pravidlo je $P(H | e) = \alpha P(e | H)P(H)$.

Inference v síti: enumerace přímo sčítá přes skryté proměnné. Eliminace proměnných průběžně tvoří faktory a sčítá jen jednou, takže šetří opakovanou práci. Aproximace generuje vzorky: Monte Carlo obecně, zamítání zahodí vzorky neslučitelné s evidencí, vážení věrohodností vzorky nezahazuje, ale váží je pravděpodobnostmi evidence.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Pravděpodobnostní uvažování umožňuje agentovi zacházet s nejistotou. Zdrojem nejistoty je částečná pozorovatelnost, kdy nelze snadno zjistit aktuální stav, a nedeterminismus, kdy není jistý výsledek akce. Agent si může udržovat belief state, tedy pravděpodobnostní distribuci přes možné stavy světa. Úplná sdružená distribuce obsahuje pravděpodobnosti všech kombinací hodnot proměnných a lze z ní získat libovolný dotaz výčtem. Marginál se získá vysčítáním skrytých hodnot, například $P(Y) = \sum_z P(Y | z)P(z)$ nebo obecně $P(X) = \sum_y P(X, y)$. Podmíněná pravděpodobnost se získá normalizací $P(Y | e) = P(Y \wedge e) / P(e) = \alpha P(Y \wedge e)$, kde α je normalizační konstanta. Bayesovo pravidlo je

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)} = \alpha P(X | Y)P(Y).$$

Umožňuje přejít z příčinného směru $P(\text{effect} | \text{cause})$, který bývá známý, na diagnostický směr $P(\text{cause} | \text{effect})$, který často chceme zjistit. Chain rule říká $P(x_1, \dots, x_n) = \prod_i P(x_i | x_{i-1}, \dots, x_1)$ a plyne z product rule. Naivní bayesovský model předpokládá podmíněnou

nezávislost důsledků při známé příčině:

$$P(C, E_1, \dots, E_n) = P(C) \prod_i P(E_i | C).$$

2 Bayesovské sítě a exaktní inference: Bayesovská síť je orientovaný acyklický graf, jehož vrcholy jsou náhodné proměnné, hrany vyjadřují přímé závislosti a u každého vrcholu je CPD tabulka $P(X_i | \text{Parents}(X_i))$. Pro každou kombinaci hodnot rodičů dává tabulka pravděpodobnosti hodnot dané proměnné; u binární proměnné stačí jeden sloupec, druhý se dopočte doplnkem do jedné. Síť reprezentuje úplnou sdruženou distribuci faktorizací

$$P(x_1, \dots, x_n) = \prod_i P(x_i | pa_i).$$

Konstrukce sítě začíná pořadím proměnných, potom se postupně přidávají hrany tak, aby rodiče nesli všechny nutné závislosti. Prakticky je lepší řadit proměnné od příčin k důsledkům, protože síť i CPD tabulky bývají menší a vyplňují se snáze. Při špatném pořadí, například MaryCalls, JohnCalls, Alarm, Burglary, Earthquake, vznikají hrany mezi důsledky a pozdějšími příčinami: MaryCalls ovlivní JohnCalls, obě volání vedou do Alarmu, Alarm do Burglary a Earthquake a Burglary do Earthquake, protože při známém alarmu a neexistenci vloupání roste pravděpodobnost zemětřesení. Exaktní inference počítá dotazy typu $P(X | e) = \alpha P(X, e) = \alpha \sum_y P(X, e, y)$, kde e je evidence a y skryté proměnné. Enumerace přímo počítá přes skryté proměnné a hodnoty $P(X, e, y)$ určuje z CPD tabulek. Eliminace proměnných používá faktory, tedy tabulky odpovídající CPD nebo mezivýpočtům; faktory se násobí a proměnná se eliminuje vysčítáním přes její hodnoty. Například faktor $P(A, B, C)$ lze rozdělit na část pro A a pro $\neg A$ a odpovídající řádky sečíst, čímž vznikne faktor nad B, C .

3 Aproximace a detaily: Úplná sdružená distribuce je často příliš velká, ale podmíněné nezávislosti dovolují kompaktní reprezentaci. Při inferenci se některé větve výpočtu opakují, proto eliminace proměnných využívá dynamické programování. Aproximační inference se používá, když je síť velká nebo hustě propojená a přesný výpočet je drahý. Monte Carlo metody generují mnoho vzorků a odhadují pravděpodobnosti statisticky. Vzorky v Bayesovské síti se generují topologicky podle CPD tabulek. Rejection sampling pro dotaz $P(X | e)$ zahodí vzorky, v nichž evidence e neplatí, a odhaduje

$$P(X | e) \approx \frac{N(X, e)}{N(e)},$$

kde $N(X, e)$ je počet vzorků splňujících X i e a $N(e)$ počet vzorků splňujících evidenci. Riziko je, že při málo pravděpodobné evidenci zahodíme téměř vše. Likelihood weighting evidenci zafixuje, generuje jen zbylé proměnné a vzorkům přidělí váhu podle pravděpodobnosti evidence, takže je nezahazuje; problém nastává, když jsou váhy velmi malé.

Vyzkoušej se (zakryj text a odpovez nahlas)

- Jak dostaneš $P(X | e)$ z $P(X, e)$?
- Co přesně faktorizuje Bayesovská síť?
- Proč je eliminace proměnných úspornější než čistá enumerace?
- Jaký je rozdíl mezi zamítáním a vážením věrohodností?

Pozor (častá past): Neříkej, že chybějící hrana znamená obyčejnou nezávislost. V Bayesovské síti jde hlavně o podmíněné nezávislosti vzhledem k rodičům a evidenci.

6.5 Reprezentace znalostí

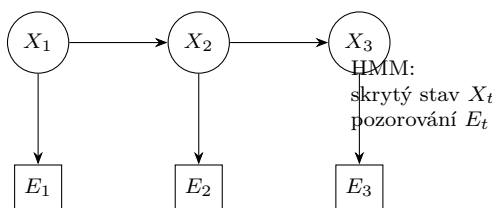
priorita: vysoká cíl: zelená (2 body)

Reprezentace znalostí

- situační kalkulus, problém rámce
- Markovské modely
- filtrace, predikce, vyhlazování, nejpravděpodobnější průchod
- skryté Markovské modely (HMM) vs. dynamické Bayesovské sítě

Učivo (jak na to):

Reprezentace znalostí řeší, jak zapsat svět tak, aby nad ním šlo odvozovat. Situační kalkulus je logický zápis změn po akcích, Markovské modely jsou pravděpodobnostní zápis vývoje stavu v čase.



Situační kalkulus popisuje situace vzniklé posloupností akcí, predikáty o situacích a akci $Result(s, a)$. Problém rámce je otázka, jak stručně zapsat, co se akcí nemění. HMM má skrytý stav X_t , pozorování E_t , přechod $P(X_t | X_{t-1})$ a senzorový model $P(E_t | X_t)$.

Markovské úlohy: filtrace počítá $P(X_t | e_{1:t})$, tedy aktuální stav po dosavadních pozorováních. Predikce počítá budoucí stav $P(X_{t+k} | e_{1:t})$. Vyhlazování zpřesňuje minulý stav $P(X_k | e_{1:t})$ pro $k < t$. Nejpravděpodobnější průchod hledá nejpravděpodobnější posloupnost skrytých stavů, typicky Viterbiho algoritmem.

HMM vs. dynamická Bayesovská síť: HMM má jednu skrytou stavovou proměnnou na čas. Dynamická Bayesovská síť rozkládá stav na více proměnných a dovolí bohatší závislosti mezi nimi.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Problém rámce vzniká při popisu měnícího se světa logikou: kdybychom pro každý časový krok používali časově anotované výrokové proměnné, museli bychom přidat mnoho axiomů říkajících, že se většina hodnot mezi kroky nemění sama od sebe. Situační kalkulus používá logiku prvního řádu. Svět se vyvíjí přes situace, počáteční situace je s_0 a situace po aplikaci akce a v situaci s je $Results(s, a)$. Stavby jsou popsány relacemi mezi objekty; relace měnící se v čase dostávají argument situace, například $at(robot, location, s)$, zatímco stálé relace, například $connected(l, l')$, argument situace nepotřebují. Předpoklady akcí popisují possibility axiomy tvaru $\varphi(s) \Rightarrow Poss(a, s)$, například $at(a, l, s) \wedge connected(l, l') \Rightarrow Poss(go(a, l, l'), s)$. Vlastnosti dalšího stavu popisují successor-state axiomy, které říkají, kdy fluent platí po akci: buď jej akce způsobila, nebo platil dříve a akce jej nezrušila. Plánování v situačním kalkulu se ptá, zda existuje situace, kde platí cílová podmínka.

2 Markovské modely a inference: V časovém pravděpodobnostním modelu je stav světa popsán náhodnými proměnnými. Skryté proměnné X_t popisují reálný stav, pozorovatelné proměnné e_t popisují měření a čas t je diskretní. Zápis $X_{a:b}$ označuje posloupnost proměnných od času a do b . Skrytý Markovův model je dvojice procesů (X_n, e_n) , kde X_n je nepozorovatelný markovský proces. Přechodový model obecně popisuje $P(X_t | X_{0:t-1})$, ale Markovův předpoklad ho zjednoduší na závislost jen na X_{t-1} ; stacionarita říká, že tabulky $P(X_t | X_{t-1})$ jsou stejné pro všechna t . Senzorový model obecně popisuje $P(e_t | X_{0:t}, e_{1:t-1})$, ale sensor Markov assumption říká, že pozorování závisí jen na aktuálním stavu, tedy $P(e_t | X_t)$. Filtrace počítá aktuální stav

z dosavadních pozorování:

$$P(X_{t+1} | e_{1:t+1}) = \alpha P(e_{t+1} | X_{t+1}) \sum_{x_t} P(X_{t+1} | x_t) P(x_t | e_{1:t}).$$

Je rekurzivní a používá forward algoritmus. Predikce počítá budoucí stav bez dalších měření:

$$P(X_{t+k+1} | e_{1:t}) = \sum_{x_{t+k}} P(X_{t+k+1} | x_{t+k}) P(x_{t+k} | e_{1:t}).$$

Po mixing time může distribuce predikcí konvergovat ke stacionárnímu rozdělení procesu. Vyhlazování počítá minulý stav s využitím pozdější evidence:

$$P(X_k | e_{1:t}) = \alpha P(X_k | e_{1:k}) P(e_{k+1:t} | X_k),$$

kde první člen je dopředná filtrace a druhý zpětný vliv budoucích pozorování; používá se forward-backward algoritmus. Nejpravděpodobnější vysvětlení hledá posloupnost $X_{1:t}$ maximalizující pravděpodobnost při evidenci, rekurzivně si pamatuje nejlepší cesty do stavů a používá Viterbiho algoritmus.

3 HMM a DBN: Pokud je stav procesu popsán jedinou diskrétní proměnnou X_t a pozorování jedinou proměnnou E_t , lze základní algoritmy pro HMM implementovat maticově. Dynamická Bayesovská síť reprezentuje časový pravděpodobnostní model s více stavovými proměnnými; zachycuje vztahy mezi minulou a současnou vrstvou a každá stavová proměnná má rodiče ve stejné nebo předchozí vrstvě podle Markovova předpokladu. HMM je speciální případ DBN. Naopak DBN lze zakódovat jako HMM tak, že jedna skrytá proměnná HMM nese celou n-tici hodnot stavových proměnných DBN. V HMM je tedy celý stav světa komprimován do jedné proměnné, zatímco DBN jej faktorizuje. Vztah DBN k HMM je podobný jako vztah běžné Bayesovské sítě k úplné sdružené distribuci: DBN je úspornější reprezentace.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je problém rámce v jedné větě?
- Jaký dotaz odpovídá filtraci a jaký vyhlazování?
- Co hledá Viterbiho algoritmus?
- Proč je DBN obecnější než HMM?

Pozor (častá past): Nezaměň filtraci s vyhlazováním: filtrace odhaduje současnost z minulých pozorování, vyhlazování zpětně zpřesňuje minulost i pomocí pozdější evidence.

6.6 Automatické plánování

priorita: vysoká

cíl: zelená (2 body)

Automatické plánování

- formulace plánovacího problému (definice operátoru)
- dopředné a zpětné plánování

Učivo (jak na to):

Plánování je prohledávání nad faktorizovaným stavem. Místo černé skříňky používáme fakta, předpoklady akcí a efekty akcí, takže lze rozumněji vybírat relevantní kroky.

Klasický plánovací problém má počáteční stav jako množinu faktů, cíl jako podmínku na fakta a operátory. Operátor má preconditions, tedy kdy ho lze použít, a effects, tedy co přidá nebo smaže ve stavu. Plán je posloupnost operátorů, která z počátečního stavu vyrobí stav splňující cíl.

Mini příklad: operátor $Přesuň(x,A,B)$ má předpoklady $Na(x,A)$ a $Volne(B)$, efekty přidat $Na(x,B)$ a smazat $Na(x,A)$. Dopředné plánování začíná v počátečním stavu a zkouší použitelné akce. Zpětné plánování začíná cílem a hledá akce, které by cíl mohly splnit, přičemž jejich předpoklady se stanou novými podcíli.

- **Dopředné:** jednoduché simulování akcí, ale může zkoušet mnoho irelevantních kroků.
- **Zpětné:** soustředí se na cíl, ale musí řešit, zda vybrané akce jsou vzájemně slučitelné.
- **Kdy použít:** dopředné je přirozené, když je málo použitelných akcí; zpětné, když cíl silně omezuje relevantní akce.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: V klasickém plánování je stav světa reprezentován množinou atomů, které platí; atomy, které ve stavu uvedeny nejsou, se považují za nepravdivé podle předpokladu uzavřeného světa. Pravdivost některých atomů se mění akcemi, těm se říká fluenty; jiné atomy jsou stálé, těm se říká rigid atoms. Schéma akce neboli operátor popisuje možnost agenta bez dosažení konkrétních objektů. Operátor je trojice název, předpoklady a důsledky. Název obsahuje také parametry operátoru, tedy proměnné používané v předpokladech a důsledcích. Předpoklady musí platit ve stavu, aby operátor šel použít. Důsledky jsou literály, typicky fluenty, které budou platit nebo přestanou platit po provedení operátoru. Akce je plně instanciováný operátor, kde jsou za proměnné dosazeny konstanty. Doménový model je popis operátorů. Plánovací problém je trojice (O, s_0, g) , kde O je doménový model, s_0 počáteční stav a g cílová podmínka. Plán je posloupnost akcí, která z s_0 dovede do stavu splňujícího g .

2 Dopředné a zpětné plánování: Plánování se realizuje prohledáváním stavového prostoru, který bývá velmi velký. Dopředné neboli progression planning postupuje od výchozího stavu k cíli: v každém stavu zkouší použitelné akce, aplikuje jejich efekty a hledá stav splňující cíl. Je přímočarý, ale prohledávací algoritmus potřebuje dobrou heuristiku, protože může generovat mnoho akcí irelevantních pro cíl. Zpětné neboli regression planning postupuje od cílové podmínky zpět: vybírá akce relevantní pro aktuální cíl a nahrazuje cíl jejich předpoklady doplněnými o zbylé požadavky cíle. Akce je relevantní pro cíl g právě tehdy, když k cíli přispívá a její efekty nejsou s g v konfliktu. Zpětné plánování má často menší větvení než dopředné, protože uvažuje jen relevantní akce, ale pracuje s množinami stavů nebo cílových podmínek místo individuálních stavů, a proto se pro něj hůře hledají dobré heuristiky.

3 Detaily a příklad: Pro operátor $Přesuň(x,A,B)$ mohou být předpoklady $Na(x,A)$ a $Volne(B)$, efekty přidat $Na(x,B)$ a smazat $Na(x,A)$. Výsledek aplikace akce na stav vznikne tak, že se odstraní negativní efekty a přidají pozitivní efekty. Regresní množina pro cíl a akci říká, jaké podmínky musely platit před akcí, aby po ní cíl platil; zahrnuje předpoklady akce a části cíle, které akce sama nezajišťuje. Proti obvyčejnému prohledávání není stav atomická černá skříňka, ale množina faktů, a akce mají logickou strukturu. Díky tomu lze využívat předpoklady, efekty, relevanci a regresi místo slepé expanze všech následníků.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaké části má operátor?
- Co je stav v klasickém plánování?
- Jak se liší dopředné a zpětné plánování?
- Proč je plánování vhodné pro faktorizované stavy?

Pozor (častá past): Neodpovídej jen obecně "hledání cesty". U plánování musí zaznít preconditions,

effects a cíl jako podmínka na fakta.

6.7 Markovské rozhodovací procesy (MDP)

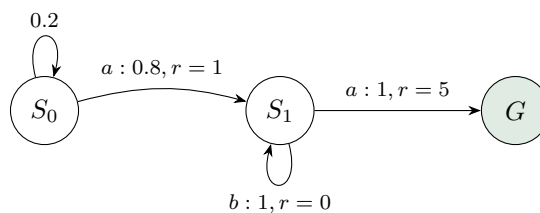
priorita: vysoká cíl: zelená (2 body)

Markovské rozhodovací procesy (MDP)

- formulace problému (výpočet užitku, strategie)
- Bellmanova rovnice
- iterace hodnot, iterace strategií
- POMDP (základní definice)

Učivo (jak na to):

MDP je plánování, kde akce nejsou jisté. Agent volí akci podle stavu, dostane odměnu a pravděpodobnostně přejde do dalšího stavu. Hledáme strategii s nejvyšším očekávaným diskontovaným užitekem.



MDP je čtveřice stavů, akcí, přechodových pravděpodobností $P(s' | s, a)$ a odměn $R(s, a, s')$, často s diskontem γ . Strategie $\pi(s)$ říká, jakou akci zvolit ve stavu s . Užitečnost stavu je očekávaný součet budoucích diskontovaných odměn.

Bellmanova rovnice optimality:

$$U(s) = \max_a \sum_{s'} P(s' | s, a) (R(s, a, s') + \gamma U(s')).$$

Iterace hodnot: opakovaně aktualizuj $U(s)$ Bellmanovou rovnicí a nakonec z ní vyčti strategii.

Iterace strategií: střídá vyhodnocení současné strategie a její zlepšení výběrem lepší akce.

POMDP: stav není přímo pozorovaný, agent udržuje belief, tedy distribuci přes možné stavy.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Markovský rozhodovací proces řeší sekvenční rozhodování v plně pozorovatelném stochastickém prostředí. Obsahuje stavy, akce dostupné ve stavu $A(s)$, Markovův přechodový model $P(s' | s, a)$ a odměnu $R(s)$, kterou agent získá ve stavu s a která je krátkodobá. Užitečnost nekonečné trajektorie je diskontovaný součet odměn

$$U([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots,$$

kde discount factor $\gamma \in (0, 1)$ vyjadřuje, jak moc si ceníme budoucích odměn. Pro odměny omezené absolutní hodnotou $R_{\max}$ je užitek konečný, protože $|U| \leq R_{\max}/(1 - \gamma)$. Řešením MDP je policy neboli strategie, funkce $\pi : S \rightarrow A$, která doporučuje akci pro každý stav. Pevná posloupnost akcí nestačí, protože v nedeterministickém prostředí můžeme skončit v jiném stavu, než jsme čekali. Optimální strategie maximalizuje střední hodnotu užitku a nezávisí na

počátečním stavu; pro další rozhodnutí je relevantní aktuální stav.

2 Bellman a algoritmy: Opravdový užitek stavu je jeho okamžitá odměna plus diskontovaná střední hodnota užítku následného stavu. Proto platí Bellmanova rovnice optimality

$$U(s) = R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) U(s').$$

Pokud je odměna vázaná na přechod, používá se ekvivalentní tvar se členem $R(s, a, s')$ uvnitř sumy. Předpokládáme, že známe odměny R i přechodový model P , a chceme najít užitky U nebo optimální strategii π . Soustava Bellmanových rovnic není lineární kvůli maximu, proto se často řeší aproximací. Value iteration nastaví počáteční odhady užiteků, například nuly, opakovaně aplikuje Bellmanovu aktualizaci a iteruje do konvergence. Policy iteration využívá fakt, že strategie se často ustálí dříve než přesné užitky: střídá policy evaluation, kde pro pevnou strategii zmizí maximum a řeší se lineární Bellmanovy rovnice, a policy improvement, kde se v každém stavu vybere lepší akce podle současných užiteků. Gaussova eliminace pro policy evaluation má složitost $O(n^3)$, u velkých stavových prostorů lze i evaluation dělat iterací hodnot. Policy iteration doběhne, protože strategií je konečně mnoho a každé zlepšení volí lepší strategii.

3 Detaily a POMDP: Policy loss je vzdálenost užítku dané strategie od optimálního užítku. U value iteration cílem nemusí být dokonale přesný užitek, protože výsledná strategie může být stabilní dříve. POMDP přidává částečnou pozorovatelnost: stále máme přechodový model $P(s' | s, a)$, akce $A(s)$ a odměny $R(s)$, ale navíc sensorový model pro vztah mezi stavem a pozorováním. Agent místo reálných stavů používá belief states, tedy distribuce přes možné stavy. Modely přechodů a sensorů lze reprezentovat dynamickou Bayesovskou sítí; přidáním rozhodování a užiteků vzniká dynamická rozhodovací síť. Řešení lze chápat jako generování stromu, kde se střídají vrstvy akcí, pozorování a belief stavů, a používá se algoritmus podobný expectimaxu.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je strategie v MDP?
- Kde v Bellmanově rovnici vystupuje nejistota?
- Jak se liší iterace hodnot a iterace strategií?
- Co je belief stav u POMDP?

Pozor (častá past): Nepleť MDP s HMM: v MDP agent volí akce a optimalizuje odměny, v HMM hlavně odhaduje skrytý stav z pozorování.

6.8 Hry a teorie her

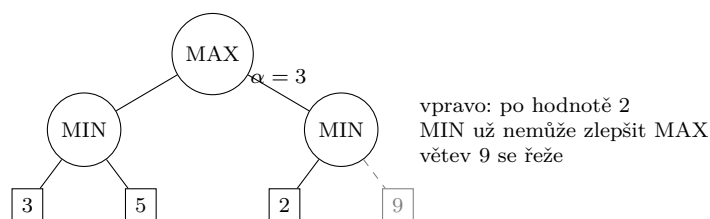
priorita: vysoká cíl: zelená (2 body)

Hry a teorie her

- algoritmy Minimax a alfa-beta prořezávání
- základy teorie her (věžňovo dilemma, Nashovo ekvilibrium)
- mechanism design (typy aukcí)

Učivo (jak na to):

Ve hrách neoptimalizuješ proti přírodě, ale proti jinému agentovi. Minimax předpokládá, že soupeř hraje proti nám co nejlépe. Alfa-beta jen šetří práci, výsledek minimaxu nemění.



Minimax počítá hodnotu hry s nulovým součtem: v uzlu MAX bere maximum z potomků, v uzlu MIN minimum. Nashovo ekvilibrium je profil strategií, kde se žádnému hráči nevyplatí jednostranně změnit strategii.

Alfa-beta: drží α jako nejlepší zatím zajištěnou hodnotu pro MAX a β jako nejlepší zatím zajištěnou hodnotu pro MIN. Když je jasné, že větev nemůže ovlivnit rozhodnutí předka, neprohledává se.

Věznovo dilema: každý má individuální motivaci zradit, i když oboustranná spolupráce je společně lepší. Ukazuje rozdíl mezi individuální racionalitou a kolektivním výsledkem.

Aukce: anglická je vzestupná, holandská sestupná, první cena znamená platbu vlastní nabídky, druhá cena typicky motivuje nabídnout skutečnou hodnotu.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Minimax řeší hry dvou hráčů s nulovým součtem a úplnou informací. Prochází strom hry, v listech nebo ve zvolené hloubce použije ohodnocení herní situace a potom hodnoty propaguje nahoru: v uzlu hráče MAX bere maximum ze synů, v uzlu hráče MIN minimum. Omezení hloubky je důležité u her s velkým větvením a vyžaduje vhodnou evaluační funkci. Výsledkem je optimální tah za předpokladu, že protihráč hraje optimálně, tedy podle nejhoršího scénáře. V teorii her v normálním tvaru hru tvoří hráči, jejich akce a užítkové funkce; hráči táhnou zároveň a každý provede jednu akci. Strategie může být čistá, tedy konkrétní akce, nebo smíšená, tedy pravděpodobnostní rozdělení přes akce. Strategický profil je množina strategií všech hráčů.

2 Vlastnosti, ekvilibria a aukce: Alfa-beta prořezávání zrychluje minimax tím, že nevyhodnocuje větve, které už nemohou změnit rozhodnutí. Při rekurzi se předávají meze α a β : α je nejlepší zatím zajištěná hodnota pro MAX, β nejlepší zatím zajištěná hodnota pro MIN. Pokud při hledání minima dostaneme prozatímní hodnotu menší než už dosažené maximum ve vyšší úrovni, další podvětvě nemohou výsledek zlepšit pro MAX a řežou se; analogicky pro opačný případ. Alfa-beta nemění výsledek minimaxu a její účinnost závisí na pořadí tahů. Strategie s_1 dominuje s_2 , pokud hráči dává větší užitek než s_2 bez ohledu na strategie ostatních; dominantní strategie dominuje všechny ostatní strategie. Nashovo ekvilibrium je strategický profil, v němž by žádný hráč jednostranně nezměnil strategii, kdyby znal strategie ostatních. Profil s Pareto dominuje s' , pokud přechodem z s' na s si nikdo nepohorší a alespoň jeden hráč si polepší. Profil je Pareto optimální, pokud ho žádný jiný profil Pareto nedominuje. Existuje také korelované ekvilibrium. Dobrý aukční mechanismus maximalizuje výnos pro prodejce, vybere jako vítěze agenta, který si věc nejvíce cení, a dává zájemcům dominantní strategii. Anglická aukce je vzestupná: začíná na $b_{\min}$ a cena roste o krok d ; dominantní strategie je přihazovat, dokud cena nepřekročí vlastní hodnotu. Holandská aukce začíná vysokou cenou a snižuje ji, dokud ji někdo nepřijme. Obálková aukce první ceny nechá vyhrát nejvyšší nabídku a vítěz platí svou nabídku; dominantní strategie obecně neexistuje, při odhadu maxima ostatních nabídek b_0 dává smysl nabídnout $b_0 + \varepsilon$, pokud je to pod vlastní hodnotou. Obálková aukce druhé ceny, Vickreyho aukce, nechá vyhrát nejvyšší nabídku, ale vítěz platí druhou nejvyšší nabídku; dominantní strategie je nabídnout vlastní hodnotu. VCG mechanismus zobecňuje tento princip pro obecnější víceparametrické aukce.

3 Příklady a detaily: Věznovo dilema má dva hráče, kteří mohou vypovídat, tedy defect, nebo odmítnout vypovídat, tedy cooperate. Pokud oba vypovídají, oba ztrácejí málo, například

užitek -5 . Pokud jeden vypovídá a druhý ne, vypovídající neztratí nic, užitek 0 , a druhý ztratí hodně, užitek -10 . Pokud oba odmítnou, oba ztrácejí velmi málo, například -1 . Vypovídat je čistá dominantní strategie, takže profil, kde oba vypovídají, je Nashovo ekvilibrium: nikomu se nevyplatí jednostranně změnit strategii. Zároveň je kolektivně horší než oboustranné odmítnutí, což ukazuje rozdíl mezi individuální racionalitou a Pareto lepším výsledkem. U alfa-beta je klíčová intuice, že často hledáme maximum z minim nebo minimum z maxim; jakmile je dílčí minimum už horší než alternativa z vyšší úrovně, jeho přesná hodnota není pro rozhodnutí relevantní.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak se počítá hodnota v MAX a MIN uzlu?
- Proč alfa-beta nemění výsledek minimaxu?
- Co znamená Nashovo ekvilibrium?
- Jaký je rozdíl mezi aukcí první a druhé ceny?

Pozor (častá past): Neříkej, že alfa-beta je jiná strategie hraní. Je to optimalizace výpočtu stejného minimaxového rozhodnutí.

6.9 Strojové učení (přehledově)

priorita: vysoká cíl: zelená (2 body)

Strojové učení

- základní druhy učení (s učitelem, bez učitele, zpětnovazební)
- rozhodovací stromy (definice, konstrukce)
- regrese, SVM (základní principy)
- Bayesovské učení, EM algoritmus
- zpětnovazební učení
- pasivní učení (definice, metody ADP a TD)
- aktivní učení (definice, explorace vs. exploitace, Q-učení, SARSA)

Učivo (jak na to):

Tady se nečeká detail celé specializace Strojové učení, ale mapa pojmů v kontextu UI. Bezpečná odpověď vždy řekne typ úlohy, co se učí, z jakých dat a jak se pozná dobrý výsledek.

Učení s učitelem má dvojice vstup plus správný výstup a řeší klasifikaci nebo regresi. Učení bez učitele hledá strukturu bez cílových štítků. Zpětnovazební učení má agenta, akce, odměny a učí strategii z interakce s prostředím.

Rozhodovací strom: vnitřní uzel testuje atribut, hrana odpovídá výsledku testu a list dává predikci. Konstrukce vybírá test, který nejlépe rozdělí data, typicky podle informačního zisku nebo Gini nečistoty, a zastavuje nebo prořezává kvůli přeučení.

Regrese a SVM: regrese predikuje spojitou hodnotu, typicky minimalizuje chybu. SVM hledá oddělující nadrovinu s maximálním marginem; pro neseparabilní data používá měkký margin a pro nelineární hranice jádra.

Bayesovské učení a EM: Bayesovské učení kombinuje prior a věrohodnost na posterior $P(h \mid data)$. EM řeší skryté proměnné střídáním E-kroku, který dopočítá očekávání skrytých přiřazení, a M-kroku, který zlepší parametry.

Zpětnovazební učení: pasivní učení hodnotí pevně danou strategii. ADP odhaduje model přechodů a odměn a pak řeší MDP. TD aktualizuje hodnoty z rozdílu mezi po sobě jdoucími odhady. Aktivní učení současně volí akce: musí vyvažovat explorační a exploitační. Q-učení je off-policy aktualizace hodnot akcí, SARSA je on-policy aktualizace podle skutečně provedeného přechodu (s, a, r, s', a') .

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Definice: Učení bez učitele hledá vzory ve vstupu bez explicitní zpětné vazby nebo štítků. Zpětnovazební učení učí agenta z odměn a trestů podle chování v prostředí. Učení s učitelem učí funkci mapující vstupy na výstupy. Rozhodovací strom přijímá vektor hodnot atributů a vrací výslednou hodnotu; vnitřní vrchol odpovídá testu jednoho atributu, hrany výsledkům testu a list hodnotě, kterou klasifikační funkce vrací. Strom lze z tabulky příkladů postavit hladovou metodou rozdělení a panuj, k dělení se používá entropie, a výhodou stromu je čitelné odůvodnění konkrétního rozhodnutí. Regrese je úloha učení s učitelem, kde položce x přiřazujeme vhodnou funkční hodnotu y . SVM stojí na lineární separaci: pokud lze třídy oddělit nadrovinou, volí maximum margin separator, tedy nadrovinu nejdále od datových příkladů. Pokud lineární nadrovina nestačí, použije kernel function pro mapování do vícedimenzionálního prostoru, kde už příklady mohou být oddělitelné. SVM je neparametrická metoda; příklady nejbližší separátoru jsou support vectors a fakticky definují separátor.

2 Bayesovské učení, EM a pasivní RL: V bayesovském učení máme množinu hypotéz a podle pozorování upravujeme jejich pravděpodobnosti. Maximálně věrohodná hypotéza je hypotéza s největší pravděpodobností naměřených dat:

$$\arg \max_h p(\text{data} | h) \quad \text{nebo} \quad \arg \max_h p_h(\text{data}).$$

U parametrizované hypotézy hledáme parametr maximalizující likelihood, často přes nulovou derivaci logaritmu pravděpodobnosti; tomu se říká maximum-likelihood parameter learning. Posterior hypotézy je

$$p(h | \text{data}) = \frac{p(\text{data} | h)p(h)}{\sum_{h'} p(\text{data} | h')p(h')}.$$

Pokud mají hypotézy stejné apriorní pravděpodobnosti, stačí likelihoody normalizovat. Bayesovsky optimální predikce pro výstup z je

$$p(z | \text{data}) = \sum_h p(z | h)p(h | \text{data}).$$

EM algoritmus se používá pro maximum-likelihood odhad, když ho nelze jednoduše spočítat kvůli skrytým proměnným. Začne náhodnými parametry a iteruje do konvergence: E-step dopočte střední hodnoty skrytých proměnných, M-step upraví parametry tak, aby maximalizovaly likelihood modelu. Pro směšový model se skrytým shlukem C a pozorováním x_j je odpovědnost

$$r_{jk} = P(C = k | x_j; \theta) = \frac{P(C = k; \theta)P(x_j | C = k; \theta)}{\sum_{\ell} P(C = \ell; \theta)P(x_j | C = \ell; \theta)}.$$

V Bayesovské síti s podmíněně nezávislými pozorovanými potomky platí například $P(x_j | C = k) = P(\text{Color} = c_j | C = k)P(\text{Holes} = h_j | C = k)$. M-krok pak používá očekávané četnosti:

$$P(C = k) \leftarrow \frac{1}{N} \sum_j r_{jk}, \quad P(A = a | C = k) \leftarrow \frac{\sum_{j: A_j=a} r_{jk}}{\sum_j r_{jk}}.$$

Příklad: z výšek bez pohlaví chceme odhadnout průměr mužů μ_0 a žen μ_1 . Pohlaví je skrytá proměnná; v E-kroku spočteme pravděpodobnosti, že daná výška patří ženě nebo muži, a v M-kroku přepočteme μ_0, μ_1 jako vážené průměry výšek. Parametry mohou vyjít prohozené, protože EM je učení bez učitele a nezajišťuje interpretaci tříd. Pasivní zpětnovazební učení uvažuje agenta v MDP s pevně danou strategií π ; učí se, jak je strategie dobrá, tedy utility

funkci, a nezná přechodový model ani reward function. Přímý odhad utility používá běhy, traces, a pro každý stav průměruje reward-to-go, tedy odměnu současného a budoucích stavů, případně diskontovanou faktorem γ . Je to podobné učení s učitelem se stavem na vstupu a užitek na výstupu, ale konverguje pomalu, protože ignoruje Bellmanovy vazby a hledá v příliš velkém prostoru hypotéz. ADP se učí přechodový model a odměny z pozorovaných přechodů a utility potom počítá z Bellmanových rovnic, například iterací hodnot; tím vynucuje konzistenci odhadů užiteků. TD learning model nepotřebuje a při přechodu $s \rightarrow s'$ aktualizuje

$$U^\pi(s) \leftarrow U^\pi(s) + \alpha(R(s) + \gamma U^\pi(s') - U^\pi(s)),$$

kde α je learning rate. TD je model-free a konverguje pomaleji než ADP.

3 Aktivní RL a detaily: Aktivní zpětnovazební učení učí strategii i utility funkci. Pouhé použití ADP může dát hladového agenta, který opakuje zažité akce a nezkouší nové. Optimální akce podle dosavadního modelu může vést k suboptimálním výsledkům, protože naučený model se liší od reálného prostředí, akce nejsou jen zdrojem odměn, ale i dat pro učení, a zlepšováním modelu mohou růst budoucí odměny. Je nutné vyvažovat exploration a exploitation. Pure exploration nevyužívá naučené znalosti, pure exploitation riskuje opakování špatně naučených vzorů. Základní intuice je prozkoumávat více na začátku a později víc využívat znalosti. Jednoduchá exploration policy zvolí náhodnou akci s pravděpodobností $1/t$ a jinak hladovou akci; konverguje k optimální strategii, ale může být velmi pomalá. Rozumnější je přidávat váhu akcím, které agent ještě nezkusil. Q-learning používá $Q(s, a)$, hodnotu provedení akce a ve stavu s , a platí $U(s) = \max_a Q(s, a)$. Modelová rovnice je

$$Q(s, a) = R(s) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q(s', a'),$$

ale bezmodelové Q-učení nepotřebuje znát P a při přechodu ze s akcí a do s' aktualizuje

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s) + \gamma \max_{a'} Q(s', a') - Q(s, a)).$$

SARSA, state-action-reward-state-action, je varianta Q-učení s aktualizací

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s) + \gamma Q(s', a') - Q(s, a)),$$

která se aplikuje po pěti s, a, r, s', a' , tedy po volbě další reálné akce a' . Pro hladového agenta, který vždy volí největší Q , jsou SARSA a základní Q-learning stejné. Rozdíl je u průzkumného agenta: SARSA je on-policy a bere v úvahu skutečně zvolenou akci, zatímco Q-learning je off-policy, učí se optimální strategii, i když se během učení používá jiná strategie. V aktivním učení je výhodnější učit se Q -hodnoty než samotné utility, protože k volbě nejlepší akce podle utility bychom potřebovali přechodové pravděpodobnosti, zatímco $Q(s, a)$ přímo říká, jak užitečné je danou akci provést.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak rozlišíš klasifikaci a regresi?
- Co je list a co vnitřní uzel v rozhodovacím stromu?
- Co se děje v E-kroku a M-kroku EM algoritmu?
- Jaký je rozdíl mezi Q-učením a SARSA?

Pozor (častá past): Nezajížděj mimo oficiální přehled: žádné hluboké učení, NLP ani robotika. Tady stačí základní principy uvedených metod.

7 Část IV: Specializace - Strojové učení

Toto je **nejdůležitější** okruh: vlastní zaměření a vázaná podmínka (ze specializace potřebuješ aspoň 7/12).
Cíl: každé téma aspoň *zeleně* (2 body), nejdůležitější modely (lineární/logistická regrese, SVM, stromy, k-means, PCA) i *modře*.

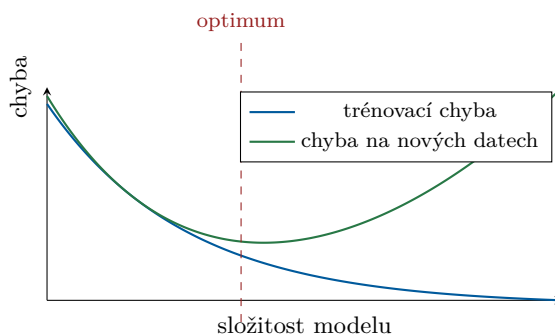
7.1 Učení s učitelem (základní pojmy, evaluace, regularizace)

priorita: vysoká **cíl:** modrá (3 body) - rámec pro celý okruh

Učení s učitelem: klasifikace a regrese jako základní typy úloh; standardní evaluační metriky; ohodnocení modelu (testovací data, křížová validace, maximální věrohodnost); přeučení a regularizace (generalizační chyba, včasné zastavení trénování, L2 a L1 regularizace); prokletí dimenzionality.

Učivo (jak na to):

Z trénovacích dvojic (vstup, správný výstup) hledáme funkci, která dobře předpoví výstup i pro **nová** data. Nejde o to namemorovat trénink, ale *generalizovat*.



Příliš jednoduchý model = podtrénování; příliš složitý = přeučení. Hledáme dno zelené křivky.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: *Klasifikace* přiřazuje položce x vhodnou třídu nebo množinu tříd; základní případ je binární klasifikace do tříd pozitivní/negativní. *Regrese* přiřazuje položce x reálnou funkční hodnotu y . U binární klasifikace: TP je správně pozitivní, FP falešně pozitivní, FN falešně negativní, TN správně negativní.

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}.$$

U regrese je základní metrika $MSE = \frac{1}{N} \sum_{i=1}^N (y_i - t_i)^2$, kde y_i je predikce a t_i skutečná hodnota. Často se používají také

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - t_i)^2}, \quad MAE = \frac{1}{N} \sum_{i=1}^N |y_i - t_i|.$$

RMSE trestá velké chyby silněji, MAE je robustnější vůči odlehlým hodnotám.

2 + k tomu: Pro pravděpodobnostní klasifikaci se používá

$$NLL = - \sum_{i=1}^N \log p_i(t_i) = - \sum_{i=1}^N \log p(t_i | x_i),$$

kde p_i je rozdělení tříd vrácené modelem pro x_i a t_i je skutečná třída. *Confusion matrix* má řádky a sloupce označené skutečnými a predikovanými třídami (pořadí je nutné uvést); buňka obsahuje počet testovacích příkladů s danou kombinací a diagonála obsahuje správné predikce. Accuracy klame při nevyvážených třídách, proto se často sledují precision, recall a F-míra.

$$F_\beta = \frac{(\beta^2 + 1)PR}{\beta^2 P + R} = \frac{TP + \beta^2 TP}{TP + FP + \beta^2(TP + FN)}.$$

ROC křivka vzniká posouváním prahu pro pozitivní výsledek: na ose x je $FPR = \frac{FP}{FP+TN}$, na ose y je $TPR = \frac{TP}{TP+FN} = R$. Náhodný klasifikátor má diagonálu a AUC 0,5, dokonalý klasifikátor má bod vlevo nahoře (0, 1) a AUC 1. Senzitivita je TPR, specificita je $TNR = \frac{TN}{TN+FP}$; obecně mají metriky typu rate ve jmenovateli všechny instance se stejnou skutečnou třídou jako číselník. Pokud je v testu P_0 skutečně pozitivních a N_0 skutečně negativních instancí, pak

$$TP = TPR \cdot P_0, \quad FN = (1 - TPR)P_0, \quad FP = FPR \cdot N_0, \quad TN = (1 - FPR)N_0.$$

Z pevné precision q plyne $q = TP/(TP + FP)$, tedy $FP = TP(1 - q)/q$; to dovoluje dopočítat bod ROC i matici konfuze z nevyvážených dat.

3 Bonus: Testovací data měří generalizaci na datech, která model neviděl, a nesmí se použít při trénování ani ladění hyperparametrů. Při ladění se používá train-dev-test split: parametry se učí na trénovací sadě, hyperparametry na vývojové/validační sadě a finální odhad výkonu na testu. Křížová validace rozdělí data na n dílů, n -krát trénuje na $n - 1$ dílech a vyhodnocuje na zbylém dílu; výsledkem je průměr skóre. Při maximální věrohodnosti fixujeme trénovací dvojice a hledáme váhy

$$L(w) = p_{\text{model}}(t \mid X; w) = \prod_i p_{\text{model}}(t_i \mid x_i; w), \quad w_{\text{MLE}} = \arg \max_w L(w).$$

U klasifikace řádku x model popisuje $p_{\text{model}}(t \mid x; w)$. *Underfitting* znamená příliš slabý model. *Overfitting* znamená špatnou generalizaci, protože se model příliš přizpůsobil tréninku. Generalizační chyba je výkon na nových datech, typicky test nebo křížová validace. Pokud se validační/testovací chyba po počátečním přibližování začne vzdalovat od trénovací chyby, model se přeučuje; pokud se ani nezačne zlepšovat, je model slabý nebo je třeba trénovat déle. *Regularizace* přičítá k loss penalizaci vah, omezuje efektivní kapacitu a preferuje menší změny modelu:

$$L^1 : \lambda \sum_j |w_j|, \quad L^2 : \lambda \sum_j w_j^2 = \lambda \|w\|^2 \quad \text{nebo} \quad \frac{\lambda}{2} \|w\|^2.$$

Váha biasu se obvykle neregularizuje. Proti přeučení pomáhá L2 regularizace, early stopping při růstu validační chyby a rozumný learning rate nebo schedule. *Prokletí dimenzionality*: s počtem příznaků roste exponenciálně množství dat potřebná k dobré generalizaci, protože chceme vzorky pro mnoho kombinací příznaků. Redukce dimenzí: *filter* vybírá příznaky nezávisle na modelu statisticky (konzistence, korelace se třídami a nekorelace mezi příznaky, vzdálenost tříd, vzájemná informace), *wrapper* trénuje model a vybírá příznaky podle jeho výkonu. U tabulárních dat se nejprve rozliší číselné a kategorické příznaky. Kategorické příznaky se pro lineární modely, kNN, SVM a neuronové modely typicky převádějí one-hot kódováním. Číselné příznaky se škálují standardizací nebo normalizací, pokud je model citlivý na měřítko, například kNN, lineární model, logistická regrese, SVM nebo PCA; stromové modely a náhodné lesy škálování obvykle nevyžadují. Chybějící číselné hodnoty lze doplnit průměrem nebo mediánem, kategorické modelem nebo zvláštní kategorií. Pro tabulární regresi bývají silným konzervativním modelem náhodné lesy nebo gradient boosting; lineární regrese je vhodná jako jednoduchý a interpretovatelný baseline.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak poznám přeučení z trénovací a testovací chyby?
- Jaký je rozdíl mezi L1 a L2 regularizací?
- K čemu je k-fold křížová validace a proč je lepší než jedno rozdělení?
- Co znamená precision vs recall?

Pozor (častá past): Model se nikdy nelaď (ani nevybírám) podle testovacích dat - na to je validační sada. Vysoká accuracy u nevyvážených tříd klame (proto F1).

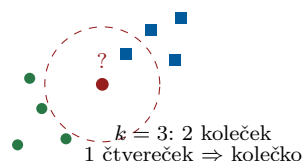
7.2 Učení založené na příkladech - k nejbližších sousedů (kNN)

priorita: střední cíl: zelená (2 body)

Učení založené na příkladech: algoritmus k-nejbližších sousedů.

Učivo (jak na to):

„Řekni mi, kdo jsou tvoji sousedé, a já ti řeknu, kdo jsi.“ Nový bod dostane třídu podle většiny svých k nejbližších trénovacích bodů. Nic se netrénuje - jen se pamatují data.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: kNN je instance-based a líné učení: při trénování si uloží data, při dotazu najde k nejbližších trénovacích bodů podle zvolené vzdálenosti. V klasifikaci s uniformními vahami hlasují sousedé a vyhrává nejčastější třída, remízu lze rozhodnout náhodně. V regresi se predikuje průměr nebo vážený průměr targetů sousedů.

2 + k tomu: Obecná predikce s vahami je

$$t = \sum_i \frac{w_i}{\sum_j w_j} t_i.$$

Pro regresi je to vážený průměr hodnot t_i . Pro klasifikaci lze t_i chápat jako one-hot vektor třídy, vzorcem dostaneme odhad pravděpodobností tříd a zvolíme největší složku. Váhy mohou být *uniform* (všichni sousedé stejně), *inverse* (nepřímo úměrné vzdálenosti) nebo *softmax* ze záporných vzdáleností.

3 Bonus: Vzdálenost se často počítá pomocí p -normy

$$\|x\|_p = \sqrt[p]{\sum_i |x_i|^p}, \quad d(x, y) = \|x - y\|_p,$$

typicky pro konečné $p \in \{1, 2, 3\}$; pro $p = \infty$ je $\|x\|_\infty = \max_i |x_i|$. Malé k je citlivé na šum a vede k přeučení, velké k hranice vyhlazuje a může podtrénovat; k se volí validací. Příznaky je nutné škálovat, jinak dominuje rozměr s velkým rozsahem. Nevýhody: drahá predikce přes mnoho uložených bodů a silné prokletí dimenzionality.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co se stane při $k = 1$ a co při $k =$ počet všech bodů?
- Proč je nutné příznaky normalizovat?

Pozor (častá past): kNN nic „nenatrénuje“ - náročný je až dotaz. A bez škálování dominuje příznak s velkým rozsahem.

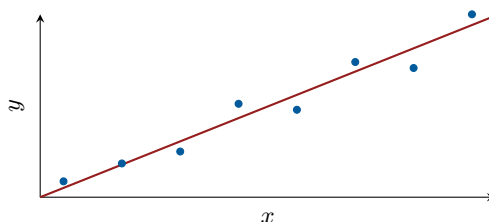
7.3 Lineární regrese

priorita: vysoká cíl: modrá (3 body)

Lineární regrese: analytické řešení metodou nejmenších čtverců; trénování pomocí stochastic gradient descent.

Učivo (jak na to):

Proložíme daty **přímku** (obecně nadrovinu) tak, aby součet *čtverců* svislých odchylek byl co nejmenší.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: Lineární regrese predikuje $y = x^T w$; bias reprezentujeme přidáním sloupce jedniček do X a další složky do w . Metoda nejmenších čtverců minimalizuje

$$E(w) = \frac{1}{2} \sum_i (x_i^T w - t_i)^2 = \frac{1}{2} \|Xw - t\|^2.$$

2 + k tomu: Derivace podle váhy w_j je

$$\frac{\partial E}{\partial w_j} = \sum_i x_{ij} (x_i^T w - t_i).$$

V optimu chceme nulovou derivaci pro všechna j , tedy

$$X^T(Xw - t) = 0, \quad X^T X w = X^T t.$$

Je-li $X^T X$ invertibilní, normální rovnice dávají analytické řešení

$$w = (X^T X)^{-1} X^T t.$$

Když je matice singulární nebo příliš velká, používá se pseudoinverze, regularizace nebo iterativní optimalizace.

3 Bonus: Gradient descent minimalizuje chybovou funkci $E(w)$ iterací

$$w \leftarrow w - \alpha \nabla_w E(w),$$

kde α je learning rate. Batch gradient descent počítá gradient ze všech dat, stochastic/online z jednoho náhodného řádku a minibatch SGD z B náhodných nezávislých řádků. Minibatch algoritmus: inicializuj w náhodně nebo nulou, opakuj do konvergence nebo zastavení, vyber minibatch indexů $\mathbb{B}$ uniformně náhodně nebo po epochách přes náhodné minibatche, aktualizuj

$$w \leftarrow w - \alpha \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} ((x_i^T w - t_i) x_i) - \alpha \lambda w.$$

Tato aktualizace odpovídá loss

$$E(w) = \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} \frac{1}{2} (x_i^T w - t_i)^2 + \frac{\lambda}{2} \|w\|^2.$$

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak zní normální rovnice a kdy ji nelze (rozumně) použít?
- Co řídí parametr α u SGD?

Pozor (častá past): Nejmenší čtverce trestají odlehle body kvadraticky - jsou citlivé na outliery.

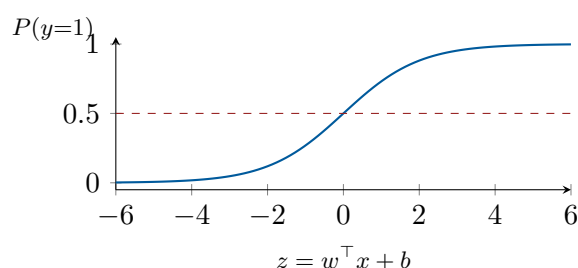
7.4 Logistická regrese

priorita: vysoká cíl: modrá (3 body)

Logistická regrese: binární klasifikace (sigmoid, metody trénování); klasifikace do více tříd (softmax, metody trénování).

Učivo (jak na to):

Navzdory názvu je to **klasifikátor**: lineární skóre $w^\top x$ protlačíme funkcí sigmoid, aby vyšla *pravděpodobnost* třídy mezi 0 a 1.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: Logistická regrese je klasifikátor. Pro binární třídy $\{0, 1\}$ má sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

lineární složku $\bar{y}(x; w) = x^T w$ a predikci

$$y(x; w) = \sigma(\bar{y}(x; w)) = \sigma(x^T w) = P(t = 1 \mid x; w).$$

Třídu získáme prahem nebo zaokrouhlením; bias je další váha po přidání konstantního příznaku.

2 + k tomu: Pro target $t \in \{0, 1\}$ platí

$$p(t \mid x; w) = \sigma(x^T w)^t (1 - \sigma(x^T w))^{1-t}.$$

Trénuje se maximalizací věrohodnosti, ekvivalentně minimalizací NLL/cross-entropy, obvykle minibatch SGD. Algoritmus: inicializuj w , opakuj do konvergence, vyber minibatch $\mathbb{B}$ a aktualizuj

$$w \leftarrow w - \alpha \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} (\sigma(x_i^T w) - t_i) x_i - \alpha \lambda w.$$

Gradient má stejný tvar jako u lineární regrese, $(y(x_i) - t_i)x_i$, jen y je sigmoid.

3 Bonus: Vícetřídní logistická regrese klasifikuje do jedné z K tříd. Parametry jsou váhová matice W (sloupce odpovídají třídám, řádky příznakům) a jeden bias pro každou třídu. Softmax je

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad \text{softmax}(z + c)_i = \text{softmax}(z)_i.$$

Predikce je

$$p(C_i \mid x; W) = y(x; W)_i = \text{softmax}(\bar{y}(x; W))_i = \text{softmax}(x^T W)_i,$$

což normalizuje skóre na pravděpodobnosti se součtem 1. Sigmoid je speciální případ softmaxu, například

$$\sigma(x) = \text{softmax}([x, 0])_0 = \frac{e^x}{e^x + e^0}.$$

Lineární softmax má konvexní rozhodovací oblasti. Minibatch SGD pro targety $\{0, \dots, K-1\}$ používá one-hot vektor 1_{t_i} :

$$W \leftarrow W - \alpha \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} \left((\text{softmax}(x_i^T W) - 1_{t_i}) x_i^T \right)^T - \alpha \lambda W.$$

Binární gradient je $(y(x) - t)x$, vícetřídní gradient je $((y(x) - 1_t)x^T)^T$.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Proč se nepoužívá MSE, ale cross-entropy?
- Čím se liší sigmoid a softmax?

Pozor (častá past): Logistická regrese je klasifikace, ne regrese. Hranice je lineární (na nelineární potřebuješ příznaky navíc nebo jádro).

7.5 Rozhodovací stromy

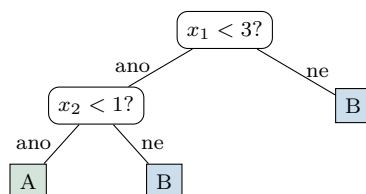
priorita: vysoká

cíl: zelená (2 body)

Rozhodovací stromy: algoritmus učení a kritéria větvení; prořezávání.

Učivo (jak na to):

Posloupnost otázek typu „je příznak $>$ práh?“. Každá otázka rozdělí data; chceme otázky, po kterých jsou výsledné skupiny co *nejčistší* (jedna třída).



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

❶ **Minimum:** Rozhodovací strom má ve vnitřním vrcholu test příznaku a hodnotu/práh rozhraní, hrany odpovídají výsledku testu a list predikuje podle dat v listu. Každému vrcholu $\mathcal{T}$ náleží množina indexů $I_{\mathcal{T}}$. Při regresi list predikuje průměr targetů, při klasifikaci většinou třídu.

❷ **+ k tomu:** Kritéria větvení měří nehomogenitu vrcholu. Pro regresi se používá squared error

$$c_{SE}(\mathcal{T}) = \sum_{i \in I_{\mathcal{T}}} (t_i - \hat{t}_{\mathcal{T}})^2,$$

kde $\hat{t}_{\mathcal{T}}$ je průměr targetů ve vrcholu. Pro klasifikaci je Gini impurity

$$c_{Gini}(\mathcal{T}) = |I_{\mathcal{T}}| \sum_k p_{\mathcal{T}}(k)(1 - p_{\mathcal{T}}(k)),$$

což je očekávaná chybovost náhodné klasifikace podle rozdělení tříd ve vrcholu. V binární klasifikaci odpovídá skoro squared error ztrátě. Entropické kritérium je

$$c_{entropy}(\mathcal{T}) = |I_{\mathcal{T}}| H(p_{\mathcal{T}}) = -|I_{\mathcal{T}}| \sum_k p_{\mathcal{T}}(k) \log p_{\mathcal{T}}(k),$$

ve kterém vynecháme třídy s nulovou pravděpodobností; lze ho odvodit z NLL. Dělení volíme tak, aby co nejvíc snížilo kritérium, tedy maximalizovalo čistotu.

❸ **Bonus:** CART začíná jedním listem s trénovacími daty. Vybraný list rozdělí na vnitřní vrchol a dva nové listy. Split volí tak, aby $c_{\mathcal{T}_L} + c_{\mathcal{T}_R} - c_{\mathcal{T}}$ bylo co nejmenší, tedy aby pokles kritéria byl co největší. Lze omezit maximální hloubku, minimální počet dat v děleném listu a maximální počet listů. Obvykle se dělí do hloubky; při omezeném počtu listů je vhodné dělit listy s největším zlepšením. Minimalizací dělicího kritéria minimalizujeme nejlepší dosažitelnou hodnotu loss pro daný strom. Prořezávání zjednodušuje strom a omezuje přeučení; může být top-down nebo bottom-up. Reduced error pruning je bottom-up: na validačních datech měříme accuracy a zkoušíme rušit větvení ve vnitřních vrcholech; pokud přesnost neklesne, větve nahradíme listem s většinou třídou.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co měří entropie / Gini a co je informační zisk?
- Proč a jak se strom prořezává?

Pozor (častá past): Hluboký strom se skoro vždy přeučí. Stromy také snadno preferují příznaky s mnoha hodnotami.

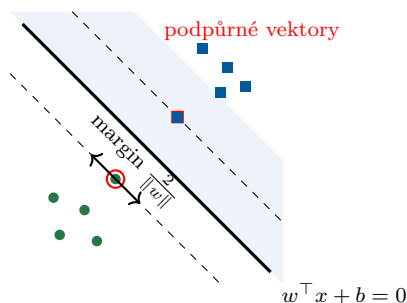
7.6 Metoda podpůrných vektorů (SVM)

priorita: vysoká cíl: zelená (2 body)

Metoda podpůrných vektorů: klasifikátor pro lineárně separabilní třídy; klasifikátor pro lineárně neseperabilní třídy; jádrové funkce; klasifikace do více tříd.

Učivo (jak na to):

Mezi dvě třídy chceme proložit **co nejširší ulici** (pruh bez bodů). Hranice vede středem. Širší ulice = lepší generalizace.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: SVM je binární klasifikátor pro třídy $\{-1, 1\}$. Pro hard-margin hledá dělicí nadrovinu s maximálním marginem. Uvažujeme mapování příznaků φ a skóre

$$y(x) = \varphi(x)^T w + b.$$

Protože w, b lze škálovat, zvolí se kanonické škálování, ve kterém nejbližší body splňují $t_i y(x_i) = 1$. Hard-margin úloha je

$$\arg \min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{za podmínky} \quad t_i y(x_i) \geq 1 \text{ pro všechna } i.$$

Klasifikace je podle znaménka skóre a šířka marginu je $\frac{2}{\|w\|}$.

2 + k tomu: SVM je jádrová metoda a v duální formulaci neparametrický model: místo explicitních vah pracuje s koeficienty u trénovacích dat, protože optimální w je lineární kombinace trénovacích bodů. Omezená optimalizace se řeší přes Lagrangian a KKT podmínky, tradičně s multiplikátory a_i . Důležitá podmínka je

$$a_i(t_i y(x_i) - 1) = 0.$$

Proto se při predikci použijí jen body s $a_i > 0$, tedy body na okraji marginu, nazývané podpůrné vektory. Predikce v duálu je

$$y(x) = \sum_i a_i t_i K(x, x_i) + b.$$

Soft-margin SVM řeší lineárně neseparabilní data: zavede slack proměnné $\xi_i \geq 0$ a nahradí podmínku

$$t_i y(x_i) \geq 1 \quad \text{podmínkou} \quad t_i y(x_i) \geq 1 - \xi_i.$$

Podpůrné vektory jsou pak body na marginu i body s kladným ξ_i , tedy uvnitř marginu nebo chybně klasifikované.

3 Bonus: Pro porušení marginu platí

$$\xi_i = \max(0, 1 - t_i y(x_i)),$$

což je hinge loss. Soft-margin lze zapsat jako

$$\arg \min_{w, b} C \sum_i \mathcal{L}_{\text{hinge}}(t_i, y(x_i)) + \frac{1}{2} \|w\|^2.$$

Parametr C má opačný efekt než běžná regularizační konstanta: větší C silněji trestá chyby a znamená slabší regularizaci. V duálu se proti hard-margin navíc objeví horní omezení $a_i \leq C$. K trénování se používá sequential minimal optimization, která postupně zlepšuje Lagrangian přes vybrané koeficienty a_i, a_j a bias. Kernel je funkce

$$K(x, z) = \varphi(x)^T \varphi(z),$$

tedy skalární součin transformovaných příznaků bez explicitního výpočtu φ . Například pro příznaky řádů 0 až 3 může platit $\varphi(x)^T \varphi(z) = 1 + x^T z + (x^T z)^2 + (x^T z)^3$. Homogenní polynomiální kernel stupně d je

$$K(x, z) = (\gamma x^T z)^d,$$

nehomogenní polynomiální kernel stupně nejvýše d je

$$K(x, z) = (\gamma x^T z + 1)^d,$$

a Gaussian RBF kernel je

$$K(x, z) = e^{-\gamma \|x - z\|^2}.$$

RBF vrací 1 pro stejné vektory, téměř 0 pro velmi vzdálené vektory, něco mezi pro blízké vektory; γ určuje, co už je daleko, a lze ho chápat jako rozšířenou podobnost kNN. Kernely se používají tam, kde chceme data transformovat do vyšší dimenze, aby byla lineárně separabilní. Více tříd se řeší kombinací binárních klasifikátorů. One-versus-rest natrénuje K klasifikátorů "třída proti zbytku" a vybírá největší skóre nebo pravděpodobnost; u SVM se také používá, ale skóre může vyžadovat kalibraci. One-versus-one trénuje klasifikátor pro každou dvojici tříd; klasifikátorů je více, ale každý vidí méně dat, a predikce probíhá většinovým hlasováním.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co jsou podpůrné vektory a co dělá parametr C ?
- Proč jádro umožní oddělit neseparabilní data?

Pozor (častá past): Velké C není „lepší“ - úzký margin se přeučí. Jádro nemění SVM, jen skalární součin.

7.7 Kombinace více modelů (ensemble)

priorita: vysoká cíl: zelená (2 body)

Kombinace více modelů: bagging a boosting; metoda náhodných lesů.

Učivo (jak na to):

Více slabších modelů dohromady často předčí jeden velký - „moudrost davu“. Liší se tím, *jak* je spojíme.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: Ensembles kombinují více modelů. *Bagging* trénuje modely nezávisle, typicky na bootstrap vzorcích, tedy náhodných výběrech trénovacích dat s opakováním. Predikce se kombinuje průměrováním u regrese nebo hlasováním u klasifikace. *Boosting* trénuje modely sekvenčně a každý další model se snaží opravit chyby předchozích.

2 + k tomu: AdaBoost postupně trénuje slabé klasifikátory, například mělké rozhodovací stromy nebo pařezy, na vážených trénovacích datech. V každé iteraci zvýší váhy špatně kla-

sifikovaných příkladů, aby na ně další klasifikátor více mířil. Slabým klasifikátorům zároveň přiřazuje váhy podle jejich accuracy, takže přesnější modely mají větší vliv na finální hlasování.

3 Bonus: Náhodný les je bagging rozhodovacích stromů plus náhodnost v příznacích. Pro každý strom vybere náhodný bootstrap vzorek dat a v každém vrcholu uvažuje jen náhodnou podmnožinu příznaků pro dělení. Při regresi průměruje hodnoty stromů, při klasifikaci hlasuje. Jeden strom se rychle trénuje a je dobře interpretovatelný jako série rozhodnutí, zatímco les je obvykle přesnější a lépe generalizuje, ale predikce je méně přímo interpretovatelná.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Bagging vs boosting: paralelně/sekvenčně, snižuje rozptyl/bias?
- Čím se náhodný les liší od prostého baggingu stromů?

Pozor (častá past): Boosting nezaměňuj s baggingem: boosting je sekvenční a typicky cílí hlavně na bias, bagging paralelní na rozptyl.

7.8 Statistické testy

priorita: střední cíl: zelená (2 body)

Statistické testy: studentův t-test (jednovýběrový a dvouvýběrový); chí-kvadrát test (test dobré shody).

Učivo (jak na to):

Ptáme se: je pozorovaný rozdíl **reálný**, nebo jen náhoda? Spočteme testovou statistiku a porovnáme s tím, co by dělala náhoda za platnosti *nulové hypotézy*.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: Statistický test pracuje s nulovou hypotézou H_0 a testovou statistikou, jejíž rozdělení známe za platnosti H_0 . Hypotézu zamítáme, když vypočtená hodnota spadne do kritického oboru, ekvivalentně když je p-hodnota menší než zvolená hladina α .

2 + k tomu: Jednovýběrový Studentův t-test dává intervalový odhad a test pro střední hodnotu, když neznáme rozptyl σ^2 . Pro test H_0 s hodnotou θ_0 používá statistiku

$$T = \frac{\overline{X_n} - \theta_0}{\bar{\sigma}/\sqrt{n}},$$

kteřá má za platnosti H_0 t-rozdělení s $n - 1$ stupni volnosti. Intervalový odhad pro parametr θ se spočte z bodového odhadu $\hat{\theta}$, například výběrového průměru pro μ , a

$$\delta = \frac{\bar{\sigma}}{\sqrt{n}} \cdot \Psi_{n-1}^{-1}(1 - \alpha/2), \quad [\hat{\theta} - \delta, \hat{\theta} + \delta],$$

kde Ψ_{n-1}^{-1} je kvantil t-rozdělení. Stupňů volnosti je $n - 1$, protože při fixním průměru lze z $n - 1$ hodnot dopočítat poslední.

3 Bonus: Dvouvýběrový t-test porovnává střední hodnoty dvou nezávisle samplovaných populací, například $H_0 : \mu_X = \mu_Y$, i s různými počty vzorků a při přibližné normalitě nebo dost velkých výběrech. Welchova varianta pro neznámé a obecně různé rozptyly používá

$$T = \frac{\overline{X_n} - \overline{Y_m}}{se}, \quad se = \sqrt{\frac{\bar{\sigma}_X^2}{n} + \frac{\bar{\sigma}_Y^2}{m}},$$

případně interval přes $\delta = se \cdot \Psi_{\nu}^{-1}(1 - \alpha/2)$ s Welchovými stupni volnosti ν . Při předpokladu stejných rozptylů se používá pooled variance a t-rozdělení s $n + m - 2$ stupni volnosti. Párový t-test se používá pro související dvojice dat: vezmeme rozdíly $R_i = Y_i - X_i$ a použijeme jednovýběrový test na rozdíly. Pearsonův chí-kvadrát test dobré shody porovnává pozorované četnosti O_i a očekávané četnosti E_i kategorií. Testová statistika je

$$\chi^2 = \sum_i \frac{(E_i - O_i)^2}{E_i}.$$

Typická úloha je spravedlivost kostky; pro šest kategorií použijeme χ^2 rozdělení s pěti stupni volnosti. Obecně H_0 říká, že pozorované četnosti odpovídají očekávaným, a pokud H_0 platí a neodhaduje se žádný parametr z dat, má statistika χ^2 rozdělení s $n - 1$ stupni volnosti pro n kategorií. H_0 zamítáme, když je statistika větší než kritická hodnota.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je p-hodnota a kdy zamítám H_0 ?
- Kdy t-test a kdy chí-kvadrát?

Pozor (častá past): Malá p-hodnota neznamena „velký efekt“, jen „těžko vysvětlitelný náhodou“.

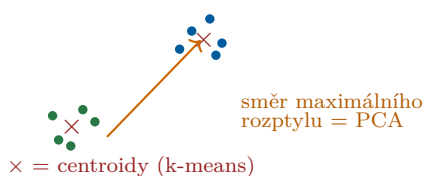
7.9 Učení bez učitele (shlukování, PCA)

priorita: vysoká cíl: zelená (2 body)

Učení bez učitele: shlukování (algoritmus k-means); hierarchické shlukování; redukce dimenzionality (analýza hlavních komponent, PCA).

Učivo (jak na to):

Nemáme správné odpovědi, jen data. Hledáme v nich *strukturu*: přirozené skupiny (shlukování) nebo směry, kde se data nejvíc mění (PCA).



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

1 Minimum: Učení bez učitele nemá targety. k-means dělí body $x_1, \dots, x_N$ do K shluků s centroidy $\mu_1, \dots, \mu_K$ a optimalizuje

$$J = \sum_{i=1}^N \sum_{k=1}^K z_{i,k} \|x_i - \mu_k\|^2,$$

kde $z_{i,k}$ indikuje přiřazení bodu x_i do shluku k . PCA redukuje dimenzi hledáním směrů největšího rozptýlu; získáme je ze SVD centrované datové matice nebo jako vlastní vektory kovarianční matice.

2 + k tomu: Algoritmus k-means: na vstupu jsou body a počet shluků K . Inicializuj středy náhodně ze vstupních bodů nebo pomocí kmeans++. Pak opakuj do konvergence: každý bod

přiřadí shluku s nejbližším středem a každý střed aktualizuj jako průměr bodů přiřazených do daného shluku. kmeans++ vybere první střed náhodně a každý další z nepoužitých bodů s pravděpodobností úměrnou druhé mocnině vzdálenosti k nejbližšímu už zvolenému středu. Přiřazovací krok loss nezvyšuje a aktualizace centroidů loss nezvyšuje, proto k-means konverguje k lokálnímu optimu. Použití: shlukování dat bez targetů, například podobné barvy pixelů nebo skupiny zákazníků.

3 Bonus: Hierarchické shlukování může být aglomerativní (spojování menších shluků) nebo divisivní (dělení velkých shluků). Aglomerativní postup začíná s každým bodem jako singletonem a opakovaně slučuje nejpodobnější shluky, dokud nedostane cílový počet shluků. Podobnost shluků lze měřit vzdáleností centroidů, vzdálenostmi bodů, průměry vzdáleností a podobně. Divisivní DIANA začne jedním shlukem se všemi body, najde bod nejvíce odlišný od ostatních a oddělí ho, poté do nového shluku postupně přidává další body, pokud jsou mu blíže než původnímu shluku. V každé fázi se jeden shluk rozdělí na dva, dokud není každý bod sám. Výsledkem je dendrogram, který lze zaříznout na vhodném počtu shluků. PDDP používá princip PCA a dělí shluky projekcí na hlavní komponenty. Hierarchické algoritmy jsou většinou hladové, takže negarantují optimum. SVD: pro matici $X \in \mathbb{R}^{m \times n}$ s hodnotami r platí

$$X = U\Sigma V^T,$$

kde $U \in \mathbb{R}^{m \times m}$ a $V \in \mathbb{R}^{n \times n}$ jsou ortonormální a $\Sigma \in \mathbb{R}^{m \times n}$ je diagonální s nezápornými sestupně seřazenými singulárními čísly. Rozklad lze psát

$$X = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T.$$

Protože existuje nejvýše r nenulových singulárních čísel, lze použít redukovanou SVD s Σ rozměru $r \times r$, U rozměru $m \times r$ a V^T rozměru $r \times n$ bez ztráty informace. Pro aproximaci hodnot $k < r$ necháme jen k největších singulárních čísel a ostatní zahodíme. PCA dimenze M z datové matice X : spočti průměr řádků $\bar{x}$, proved SVD matice $X - \bar{x}$, nech prvních M řádků V^T (ekvivalentně M sloupců V) a nové komponenty získej jako $(X - \bar{x})V_M$, matici $N \times M$. Funguje to proto, že SVD centrovaných dat dává vlastní vektory kovarianční matice

$$\text{cov}(X) = \frac{1}{N}(X - \bar{x})^T(X - \bar{x}).$$

Vlastní vektory lze hledat power iteration: ulož $S = \text{cov}(X)$, pro každou komponentu začni náhodným vektorem, opakovaně ho násob zleva maticí S a normalizuj, dokud nekonverguje, a po nalezení komponenty odečti od S člen $\lambda_i v_i v_i^T$, kde λ_i je norma vektoru před poslední normalizací. PCA se používá k aproximaci matic, snížení šumu, redukci dimenze, extrakci příznaků, vizualizaci a v doporučovacích systémech; geometricky postupně hledá osy s největším rozptylem.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaké dva kroky k-means opakuje?
- Co maximalizují hlavní komponenty a jak souvisí s kovarianční maticí?
- Výhoda hierarchického shlukování oproti k-means?

Pozor (častá past): k-means hledá zhruba kulové, podobně velké shluky a vyžaduje předem k ; na protáhlé/různě husté shluky selhává.

8 Vzorové výpočty a kódové vzory (drily na 2. a 3. bod)

Vzorová řešení zkoušek skoro vždy chtějí ve 2. a 3. podotázce *provést* výpočet nebo napsat kód. Tady je u nejčastějších typů jeden krátký vzor postupu. Nauč se **postup**, ne konkrétní čísla; pak ho zopakuj na zkouškovém zadání.

8.1 Matematika: typové výpočty

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

Určitý integrál a extrém (analýza): $\int_0^1 (x^2 + 1) dx = [\frac{x^3}{3} + x]_0^1 = \frac{1}{3} + 1 = \frac{4}{3}$. Hledání minima $g(p) = p + \frac{1}{p}$ na $p > 0$: $g'(p) = 1 - \frac{1}{p^2} = 0 \Rightarrow p = 1$; $g''(p) = \frac{2}{p^3} > 0$, tedy minimum, $g(1) = 2$.

Vlastní čísla a diagonalizace $A = (\frac{2}{1} \frac{1}{2})$: $\det(A - \lambda I) = (2 - \lambda)^2 - 1 = 0 \Rightarrow \lambda \in \{1, 3\}$. Pro $\lambda = 3$ je vlastní vektor $(1, 1)$, pro $\lambda = 1$ je $(1, -1)$. Pak $A = S\Lambda S^{-1}$ s $S = (\frac{1}{1} \frac{1}{-1})$, $\Lambda = (\frac{3}{0} \frac{0}{1})$.

Gram-Schmidt $u_1 = (1, 1, 0)$, $u_2 = (1, 0, 1)$: $v_1 = u_1$; $w_2 = u_2 - \frac{\langle u_2 | v_1 \rangle}{\langle v_1 | v_1 \rangle} v_1 = (1, 0, 1) - \frac{1}{2}(1, 1, 0) = (\frac{1}{2}, -\frac{1}{2}, 1)$; pak znormuj v_1, w_2 .

Eulerova formule (rovinný graf): souvislý rovinný graf má $v - e + f = 2$; z toho pro jednoduchý graf bez trojúhelníků $e \leq 2v - 4$, obecně $e \leq 3v - 6$ (dosadíš a ověříš, zda graf může být rovinný).

8.2 Pravděpodobnost a statistika: typové výpočty

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

Bayes / E-krok (responsibility): model $P(C=1) = 0.6$, $P(\text{blue}|1) = 0.6$, $P(\text{no}|1) = 0.4$; $P(C=2) = 0.4$, $P(\text{blue}|2) = 0.4$, $P(\text{no}|2) = 0.6$. Pro pozorování (blue, no): $r_1 \propto 0.6 \cdot 0.6 \cdot 0.4 = 0.144$, $r_2 \propto 0.4 \cdot 0.4 \cdot 0.6 = 0.096$, takže $\gamma_1 = \frac{0.144}{0.144+0.096} = 0.6$ (M-krok pak váží data těmito γ).

Chí-kvadrát test dobré shody: pozorováno $(30, 10)$, očekáváno $(20, 20)$: $\chi^2 = \frac{(30-20)^2}{20} + \frac{(10-20)^2}{20} = 5 + 5 = 10$, $\text{df} = k - 1 = 1$; porovnej s kritickou hodnotou na hladině α .

Markovova nerovnost: pro $X \geq 0$ s $E[X] = 2$ je $P(X \geq 8) \leq 2/8 = 0.25$.

8.3 Strojové učení a UI: typové výpočty

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

Entropie a informační zisk (dělení stromu): uzel se 6 příklady, 3 ano / 3 ne $\Rightarrow H = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$ bit (Gini = $1 - 2 \cdot 0.25 = 0.5$). Dělení podle A : $A=0$: {3 ano} ($H=0$), $A=1$: {3 ne} ($H=0$), vážená entropie 0, *informační zisk* = $1 - 0 = 1$ (dokonalé dělení).

Matice záměn (TP= 40, FP= 10, FN= 20, TN= 30): $\text{accuracy} = \frac{40+30}{100} = 0.70$, $\text{precision} = \frac{40}{50} = 0.80$, $\text{recall} = \frac{40}{60} \approx 0.67$, $F_1 = \frac{2 \cdot 0.8 \cdot 0.67}{0.8+0.67} \approx 0.73$. Rekonstrukce z metrik: z recall a počtu pozitivních dopočítáš TP a FN, z precision pak FP.

k-means (1 iterace): body $A(1, 1), B(2, 1), C(4, 3), D(5, 4)$, centra $c_1=(1, 1), c_2=(5, 4)$. Přiřazení: $A, B \rightarrow c_1, C, D \rightarrow c_2$. Nová centra: $c_1 = \frac{A+B}{2} = (1.5, 1)$, $c_2 = \frac{C+D}{2} = (4.5, 3.5)$; opakuj do ustálení.

AC-3 (1 krok): $X, Y \in \{1, 2, 3\}$, podmínka $X < Y$. Hrana (X, Y) : pro $X=3$ není $Y>3 \Rightarrow$ smaž 3 z X , $X \in \{1, 2\}$. Hrana (Y, X) : pro $Y=1$ není $X<1 \Rightarrow$ smaž 1 z Y , $Y \in \{2, 3\}$.

8.4 Informatika: kódové vzory (C#)

Vzory ukazují kostru; na zkoušce piš v C#. Vlastní psaní kódu si pár krát vyzkoušej.

Čtení dat z binárního souboru (big-endian, práce s bajty):

```
// 16bit unsigned big-endian z bufferu na pozici i
int ReadU16BE(byte[] b, int i) => (b[i] << 8) | b[i + 1];
// 16bit signed: dvojkový doplněk
short ReadS16BE(byte[] b, int i) => (short)((b[i] << 8) | b[i + 1]);

class Reader {
    private byte[] data; private int attrCount;
    public Reader(BinaryFile f) {          // konstruktor načte hlavičku
        byte[] h = new byte[2]; f.readBytes(0, h, 2);
        attrCount = ReadU16BE(h, 0);
    }
    public int Attributes() => attrCount;
}
```

Producent-konzument se semaforey (synchronizace vláken):

```
SemaphoreSlim empty = new(capacity);    // volná místa v bufferu
SemaphoreSlim full = new(0);            // obsazená místa
object mtx = new();
Queue<int> buf = new();

void Produce(int x) {
    empty.Wait();                        // počkej na volné místo
    lock (mtx) { buf.Enqueue(x); }
    full.Release();                      // přibyla položka
}

int Consume() {
    full.Wait();                         // počkej na položku
    int x; lock (mtx) { x = buf.Dequeue(); }
    empty.Release();                     // uvolnilo se místo
    return x;
}
```

Návrhový vzor Visitor (oddělení operace od struktury):

```
interface IVisitor { void Visit(Circle c); void Visit(Square s); }
interface IShape { void Accept(IVisitor v); }
class Circle : IShape { public void Accept(IVisitor v) => v.Visit(this); }
class Square : IShape { public void Accept(IVisitor v) => v.Visit(this); }
// nová operace = nový IVisitor, beze změny tříd tvarů
```

9 Tahák: kostry odpovědí na 1 bod (sít proti nulám)

Tohle je tvoje pojistka: u *každého* z 28 témat aspoň jedna správná věta, aby z žádné přidělené otázky nebyla nula. Pro průchod pak stačí pár otázek dotáhnout na 2 body. Nauč se nejdřív tohle celé, pak teprve prohlubuj (zejména specializaci).

Část I: Společná matematika

- **Dif./int. počet:** $\lim a_n = L \iff \forall \varepsilon > 0 \exists n_0 : \forall n \geq n_0 |a_n - L| < \varepsilon$; spojitost $\lim_{x \rightarrow a} f(x) = f(a)$; $f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$.
- **Lin. algebra:** báze = lin. nezávislá generující množina, dimenze = její velikost; hodnota = dimenze sloupcového prostoru; vlastní vektor $Ax = \lambda x$.
- **Diskrétní mat.:** ekvivalence = reflexivní + symetrická + tranzitivní; částečné uspořádání = reflexivní + antisymetrické + tranzitivní; bijekce = prostá i na.
- **Teorie grafů:** strom = souvislý bez kružnic ($n - 1$ hran); bipartitní = hrany jen mezi dvěma částmi; $\chi(G) = \min.$ počet barev; Euler $v - e + f = 2$.
- **Pravděpodobnost:** prostor $(\Omega, \mathcal{F}, P)$; $P(A|B) = \frac{P(A \cap B)}{P(B)}$; $E[X] = \sum x_i p_i$, $\text{var}(X) = E[X^2] - E[X]^2$.
- **Logika:** tautologie = pravdivá vždy, splnitelná = aspoň v jednom modelu; CNF = konjunkce klauzulí; rezoluce do prázdné klauzule = spor.

Část II: Společná informatika

- **Automaty:** DFA $(Q, \Sigma, \delta, q_0, F)$; regulární = konečný automat/regex; bezkontextové = PDA/CFG; rekurzivně spočetné = Turingův stroj.
- **Algoritmy:** $f \in O(g) \iff \exists c > 0 : f \leq cg$ od jistého n ; P = poly čas; NP = poly ověřitelný certifikát; NP-úplný = v NP a NP-těžký.
- **Prog. jazyky (C#):** třída/rozhraní/dědičnost/poly-morfismus; hodnotové vs referenční typy; garbage collector; výjimky; tabulka virtuálních metod.
- **Architektura/OS:** čísla ve dvojkovém doplňku; privilegovaný režim + jádro; proces/vláknko a přepínání kontextu; virtuální paměť (stránka + offset $\rightarrow$ rámec).

Část III: Základy UI

- **Prohledávání:** úloha = počáteční stav, akce, test cíle, cena; přípustná heuristika $h \leq h^*$; konzistentní $h(n) \leq c + h(n')$.
- **CSP:** proměnné + domény + relace podmínek, výstup = úplné konzistentní přiřazení; AC-3 = hranová konzistence; MAC = backtracking + AC.
- **Logické uvažování:** CNF; DPLL; rezoluce; dopředné/zpětné řetězení (Hornovy klauzule).
- **Pravděp. uvažování:** Bayesovská síť = DAG + $P(X_i | \text{Parents}(X_i))$, faktorizace $\prod_i P(x_i | pa_i)$.
- **Reprezentace znalostí:** HMM = skryté X_t , pozorování E_t , přechod $P(X_t | X_{t-1})$, senzor $P(E_t | X_t)$; Viterbi = max. cesta.
- **Plánování:** stav + operátory (precond/efekty); dopředné a zpětné hledání plánu.
- **MDP:** $S, A, P(s' | s, a), R, \gamma$; strategie $\pi(s)$; Bellman $U(s) = \max_a \sum_{s'} P(s' | s, a)(R + \gamma U(s'))$.
- **Hry:** minimax + alfa-beta prořezávání; Nashovo ekvilibrium; věžňovo dilema.
- **ML přehledově:** s učitelem / bez učitele / zpětnova-zební; stromy, regrese, SVM; Q-učení.

Část IV: Strojové učení

- **Učení s učitelem:** MLE $\arg \max_h P(D | h)$; accuracy $(TP + TN)/N$, precision $TP/(TP + FP)$, recall $TP/(TP + FN)$, $F1 = 2PR/(P + R)$.
- **kNN:** třída podle většiny k nejbližších bodů; líné učení; nutno škálovat příznaky.
- **Lineární regrese:** $\hat{y} = w^\top x + b$, minimalizuj $MSE = \frac{1}{n} \sum_i (\hat{y}_i - t_i)^2$ neboli L2 chybu; normální rovnice při regulární $X^\top X$.
- **Logistická regrese:** $\sigma(w^\top x + b)$ = pravděpodobnost třídy, práh 0,5; softmax pro více tříd.
- **Rozhodovací stromy:** entropie $-\sum p_c \log p_c$, Gini $1 - \sum p_c^2$; informační zisk = pokles entropie; prořezávání proti přeučení.
- **SVM:** nadrovina $w^\top x + b = 0$ s max. marginem $2/\|w\|$; podpůrné vektory; měkký margin a jádro.
- **Ensemble:** bagging (paralelně, ↓rozptyl) vs boosting (sekvenčně, ↓bias); náhodný les.
- **Statistické testy:** $H_0; p < \alpha \Rightarrow$ zamítní; t-test (průměry), chí-kvadrát (četnosti).
- **Učení bez učitele:** k-means minimalizuje $\sum_j \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$; PCA = vlastní směry kovariance s max. rozptylem.